GOOGLE | ALL SOURCES | TOPIC 4 OF 10 | CORE TOPIC

Graphs and Search

25 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

21	4
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Graphs and Search | 25 questions | tap a title to jump to it

BFS AND DFS (15)

M 1. Count Islands in a Grid and Track the Count as Land Cells Are Added — **PracHub 2026-10-02**

M 2. Path Minimizing the Maximum Cell (Swim in Rising Water) — **1Point3Acres 2026-09-29**

M 3. Rotten Oranges / Multi-Source BFS (taxis) — **1Point3Acres 2026-09-08**

M 4. Count Clusters of 2D Points Within a Radius — **PracHub 2026-06-18**

M 5. Find shortest relationship path using BFS — **PracHub 2026-02-12**

M 6. Validate parent array forms a tree — **PracHub 2026-02-11**

M 7. Solve matrix groups and recipe inventory — **PracHub 2026-01-22**

M 8. Compute distance to nearest taxi in grid — **PracHub 2026-01-10**

M 9. Validate course catalog dependencies — **PracHub 2025-08-01**

M 10. Solve Grid and Counting Problems — **PracHub 2025-01-25**

M 11. Course Schedule — **Exponent no date**

M 12. Coin Change — **Exponent no date**

H 13. Infection Spread on a Grid (Cellular Automaton) — **PracHub 2026-06-14**

H 14. Edit distance — **Exponent no date**

H 15. Given two strings and two allowed operations, swapping positions of letters and converting all occurre... — **Exponent no date**

TOPOLOGICAL SORT AND CYCLES (4)

M 16. Template Variable Expansion with Follow-Ups — **1Point3Acres 2026-10-17**

M 17. Course Scheduling and Dependency Variants — **1Point3Acres 2026-10-04**

M 18. Determine whether two nodes are related — **PracHub 2026-01-22**

M 19. Shortest Directed Cycle Through a Node — **1Point3Acres no date**

UNION-FIND (3)

M 20. Number of Islands — **1Point3Acres 2026-09-30**

M 21. 2D Points Connected Components by Radius — **1Point3Acres 2026-08-04**

H 22. Friend Graph with Timeline (Friend / Unfriend Events) — **1Point3Acres 2026-09-29**

SHORTEST PATHS (3)

M 23. Train / Flight Schedule Reachability — **1Point3Acres 2026-10-08**

M 24. Shortest Path Nodes and Escaping a Pursuer — **1Point3Acres 2026-09-29**

M 25. Minimum Direction Violations in a Directed Graph — **1Point3Acres 2026-07-19**

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 305 | PracHub | Technical Screen | 1 of 25

Count Islands in a Grid and Track the Count as Land Cells Are Added

POSTED 2026-10-02 | 18 DAYS AGO | LAST 30 DAYS

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 104/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #305 Number of Islands II

LeetCode match: LC 305 Number of Islands II

Posted on PracHub: 2026-10-02 (updated 2026-10-03)

Original: https://prachub.com/coding-questions/count-islands-in-a-grid-and-track-the-count-as-land-cells-are-added

QUESTION

A map is a grid with `m` rows and `n` columns in which every cell is land (`1`) or water (`0`). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- `grid[r][c]` is `1` for land and `0` for water, where `r` is the row index and `c` is the column index.
- An addition `(r, c)` turns cell `(r, c)` into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length `len(additions) + 1`. Element `0` is the number of islands in the initial grid. For `i >= 1`, element `i` is the number of islands after the first `i` additions have been applied.

Constraints

- `1 <= m <= 1000`, `1 <= n <= 1000` and `m * n <= 200000`
- Every row of `grid` has exactly `n` entries, each `0` or `1`.
- `0 <= len(additions) <= 200000`
- Every addition `(r, c)` satisfies `0 <= r < m` and `0 <= c < n`. The same cell may appear more than once.
- Every count is at most `m * n`, so all values fit in a 32-bit signed integer.

Examples

QUESTION

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

APPROACH

Union-find over cells: start with initial islands, then each addition creates a component and unions with the four neighbors; track the component count. Ignore repeated cells. O((mn + k) alpha).

HOW TO THINK ABOUT IT

Re-running a flood fill after every addition costs O(mn) each time. Land only ever appears, so islands only merge: Union-Find fits exactly. A new land cell starts as a new island (count + 1) and every union with a neighbouring land cell that belongs to a different island removes one island (count - 1).

Basis: LeetCode 305, Number of Islands II. Report the count before any addition and after each; adding a cell that is already land changes nothing.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_land(r, c):
    grid[r][c] = 1
    return count_islands(grid)  # full flood fill
answers = [add_land(r, c) for r, c in additions]
```

TIME	SPACE
O(A * m * n)	O(m * n)

WHY IT IS SLOW

BRUTE FORCE

Each addition costs a pass over the whole grid, O(A * m * n). Union-Find merges the new cell with its land neighbours and updates the count in near-constant time.

OPTIMAL SOLUTION · Union-Find updated cell by cell

TIME	SPACE
O(m * n + A * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Land can also be removed (or turned back to water).

Union-Find cannot split sets. Either recompute the affected island with a local flood fill, or process all operations offline with a segment tree over time and a rollback Union-Find.

FOLLOW-UP 2 The grid is 10^9 x 10^9 with a few additions.

Keep the Union-Find on a dictionary keyed by coordinates, as the code does; memory depends on the number of land cells only, never on the grid size. **Cost:** O(a * alpha) for a additions and O(a) memory, independent of the grid size.

FOLLOW-UP 3 Report the size of the largest island after each addition.

Store a size per root in the Union-Find (union by size) and keep a running maximum, which only grows when land is added; each step stays near-constant time. Checked against recomputing the largest island from scratch. **Cost:** O(alpha) per addition with size tracking; the maximum is one running variable.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Islands in a Grid and Track the
# Count as Land Cells Are Added
#
# A map is a grid with m rows and n columns in which every
# cell is land (1) or water (0). An island is a maximal
# group of land cells connected horizontally or
# vertically. Cells that touch only at a corner are not
# connected. You are given the initial map and a sequence
# of additions, each of which turns one cell into land.
# Report the number of islands before any addition, and
# again after each addition. ### Function Signature python
# def island_counts(grid: list[list[int]], additions:
# list[tuple[int, int]]) -> list[int]: ### Rules -
# grid[r][c] is 1 for land and 0 for water, where r is the
# row index and c is the column index. - An addition (r,
# c) turns cell (r, c) into land. If that cell is already
# land, either initially or because of an earlier
# addition, the map does not change. - Additions are
# applied in the given order. Land never turns back into
# water.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Land can also be removed (or turned back to water).
# 2. The grid is 10^9 x 10^9 with a few additions.
# 3. Report the size of the largest island after each
# addition.
#
# INTERVIEW NOTES
# Re-running a flood fill after every addition costs O(mn)
# each time. Land only ever appears, so islands only
# merge: Union-Find fits exactly. A new land cell starts
# as a new island (count + 1) and every union with a
# neighbouring land cell that belongs to a different
# island removes one island (count - 1).
#
# BRUTE FORCE (say it first): Recount all islands after
# every addition
# Apply the addition and run a full flood-fill count
# again. Correct and the oracle; every addition costs a
# pass over the entire grid. Time O(A * m * n). Space O(m
# * n).


# BRUTE FORCE (say it first): Recount all islands after
# every addition | time O(A * m * n)
def _count_islands(grid):
    m, n = len(grid), len(grid[0]) if grid else 0
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in seen:
                count += 1
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    for nx, ny in (
                        (x + 1, y),
                        (x - 1, y),
                        (x, y + 1),
                        (x, y - 1),
                    ):
                        if (
                            0 <= nx < m
                            and 0 <= ny < n
                            and grid[nx][ny] == 1
                            and (nx, ny) not in seen
                        ):
                            seen.add((nx, ny))
                            stack.append((nx, ny))
    return count


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def island_counts_recount(grid, additions):
    g = [row[:] for row in grid]
    out = [_count_islands(g)]
    for r, c in additions:
        g[r][c] = 1
        out.append(_count_islands(g))
    return out


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
```

```
# already land; otherwise make it a new set (count + 1)
```

```
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
class Islands:
    def __init__(self):
        # land cell -> parent cell, and sizes of the roots
        self.parent, self.size = {}, {}
        self.count, self.largest = 0, 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add(self, r, c):
        if (r, c) in self.parent:
            return
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1
        self.largest = max(self.largest, 1)
        # join with every land neighbour
        for dr, dc in STEPS:
            nb = (r + dr, c + dc)
            if nb in self.parent:
                self.union((r, c), nb)

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
        # union by size: the small island joins the big
        # one
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        self.count -= 1
        self.largest = max(self.largest, self.size[a])


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
```

```
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
def island_counts(grid, additions):
    # counts before any addition, then after each one
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.count]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.count)
    return out


# FOLLOW-UP 3: Report the size of the largest island after
# each addition.
# Answer: Store a size per root in the Union-Find (union
# by size) and keep a running maximum, which only grows
# when land is added; each step stays near-constant time.
# Checked against recomputing the largest island from
# scratch.
def largest_after_each(grid, additions):
```

```
    # the follow-up: size of the biggest island each time
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.largest]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.largest)
    return out


if __name__ == "__main__":
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    # 2 islands; the adds join them into one; a repeat is
    # ignored
    assert island_counts(grid, adds) == [2, 2, 2, 1, 1]
    sizes = largest_after_each(grid, adds)
```

```
    assert sizes == [1, 2, 3, 5, 5]
    # corners do not connect cells
    assert island_counts([[1, 0], [0, 1]], []) == [2]
    # the grid may be huge: only land cells are stored
    big = Islands()
    big.add(10 ** 9, 10 ** 9)
    big.add(10 ** 9, 10 ** 9 + 1)
    assert big.count == 1 and big.largest == 2
```

FUNCTION AND TEST CASES

Signature

```
def island_counts(grid: list[list[int]], additions: list[l
ist[int]]) -> list[int]:
```

Sample call

```
island_counts([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]],
[])
```

Test cases

Input	Expected
`([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], [])`	`[3]`
`([[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0], [0, 1], [1, 2], [2, 1], [1, 1], [1, 1]])`	`[0, 1, 1, 2, 3, 1, 1]`

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

Site tags: graph, heap, algorithm, medium

MEDIUM | IN APP LC 778 | 1Point3Acres | onsite-coding | 2 of 25

Path Minimizing the Maximum Cell (Swim in Rising Water)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE ALL TIME 40 30 DAYS 33 PRIORITY 55/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: your app's interview approach (LeetCode #778)

LeetCode match: LC 778 Swim in Rising Water

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

QUESTION

A bottleneck-path family: minimize the maximum value encountered on a grid path, then generalize the same reasoning to a weighted graph whose edges carry safety values and an admissibility threshold.

Grid variant

Input is a grid of heights and start/end cells; movement is through four-neighbors.
Cost of a path is the maximum cell value along it.
Return the minimum possible path cost.

Weighted-graph safety variant

Input is a graph whose edges carry safety values, a start node, an end node, and a safety threshold.
Find the lowest safety value required by any valid start-to-end path while every traversed edge remains under the supplied threshold.
Return -1 when no admissible path exists.
Clarify whether the threshold comparison is strict or inclusive before coding.

Mouse-and-cat safest-path variant (L4 phone screen)

Warm-up: an N x N grid with a start S, a target T, and impassable water cells; with 4-directional moves, decide whether any path from S to T exists.
Main question: the grid also contains a cat at C. The mouse moves only on land, 4-directionally, from S to the cheese at T. Find a path that maximizes the minimum Manhattan distance to the cat over all cells on the path.
This flips the objective (maximize a minimum instead of minimize a maximum), but it is the same bottleneck-path structure once each cell's distance to the cat is known.

Follow-up: Network Delay Time variant

In a directed graph with edge weights, return the maximum shortest-path distance from a source to all nodes, or -1 if any node is unreachable.

QUESTION

EXAMPLES
Grid

```
[ 0  1  2  3  4 ]
[24 23 22 21  5 ]
[12 13 14 15 16 ]
[11 17 18 19 20 ]
[10  9  8  7  6 ]
```

Start=(0,0), End=(4,0) in (row, col) — the bottom-left cell with value 10. Answer: 16 (the path's max cell is 16).

APPROACH

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, not the sum. That cost can only grow as the path grows, so Dijkstra still works if you replace 'distance + weight' by max(distance, weight): the first time a cell leaves the heap its cost is final. The cat version flips the objective (maximise a minimum), which is the same shape with a max-heap and each cell scored by its distance to the cat.

Basis: LeetCode 778, Swim in Rising Water (bottleneck path), plus the 'safest path from a cat' flip: maximise the minimum Manhattan distance.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, start, end):
    for t in sorted(set(all_values(grid))):
        if bfs_reaches(grid, start, end, t):
            return t  # smallest limit
```

TIME	SPACE
O(V^2)	O(V)

WHY IT IS SLOW

One full BFS per distinct value, O(V^2). Dijkstra on the running maximum finds the answer in one pass, O(V log V).

OPTIMAL SOLUTION · Dijkstra on the running maximum

TIME	SPACE
O(V log V)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Maximise the minimum distance to a cat instead.

Same bottleneck shape with the objective flipped: score each cell by its Manhattan distance to the cat, run Dijkstra with a max-heap on the running minimum, and skip water. The same monotonicity argument holds (extending a path can only lower its minimum).

FOLLOW-UP 2 Why is Dijkstra correct for a max-along-path cost?

The cost max(path) never decreases when a path is extended, so a vertex's cheapest cost cannot be improved by going through a vertex that is popped later. That is the only property Dijkstra needs from its cost function (monotone, non-negative extension).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Path Minimizing the Maximum Cell (Swim in
# Rising Water)
#
# A bottleneck-path family: minimize the maximum value
# encountered on a grid path, then generalize the same
# reasoning to a weighted graph whose edges carry safety
# values and an admissibility threshold. Grid variant
# Input is a grid of heights and start/end cells; movement
# is through four-neighbors. Cost of a path is the maximum
# cell value along it. Return the minimum possible path
# cost. Weighted-graph safety variant Input is a graph
# whose edges carry safety values, a start node, an end
# node, and a safety threshold. Find the lowest safety
# value required by any valid start-to-end path while
# every traversed edge remains under the supplied
# threshold. Return -1 when no admissible path exists.
# Clarify whether the threshold comparison is strict or
# inclusive before coding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Maximise the minimum distance to a cat instead.
# 2. Why is Dijkstra correct for a max-along-path cost?
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, not the sum.
# That cost can only grow as the path grows, so Dijkstra
# still works if you replace 'distance + weight' by
# max(distance, weight): the first time a cell leaves the
# heap its cost is final. The cat version flips the
# objective (maximise a minimum), which is the same shape
# with a max-heap and each cell scored by its distance to
# the cat.
#
# BRUTE FORCE (say it first): Try every threshold and test
# reachability
# For each candidate value t in increasing order run a BFS
# using only cells <= t and stop at the first t that
# connects start to end. Correct, but it repeats a full
# search per value. Time O(V^2). Space O(V).


from collections import deque
import heapq
```

```
# BRUTE FORCE (say it first): Try every threshold and test
# reachability | time O(V^2)
def bottleneck_brute(g, start, end):
    n, m = len(g), len(g[0])
    for t in sorted({x for row in g for x in row}):
        if (
            g[start[0]][start[1]] > t
            or g[end[0]][end[1]] > t
        ):
            continue
        seen, q = {start}, deque([start])
        while q:
            r, c = q.popleft()
            if (r, c) == end:
                return t
            for a, b in (
                (r + 1, c),
                (r - 1, c),
                (r, c + 1),
                (r, c - 1),
            ):
                if (
                    0 <= a < n
                    and 0 <= b < m
                    and (a, b) not in seen
                    and g[a][b] <= t
                ):
                    seen.add((a, b))
                    q.append((a, b))
    return None


# HELPERS for the base question
INF = float("inf")


def around(r, c, n, m):
    # four-neighbour cells inside an n x m grid
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < n and 0 <= b < m:
            yield a, b


# BASE QUESTION (optimal): Dijkstra on the running maximum
```

```
# Idea: Heap entries are (cost so far, cell). Popping a
# cell fixes its cost; each neighbour gets max(cost,
# neighbour value). Stop when the end cell is popped. Skip
# stale heap entries.
def min_max_path(g, start, end):
    n, m = len(g), len(g[0])
    # best[cell] = smallest possible "highest cell so far"
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, (r, c) = heapq.heappop(heap)
        # skip outdated heap entries
        if cost > best[(r, c)]:
            continue
        if (r, c) == end:
            return cost
        for a, b in around(r, c, n, m):
            # path cost = highest cell so far or this cell
            nxt = max(cost, g[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, (a, b)))
    return None
```

```
# VARIANT: weighted graph with a safety threshold
def min_bottleneck_edge(graph, s, t, limit):
    # graph[u] = [(v, safety)]; every edge must be < limit
    best = {s: 0}
    heap = [(0, s)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:
            continue
        if u == t:
            return cost
        for v, w in graph.get(u, []):
            # edges at or above the limit are not allowed
            if w >= limit:
                continue
            nxt = max(cost, w)
            if nxt < best.get(v, INF):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    # no admissible path
```

```
    return -1


# RELATED: network delay (plain Dijkstra)
def network_delay(n, edges, src):
    # plain Dijkstra: largest shortest distance, or -1
    graph = {}
    for u, v, w in edges:
        graph.setdefault(u, []).append((v, w))
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    # unreachable nodes make the answer -1
    return max(dist.values()) if len(dist) == n else -1


# FOLLOW-UP 1: Maximise the minimum distance to a cat
# instead.
# Answer: Same bottleneck shape with the objective
# flipped: score each cell by its Manhattan distance to
# the cat, run Dijkstra with a max-heap on the running
# minimum, and skip water.
def safest_path_score(grid, start, target, cat):
    # maximise the smallest cat distance ('#' is water)
    n, m = len(grid), len(grid[0])

    def dist(r, c):
        return abs(r - cat[0]) + abs(c - cat[1])

    best = {start: dist(*start)}
    heap = [(-best[start], start)]
    while heap:
        neg, (r, c) = heapq.heappop(heap)
        if -neg < best[(r, c)]:
            continue
        if (r, c) == target:
            return -neg
        for a, b in around(r, c, n, m):
```

```
            if grid[a][b] == "#":
                continue
            # the path is only as safe as its closest cell
            score = min(-neg, dist(a, b))
            if score > best.get((a, b), -1):
                best[(a, b)] = score
                heapq.heappush(heap, (-score, (a, b)))
    return None


if __name__ == "__main__":
    g = [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5],
         [12, 13, 14, 15, 16], [11, 17, 18, 19, 20],
         [10, 9, 8, 7, 6]]
    assert min_max_path(g, (0, 0), (4, 0)) == 16
    # start equals end: just that cell
    assert min_max_path(g, (2, 2), (2, 2)) == 14
    gr = {0: [(1, 5), (2, 9)], 1: [(2, 3)]}
    assert min_bottleneck_edge(gr, 0, 2, 10) == 5
    assert min_bottleneck_edge(gr, 0, 2, 5) == -1
    cat_grid = ["...", ".#.", "..."]
    cat, a, b = (0, 2), (0, 0), (2, 2)
    score = safest_path_score(cat_grid, a, b, cat)
    assert score == 2
    chain = [(0, 1, 2), (1, 2, 3)]
    assert network_delay(3, chain, 0) == 5
    assert network_delay(3, [(0, 1, 2)], 0) == -1
```

NOTES FROM CANDIDATES

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter.

Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

In the mouse-and-cat variant, each cell's score is a direct Manhattan distance to a single cat, so it can be computed per cell; clarify whether water cells block only the mouse or also affect distance, and whether S or T themselves count toward the minimum.

PREPARATION

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

Site tags: grid, bfs, medium

MEDIUM | IN APP LC 994 | 1Point3Acres | onsite-coding, phone-screen | 3 of 25

Rotten Oranges / Multi-Source BFS (taxis)

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TYPE **ALL TIME 51.2** **30 DAYS 0** **PRIORITY 67/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Also in your app: LeetCode #994 Rotting Oranges

LeetCode match: LC 994 Rotting Oranges

Frequency: Low | Last asked: 2026-09-08 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/rotten-oranges-multi-source-bfs-taxis

QUESTION

EXAMPLES

```
Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges
    variant): return number of minutes until all
    reachable 1s become 2 (4).
```

Taxi variant: for each empty cell, distance to nearest taxi.

APPROACH

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

HOW TO THINK ABOUT IT

'Spread outward from several starting points at the same time and count the rounds' is multi-source BFS: put every source in the queue before you start, and process the queue one round at a time. Rounds are minutes. Always count the fresh ones first so that unreachable oranges are detected at the end.

Basis: LeetCode 994, Rotting Oranges, and the same multi-source BFS skeleton as LeetCode 542 and 286.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    minutes = 0
    while True:
        fresh = fresh_next_to_rotten(grid)
        if not fresh:
            break
        for r, c in fresh:
            grid[r][c] = 2
        minutes += 1
    return -1 if any_fresh(grid) else minutes
```

TIME	SPACE
O(minutes * m * n)	O(m * n)

WHY IT IS SLOW

Each minute scans the entire grid, O(minutes * m * n). A queue of the newly rotten oranges touches each cell only once, O(m * n).

OPTIMAL SOLUTION · Multi-source BFS by minute

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Taxi variant: distance from every empty cell to the nearest taxi

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

FOLLOW-UP 2 Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

FOLLOW-UP 3 Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

FOLLOW-UP 4 A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Rotten Oranges / Multi-Source BFS (taxis)
#
# Examples: Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-
# oranges variant): return number of minutes until all
# reachable 1s become 2 (4). Taxi variant: for each empty
# cell, distance to nearest taxi.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Sources move or new taxis appear. How do you keep the
# distances fresh?
# 2. Why is multi-source BFS the same cost as single-
# source BFS?
# 3. Signal variant: a signal spreads from sources, how
# long until every target gets it?
# 4. A signal spreads from sources to targets: how long
# until every target is reached?
#
# INTERVIEW NOTES
# 'Spread outward from several starting points at the same
# time and count the rounds' is multi-source BFS: put
# every source in the queue before you start, and process
# the queue one round at a time. Rounds are minutes.
# Always count the fresh ones first so that unreachable
# oranges are detected at the end.
#
# BRUTE FORCE (say it first): Re-scan the whole grid every
# minute
# Each minute, find every fresh orange next to a rotten
# one and rot them all at once, until nothing changes.
# Easy to reason about and the oracle; each minute touches
# the entire grid. Time O(minutes * m * n). Space O(m *
# n).


from collections import deque


# BRUTE FORCE (say it first): Re-scan the whole grid every
# minute | time O(minutes * m * n)
def oranges_by_scanning(grid):
    g = [row[:] for row in grid]
    minutes = 0
    while True:
        rot = [
            (r, c)
            for r in range(len(g))
            for c in range(len(g[0]))
            if g[r][c] == 1
            and any(
                0 <= r + dr < len(g)
                and 0 <= c + dc < len(g[0])
                and g[r + dr][c + dc] == 2
                for dr, dc in (
                    (1, 0),
                    (-1, 0),
                    (0, 1),
                    (0, -1),
                )
            )
        ]
        if not rot:
            break
        for r, c in rot:
            g[r][c] = 2
        minutes += 1
    return -1 if any(1 in row for row in g) else minutes


# HELPERS for the base question
def multi_source_bfs(rows, cols, sources, open_cell):
    # distance from the nearest source to every cell
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c),
                       (r, c + 1), (r, c - 1)):
            inside = 0 <= nr < rows and 0 <= nc < cols
            if inside and (nr, nc) not in dist:
                if open_cell(nr, nc):
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
    return dist


# BASE QUESTION (optimal): Multi-source BFS by minute
# Idea: Push every rotten orange into the queue at minute
# 0 and expand level by level; each level is one minute.
# Count the fresh oranges and decrement as they rot, so
# the answer is -1 if any remain at the end.
def rotting_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    cells = [(r, c) for r in range(rows)
             for c in range(cols)]
    rotten = [x for x in cells if grid[x[0]][x[1]] == 2]
    fresh = [x for x in cells if grid[x[0]][x[1]] == 1]
    # rot spreads through any non-empty cell
    dist = multi_source_bfs(
        rows, cols, rotten, lambda r, c: grid[r][c] != 0)
    # a fresh orange that is never reached means -1
    if any(x not in dist for x in fresh):
        return -1
    return max((dist[x] for x in fresh), default=0)


# TAXI VARIANT: distance to the nearest taxi
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked
    rows, cols = len(grid), len(grid[0])
    taxis = [(r, c) for r in range(rows)
             for c in range(cols) if grid[r][c] == "T"]
    dist = multi_source_bfs(
        rows, cols, taxis, lambda r, c: grid[r][c] != "#")
    # None where no taxi can reach the cell
    return [[dist.get((r, c)) for c in range(cols)]
```

```
            for r in range(rows)]


# FOLLOW-UP 4: A signal spreads from sources to targets:
# how long until every target is reached?
# Answer: One multi-source BFS from the sources, then the
# answer is the largest distance over the targets (-1 if
# any target is unreachable, 0 if there are no targets).
# Per-source BFS would cost O(mn * sources).
def signal_time(rows, cols, sources, targets, blocked=()):
    # how long until every target hears the signal?
    dist = multi_source_bfs(
        rows, cols, list(sources),
        lambda r, c: (r, c) not in blocked)
    if any(t not in dist for t in targets):
        return -1
    # no targets means no waiting
    return max((dist[t] for t in targets), default=0)
```

```
if __name__ == "__main__":
    assert rotting_minutes([[2, 1, 1], [1, 1, 0],
                            [0, 1, 1]]) == 4
    # a fresh orange walled off by an empty cell
    assert rotting_minutes([[2, 0, 1]]) == -1
    assert rotting_minutes([[0, 2]]) == 0
    assert nearest_taxi(["T.#", "..."]) == [
        [0, 1, None], [1, 2, 3]]
    # signal from two sources to two targets
    assert signal_time(1, 5, [(0, 0), (0, 4)],
                       [(0, 1), (0, 2)]) == 2
    assert signal_time(1, 3, [(0, 0)], [(0, 2)],
                       blocked={(0, 1)}) == -1
    assert signal_time(1, 3, [(0, 0)], []) == 0
```

NOTES FROM CANDIDATES

BFS from all sources simultaneously is O(mn); per-source BFS is O(mn × #sources).

The moving-source follow-up trades exactness for amortized cost — common answer is periodic rebuild with bounded staleness.

For the signal variant, use the maximum source-distance over targets as the completion time; an unreachable target makes that candidate infeasible. Agree that no targets requires zero time. A correct baseline for destroying at most one blocker is to run multi-source BFS with no deletion, then once with each blocker made traversable, restoring it before the next trial. Minimize the completion time across feasible trials; return -1 if none is feasible. With B blockers on an R by C grid, this takes O((B + 1) * R * C) time and O(R * C) reusable auxiliary space. A BFS state that allows each path to break its own blocker does not enforce one shared destroyed blocker for all targets.

PREPARATION

Drill multi-source BFS from memory in under 10 min.

Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.

Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | PracHub | Technical Screen | 4 of 25

Count Clusters of 2D Points Within a Radius

POSTED 2026-06-18 | 124 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 40.7** **30 DAYS 0** **PRIORITY 90/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (inferred from overview)

Posted on PracHub: 2026-06-18 (updated 2026-07-01)

Original: https://prachub.com/coding-questions/count-clusters-of-2d-points-within-a-radius

QUESTION

Overview: This question tests graph traversal skills, specifically identifying connected components among a set of 2D points using distance-based adjacency. It assesses the ability to model a geometric problem as a graph and apply DFS, BFS, or Union-Find, a common evaluation of algorithmic thinking in coding interviews. It also probes awareness of efficiency trade-offs when scaling to large point sets.

APPROACH

Union-find over points with distance <= r; count roots. Use squared distance; spatial grid hashing for speed.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the distance is Euclidean, whether the radius is inclusive (a distance of at most r connects), whether points are integers, and the number of points.

BRUTE FORCE TO SAY OUT LOUD

Union-Find over all pairs of points, comparing every pair. O(n^2) distance checks.

OPTIMAL IDEA

Compare squared distances to r squared (no square roots, exact for integers). For many points hash them into grid cells of side r so each point only checks its own and the eight neighboring cells; use Union-Find to merge and count the components with distinct roots. About O(n) expected for evenly spread points.

DRY-RUN THESE CASES

No points, one point, two points exactly r apart (connected if inclusive), a chain of points each close to the next (one cluster), duplicate points.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Dynamic points, a very large radius, or reporting the largest cluster.

PITFALLS

Using the cell size smaller than r so close points fall in non-adjacent cells. Comparing floating-point distances for equality.

HOW TO THINK ABOUT IT

Model it as a graph where points within r are neighbours and count connected components with Union-Find. The all-pairs check is O(n^2); to speed it up, hash points into square cells of side r so that only points in the same or an adjacent cell can be within r. Compare squared distances to r * r to avoid square roots.

Basis: Connected components of a distance graph (LeetCode 547 idea). Two points are connected when their Euclidean distance is at most r; a cluster is a connected component.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
parent = list(range(n))
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x
for a, b in combinations(range(n), 2):
    if dist2(points[a], points[b]) <= r * r:
        parent[find(a)] = find(b)
clusters = len({find(i) for i in range(n)})
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n^2 / 2 distance checks. Hashing points into grid cells of side r means each point is compared only with points in its own and neighbouring cells.

OPTIMAL SOLUTION · Grid hashing with cells of side r, then Union-Find

TIME	SPACE
O(n) expected for evenly spread points	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Report the size of the largest cluster.

Count points per root after the unions (a Counter of find(i)). The unions are the same.

FOLLOW-UP 2 Points arrive one at a time.

Insert the point into its grid cell, union it with every point within r in the 9 surrounding cells, and keep the component count: components += 1 - (number of distinct roots merged).

FOLLOW-UP 3 Millions of points in 3D or higher.

The grid generalises (3^d neighbouring cells), but for high dimensions use a k-d tree or locality-sensitive hashing; or DBSCAN, which is this problem with a minimum neighbour count. **Cost:** grid hashing visits 3^d neighbouring cells per point, O(n * 3^d); a k-d tree builds in O(n log n) with O(n^(1 - 1/d) + k) range queries.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Clusters of 2D Points Within a
# Radius
#
# This question tests graph traversal skills, specifically
# identifying connected components among a set of 2D
# points using distance-based adjacency. It assesses the
# ability to model a geometric problem as a graph and
# apply DFS, BFS, or Union-Find, a common evaluation of
# algorithmic thinking in coding interviews. It also
# probes awareness of efficiency trade-offs when scaling
# to large point sets.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Report the size of the largest cluster.
# 2. Points arrive one at a time.
# 3. Millions of points in 3D or higher.
#
# INTERVIEW NOTES
# Model it as a graph where points within r are neighbours
# and count connected components with Union-Find. The all-
# pairs check is O(n^2); to speed it up, hash points into
# square cells of side r so that only points in the same
# or an adjacent cell can be within r. Compare squared
# distances to r * r to avoid square roots.
#
# BRUTE FORCE (say it first): Union-Find over all pairs of
# points
# Compare every pair, union the ones within r, and count
# the roots. Correct and simple; the quadratic pair loop
# is the cost. Time O(n^2). Space O(n).


from collections import Counter, defaultdict


# BRUTE FORCE (say it first): Union-Find over all pairs of
# points | time O(n^2)
def _find_factory(n):
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    return parent, find


# HELPERS for the base question
# FOLLOW-UP 2: Points arrive one at a time.
# Answer: Insert the point into its grid cell, union it
# with every point within r in the 9 surrounding cells,
# and keep the component count: components += 1 - (number
# of distinct roots merged).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class Clusters:
    """Points arrive one by one; count the clusters."""

    def __init__(self, r):
        self.r = r
        # grid cell (side r) -> indices of points inside
        self.cells = defaultdict(list)
        self.points, self.parent = [], []
        self.components = 0

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def near(self, p, q):
        dx, dy = p[0] - q[0], p[1] - q[1]
        return dx * dx + dy * dy <= self.r * self.r

    def add(self, x, y):
        i = len(self.points)
        self.points.append((x, y))
        self.parent.append(i)
        self.components += 1
        size = self.r or 1
        cx, cy = x // size, y // size
        self.cells[(cx, cy)].append(i)
        # look only at the 9 cells around the new point
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                key = (cx + dx, cy + dy)
                cell = self.cells.get(key, ())
                for j in cell:
                    q = self.points[j]
                    if j != i and self.near((x, y), q):
                        a, b = self.find(i), self.find(j)
                        if a != b:
                            self.parent[a] = b
                            self.components -= 1
        return self.components

    def largest(self):
        sizes = Counter(self.find(i)
                        for i in range(len(self.points)))
        return max(sizes.values(), default=0)


def clusters_all_pairs(points, r):
    parent, find = _find_factory(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if (points[i][0] - points[j][0]) ** 2 + (
                points[i][1] - points[j][1]
```

```
            ) ** 2 <= r * r:
                parent[find(i)] = find(j)
    return len({find(i) for i in range(len(points))})


# BASE QUESTION (optimal): Grid hashing with cells of side
# r, then Union-Find
# Idea: Put each point in the cell (x // r, y // r). A
# point within distance r of another can only be in the
# same cell or one of the 8 neighbouring cells, so compare
# only with those.
def count_clusters(points, r):
    c = Clusters(r)
    for x, y in points:
        c.add(x, y)
    return c.components


# FOLLOW-UP 1: Report the size of the largest cluster.
# Answer: Count points per root after the unions (a
# Counter of find(i)). The unions are the same.
def largest_cluster(points, r):
    c = Clusters(r)
    for x, y in points:
```

```
        c.add(x, y)
    return c.largest()


if __name__ == "__main__":
    line = [(0, 0), (3, 0), (6, 0), (20, 20)]
    assert count_clusters(line, 3) == 2
    assert largest_cluster(line, 3) == 3
    assert count_clusters([], 3) == 0
    # online: the count changes as points arrive
    c = Clusters(2)
    assert [c.add(*p) for p in [(0, 0), (10, 10), (1, 1),
                                (5, 5)]] == [1, 2, 2, 3]
    # (3, 3) is alone; (4, 4) then bridges it to (5, 5)
    assert c.add(3, 3) == 4
    assert c.add(4, 4) == 3
    assert count_clusters([(1, 1), (1, 1)], 0) == 1
```

FUNCTION AND TEST CASES

Signature

```
def countClusters(points: list[list[int]], r: int) -> int:
```

Sample call

```
countClusters([[0, 0], [1, 0], [10, 10]], 2)
```

Test cases

Input	Expected
([[0, 0], [1, 0], [10, 10]], 2)	2
([[0, 0], [3, 0], [6, 0]], 3)	1

ALSO REPORTED · 2 more

Count Connected Clusters of Two-Dimensional Points | PracHub | Technical Screen

https://prachub.com/coding-questions/count-connected-clusters-of-two-dimensional-points

2D Points Connected Components by Radius | 1Point3Acres | phone-screen | asked 2026-08-04 | Low

ALSO REPORTED · 2 more

https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

DFS, BFS, or Union-Find all fit the core connected-components model.

For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.

Clarify squared-distance comparison to avoid unnecessary square roots.

Preparation: Implement connected components once with DFS/BFS and once with Union-Find.

Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.

Dry-run a case where three points form one cluster only through transitive links.

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 127 | PracHub | Technical Screen | 5 of 25

Find shortest relationship path using BFS

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK ALL TIME 41.8 30 DAYS 0 PRIORITY 87/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #127 Word Ladder

LeetCode match: LC 127 Word Ladder (BFS shortest path) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/find-shortest-relationship-path-using-bfs

QUESTION

Overview: This question evaluates proficiency in graph representations and traversal algorithms—specifically BFS and adjacency-structure construction—for computing unweighted shortest paths and reasoning about reachability.

APPROACH

Build an adjacency list, BFS from the source with a parent map, then reconstruct the path; O(V+E). Consider bidirectional BFS.

HOW TO THINK ABOUT IT

Unweighted means every hop costs the same, so breadth-first search finds the fewest hops: it explores all people one hop away, then two hops, and the first time it reaches the target is the shortest path. Keep a parent map to rebuild the path and a visited set so nobody is queued twice.

Basis: LeetCode 127 (Word Ladder) idea: shortest path in an unweighted graph of people. Return the list of people from source to target, or None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = None
def dfs(node, path):
    global best
    if node == target:
        if not best or len(path) < len(best):
            best = list(path)
        return
    for nxt in graph[node]:
        if nxt not in path:
            dfs(nxt, path + [nxt])
dfs(source, [source])
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

The number of simple paths is exponential. Breadth-first search reaches the target along a shortest path first, O(V + E), and bidirectional BFS explores far fewer nodes on big graphs.

OPTIMAL SOLUTION · Breadth-first search with a parent map

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is huge (a social network).

Bidirectional BFS searches from both ends and stops when the frontiers meet; with branching factor b and distance d it explores about 2 * b^(d/2) nodes instead of b^d.

FOLLOW-UP 2 Return every shortest path, or count them.

Keep, for each node, its distance and a list of all parents at that distance; counting is a DP over the BFS layers, and listing the paths is a DFS over the parent lists.

FOLLOW-UP 3 Relationships have different strengths (weights).

BFS is no longer right: use Dijkstra with a heap for non-negative weights. Keep the same parent map to rebuild the path.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find shortest relationship path using BFS
#
# This question evaluates proficiency in graph
# representations and traversal algorithms—specifically
# BFS and adjacency-structure construction—for computing
# unweighted shortest paths and reasoning about
# reachability.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is huge (a social network).
# 2. Return every shortest path, or count them.
# 3. Relationships have different strengths (weights).
#
# INTERVIEW NOTES
# Unweighted means every hop costs the same, so breadth-
# first search finds the fewest hops: it explores all
# people one hop away, then two hops, and the first time
```

```
# it reaches the target is the shortest path. Keep a
# parent map to rebuild the path and a visited set so
# nobody is queued twice.
#
# BRUTE FORCE (say it first): Depth-first search over all
# simple paths
# Try every simple path from the source and keep the
# shortest that reaches the target. Correct and the
# oracle; exponential in the size of the graph. Time
# exponential. Space O(path).


import heapq
from collections import defaultdict, deque


# BRUTE FORCE (say it first): Depth-first search over all
# simple paths | time exponential
def shortest_relationship_dfs(adj, s, t):
    best = [None]

    def go(u, path):
        if best[0] is not None and len(path) >= len(
            best[0]
        ):
            return
        if u == t:
            best[0] = list(path)
            return
        for v in adj.get(u, ()):
            if v not in path:
                path.append(v)
                go(v, path)
                path.pop()

    go(s, [s])
    return best[0]


# BASE QUESTION (optimal): Breadth-first search with a
# parent map
# Idea: Queue the source and mark it visited. Pop a
# person, queue each unvisited neighbour with parent =
# this person, and stop when the target is queued. Follow
# parents back to the source and reverse to get the path.
def shortest_path(adj, s, t):
    # BFS: the first time we reach a node is by a shortest
    # path
    if s == t:
        return [s]
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in parent:
                parent[v] = u
                if v == t:
                    # walk the parent links back to s
                    path = [v]
                    while parent[path[-1]] is not None:
                        path.append(parent[path[-1]])
                    return path[::-1]
                queue.append(v)
    return None


# FOLLOW-UP 1: The graph is huge (a social network).
# Answer: Bidirectional BFS searches from both ends and
# stops when the frontiers meet; with branching factor b
# and distance d it explores about 2 * b^(d/2) nodes
# instead of b^d.
def bidirectional(adj, s, t):
    # a huge graph: search from both ends, expand the
    # smaller
    # side, stop when the searches meet
    if s == t:
        return 0
    dist_s, dist_t = {s: 0}, {t: 0}
    front_s, front_t = [s], [t]
    while front_s and front_t:
        if len(front_s) > len(front_t):
            front_s, front_t = front_t, front_s
            dist_s, dist_t = dist_t, dist_s
        nxt = []
        for u in front_s:
            for v in adj.get(u, ()):
                if v in dist_t:
                    return dist_s[u] + 1 + dist_t[v]
                if v not in dist_s:
                    dist_s[v] = dist_s[u] + 1
                    nxt.append(v)
        front_s = nxt
    return None


# FOLLOW-UP 2: Return every shortest path, or count them.
# Answer: Keep, for each node, its distance and a list of
# all parents at that distance; counting is a DP over the
# BFS layers, and listing the paths is a DFS over the
# parent lists.
def all_shortest_paths(adj, s, t):
    # every shortest path: remember ALL parents at the
    # best level
    dist, queue = {s: 0}, deque([s])
    parents = defaultdict(list)
    while queue:
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
            if dist[v] == dist[u] + 1:
                parents[v].append(u)
    if t not in dist:
        return []
    paths = []

    def build(node, tail):
        if node == s:
            paths.append([s] + tail)
            return
        for p in parents[node]:
            build(p, [node] + tail)

    build(t, [])
    return paths


# FOLLOW-UP 3: Relationships have different strengths
# (weights).
# Answer: BFS is no longer right: use Dijkstra with a heap
# for non-negative weights. Keep the same parent map to
# rebuild the path.
def weighted_path(graph, s, t):
    # strengths differ: Dijkstra, same parent map
    dist, parent = {s: 0}, {s: None}
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, ()):
            if d + w < dist.get(v, float("inf")):
```

```
                dist[v], parent[v] = d + w, u
                heapq.heappush(heap, (d + w, v))
    if t not in dist:
        return None
    path = [t]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return dist[t], path[::-1]


if __name__ == "__main__":
    adj = {"a": ["b", "c"], "b": ["d"], "c": ["d"],
           "d": ["e"], "e": []}
    assert shortest_path(adj, "a", "e") in (
        ["a", "b", "d", "e"], ["a", "c", "d", "e"])
    assert shortest_path(adj, "a", "a") == ["a"]
    assert shortest_path(adj, "e", "a") is None
    both_ways = {"a": ["b", "c"], "b": ["a", "d"],
                 "c": ["a", "d"], "d": ["b", "c", "e"],
                 "e": ["d"]}
```

```
    assert bidirectional(both_ways, "a", "e") == 3
    assert bidirectional(both_ways, "a", "a") == 0
    paths = all_shortest_paths(adj, "a", "e")
    assert sorted(paths) == [["a", "b", "d", "e"],
                             ["a", "c", "d", "e"]]
    graph = {"a": [("b", 1), ("c", 5)], "b": [("c", 1)]}
    cheapest = weighted_path(graph, "a", "c")
    assert cheapest == (2, ["a", "b", "c"])
    assert weighted_path(graph, "c", "a") is None
```

FUNCTION AND TEST CASES

Signature

```
def solution(relationships: list[list[int]], start: int, t
arget: int) -> int:
```

Sample call

```
solution([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')
```

Test cases

Input	Expected
`([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')`	`3`
`([[1, 2], [2, 5], [1, 3], [3, 4], [4, 5]], 1, 5)`	`2`

ALSO REPORTED · 1 more

Degrees of Friendship | Exponent

https://www.aced.io/questions/3628/degrees-of-friendship

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 261 | PracHub | Technical Screen | 6 of 25

Validate parent array forms a tree

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 29** **30 DAYS 0** **PRIORITY 128/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #261 Graph Valid Tree

LeetCode match: LC 261 Graph Valid Tree

Posted on PracHub: 2026-02-11 (updated 2026-04-23)

Original: https://prachub.com/coding-questions/validate-parent-array-forms-a-tree

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2.
No cycles (following parent pointers from any node must eventually reach the root). 3.
Connectivity (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

QUESTION

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.
3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints
1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

EXAMPLES

```
Input: [-1, 0, 0, 1, 1]
Expected Output: True
```

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

```
Input: [2, -1, 1, 2]
Expected Output: True
Explanation: Node 1 is the root. The paths are
    0->2->1, 2->1, and 3->2->1, so every node reaches
    the same root.
```

Hints

First count how many roots there are. A valid tree must have exactly one **`-1`** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

APPROACH

Exactly one root (parent = -1 or self), every other node has one parent; detect cycles by walking up with a visited set or union-find; all nodes must reach the root. O(n).

HOW TO THINK ABOUT IT

Exactly one -1 gives the root; then the rest is whether every node reaches it. Following parents from each node can loop (a cycle) or run out of range. Do it with memoised walks so every node is processed once: mark nodes on the current walk, and if the walk meets one of them again it is a cycle; if it meets a node already known to reach the root, all nodes on the walk reach it too.

Basis: LeetCode 261 (Graph Valid Tree) in parent-pointer form: exactly one root (-1), no cycles, every node reaches the root.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_tree(parent):
    n = len(parent)
    roots = [i for i in range(n)
             if parent[i] == -1]
    if len(roots) != 1:
        return False
    if any(p >= n for p in parent):
        return False
    for node in range(n):
        cur, steps = node, 0
        while cur != -1 and steps <= n:
            cur = parent[cur]
            steps += 1
        if cur != -1:
            return False  # a cycle
    return True
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every node walks its whole chain, O(n^2). Marking nodes as 'on this walk' or 'verified' lets every node be processed once, O(n).

OPTIMAL SOLUTION · One root, then colour each node once while walking up

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is given as undirected edges (LeetCode 261).

A tree has exactly n - 1 edges and no cycle: check the edge count, then union endpoints and fail on an edge that joins two nodes already connected. O(n alpha(n)).

FOLLOW-UP 2 Report the reason it is invalid.

Return which rule failed: number of roots, an out-of-range parent, or the node where the cycle was detected (the node that was met twice on one walk).

FOLLOW-UP 3 A binary tree given as child arrays (LeetCode 1361).

Check that exactly one node has in-degree 0, every other node has in-degree 1, and that a traversal from the root visits all n nodes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Validate parent array forms a tree
#
# ## Problem You are given an integer array parent of
# length n describing a directed parent pointer for each
# node i (nodes are labeled 0..n-1). - parent[i] is the
# parent of node i. - Exactly one node should be the root,
# indicated by parent[root] = -1. Determine whether these
# n nodes form a **valid rooted tree**. A valid rooted
# tree must satisfy all of the following: 1. **Exactly one
# root** (exactly one index i with parent[i] = -1). 2.
# **No cycles** (following parent pointers from any node
# must eventually reach the root). 3. **Connectivity**
# (every node is reachable from the root; equivalently,
# every node has exactly one simple path to the root). ##
# Input - parent: integer array of length n where each
# value is either -1 or in [0, n-1]. ## Output - Return
# true if parent represents a valid rooted tree; otherwise
# return false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is given as undirected edges (LeetCode
# 261).
# 2. Report the reason it is invalid.
# 3. A binary tree given as child arrays (LeetCode 1361).
#
# INTERVIEW NOTES
# Exactly one -1 gives the root; then the rest is whether
# every node reaches it. Following parents from each node
# can loop (a cycle) or run out of range. Do it with
# memoised walks so every node is processed once: mark
# nodes on the current walk, and if the walk meets one of
# them again it is a cycle; if it meets a node already
# known to reach the root, all nodes on the walk reach it
# too.
#
# BRUTE FORCE (say it first): Follow the parent chain from
# every node, up to n steps
# From each node follow parents for at most n steps; if
# you do not reach the root in n steps there is a cycle.
# Correct and the oracle; each node walks its whole chain
# separately. Time O(n^2). Space O(1).


from collections import defaultdict
```

```
# BRUTE FORCE (say it first): Follow the parent chain from
# every node, up to n steps | time O(n^2)
def valid_tree_walk(parent):
    n = len(parent)
    roots = [i for i, p in enumerate(parent) if p == -1]
    if len(roots) != 1 or any(
        p != -1 and not (0 <= p < n) for p in parent
    ):
        return False
    for i in range(n):
        x, steps = i, 0
        while parent[x] != -1 and steps <= n:
            x = parent[x]
            steps += 1
        if parent[x] != -1:
            return False
    return True


# HELPERS for the base question
```

```
# FOLLOW-UP 2: Report the reason it is invalid.
# Answer: Return which rule failed: number of roots, an
# out-of-range parent, or the node where the cycle was
# detected (the node that was met twice on one walk).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def reason(parent):
    # None if valid, else the first rule that fails
    n = len(parent)
    roots = parent.count(-1)
    if roots != 1:
        return f"{roots} roots (need exactly 1)"
    for i, p in enumerate(parent):
        if p != -1 and not 0 <= p < n:
            return f"node {i}: out-of-range parent {p}"
        if p == i:
            return f"node {i} is its own parent"
    # 0 = unseen, 1 = on the current walk, 2 = checked
    state = [0] * n
    for start in range(n):
        path, x = [], start
        while x != -1 and state[x] == 0:
            state[x] = 1
            path.append(x)
            x = parent[x]
        # meeting a node twice on one walk is a cycle
        if x != -1 and state[x] == 1:
            return f"cycle through node {x}"
        for v in path:
            state[v] = 2
    return None


# BASE QUESTION (optimal): One root, then colour each node
# once while walking up
# Idea: Check there is exactly one -1 and all other values
# are in range. Then walk up from each unvisited node
# marking nodes as 'on this walk'.
def validate_parent_array(parent):
    return reason(parent) is None


# FOLLOW-UP 1: The graph is given as undirected edges
# (LeetCode 261).
# Answer: A tree has exactly n - 1 edges and no cycle:
# check the edge count, then union endpoints and fail on
# an edge that joins two nodes already connected. O(n
# alpha(n)).
def valid_tree_edges(n, edges):
    # LeetCode 261: undirected edges
    if len(edges) != n - 1:
        return False
    root = list(range(n))

    def find(x):
        while root[x] != x:
            root[x] = root[root[x]]
            x = root[x]
        return x

    for u, v in edges:
        a, b = find(u), find(v)
        # an edge inside a group closes a cycle
        if a == b:
            return False
        root[a] = b
    return True


# FOLLOW-UP 3: A binary tree given as child arrays
# (LeetCode 1361).
# Answer: Check that exactly one node has in-degree 0,
# every other node has in-degree 1, and that a traversal
# from the root visits all n nodes.
def valid_binary_tree(n, left, right):
    # LeetCode 1361: child arrays, -1 means no child
    indeg = [0] * n
    for kids in (left, right):
        for c in kids:
            if c != -1:
                indeg[c] += 1
    roots = [i for i in range(n) if indeg[i] == 0]
    if len(roots) != 1 or max(indeg) > 1:
        return False
    seen, stack = {roots[0]}, [roots[0]]
    while stack:
        u = stack.pop()
        for c in (left[u], right[u]):
            if c != -1:
                if c in seen:
                    return False
                seen.add(c)
                stack.append(c)
    # every node must be reachable from the root
    return len(seen) == n


if __name__ == "__main__":
    assert validate_parent_array([-1, 0, 0, 1])
    assert validate_parent_array([-1])
    # two roots, none, and a cycle 1 -> 2 -> 1
    assert not validate_parent_array([-1, -1])
    assert not validate_parent_array([1, 0])
    assert "cycle" in reason([-1, 2, 1])
    assert not validate_parent_array([-1, 5])
    assert "out-of-range" in reason([-1, 5])
    assert "own parent" in reason([-1, 1])
    star = [(0, 1), (0, 2), (0, 3), (1, 4)]
    assert valid_tree_edges(5, star)
    loop = [(0, 1), (1, 2), (2, 0)]
    assert not valid_tree_edges(4, loop)
    left, right = [1, -1, 3, -1], [2, -1, -1, -1]
    assert valid_binary_tree(4, left, right)
    assert not valid_binary_tree(2, [1, 0], [-1, -1])
```

FUNCTION AND TEST CASES

Signature

```
def solution(parent: list[int]) -> bool:
SAMPLE INPUT
[-1, 0, 0, 1, 1]
```

Test cases

Input	Expected
[-1, 0, 0, 1, 1]	True
[2, -1, 1, 2]	True

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | PracHub | Technical Screen | 7 of 25

Solve matrix groups and recipe inventory

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 33.8** **30 DAYS 0** **PRIORITY 113/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #2115 Find All Possible Recipes from Given Supplies

LeetCode match: No direct match; related LC 2115 Find All Possible Recipes from Given Supplies (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-12)

Original: https://prachub.com/coding-questions/solve-matrix-groups-and-recipe-inventory

QUESTION

Overview: This pair of problems evaluates block-wise grid processing and aggregation for minimum-value selection alongside dependency-resolution and resource-allocation reasoning for recipe inventory planning, testing competencies in array manipulation, tie-breaking and constraint handling, graph-based dependency management and cycle detection, and algorithmic complexity analysis; they are commonly asked to gauge an applicant's ability to implement correct, efficient solutions under input-size and edge-case constraints. They belong to the Coding & Algorithms domain with subdomains in data structures (arrays/grids), algorithm design and analysis, and graph theory, and primarily require practical application of algorithmic techniques with attention to time/space complexity rather than purely theoretical proofs.

APPROACH

Part 1: iterate block by block and take the min per group. Part 2: topological sort/DFS over recipe dependencies with an inventory map.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Matrix groups: how the grid is split into blocks (group size) and what is taken per block (the minimum). Recipes: can a recipe be made from supplies, do recipes need other recipes, are amounts counted or only availability (LeetCode 2115 is availability), how are cycles treated.

BRUTE FORCE TO SAY OUT LOUD

Matrix: scan each block directly. Recipes: recursive search from each recipe without memory.

OPTIMAL IDEA

Matrix: iterate block by block with nested loops and take the minimum of each block, O(mn). Recipes: build a graph from ingredients to recipes with in-degrees and run Kahn's algorithm starting from the supplies; a recipe becomes available when all its ingredients are; recipes on a cycle never become available. If quantities matter, DFS with memoization over recipes tracking inventory, and decide how to share a scarce ingredient between recipes.

DRY-RUN THESE CASES

A grid not divisible by the block size, a recipe whose ingredient is another recipe, a cycle between two recipes, a missing ingredient, an empty supply list.

LIKELY FOLLOW-UPS

Amounts per ingredient (resource allocation), the maximum number of each recipe, or the cheapest set of recipes.

PITFALLS

Treating availability as quantities. Not handling a cycle.

HOW TO THINK ABOUT IT

Blocks: visit each cell once and update the minimum of its block, indexed by (row // k, col // k). Recipes: this is a dependency graph; a recipe is makeable when all its ingredients are supplies or makeable recipes. Count unmet ingredients per recipe and process recipes as they become available (Kahn's algorithm); recipes on a cycle never become available.

Basis: Two parts (my copy has only the overview, so the rules are assumed): take the minimum of every k x k block of a matrix, and decide which recipes can be made from supplies where recipes may need other recipes (LeetCode 2115, Find All Possible Recipes).

BRUTE FORCE · Blocks: collect each block's values and call min

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, h, w):
    answers = []
    for block in blocks(grid, h, w):
        cells = []
        for r, c in block:
            cells.append(grid[r][c])
        answers.append(min(cells))
    return answers
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

Copies every block into a list. One pass that updates the minimum of cell (r // k, c // k) needs no copies.

BRUTE FORCE · Recipes: keep re-scanning until no recipe changes

BRUTE FORCE · PSEUDO-CODE

```
available = set(supplies)
while True:
    added = False
    for recipe in recipes:
        if recipe.name in available:
            continue
        needs = recipe.needs
        if all(i in available for i in needs):
            available.add(recipe.name)
            added = True
    if not added:
        break
```

TIME	SPACE
O(R^2 * I)	O(R)

WHY IT IS SLOW

Many passes over all recipes, O(R^2 * I). Counting each recipe's missing ingredients and processing items as they become available (Kahn's algorithm) is O(R + I).

OPTIMAL SOLUTION · Blocks: one pass updating a minimum per block

TIME	SPACE
O(m * n)	O(blocks)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Recipes: Kahn's algorithm on the ingredient graph

TIME	SPACE
O(R + I)	O(R + I)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Ingredients have quantities and a shared ingredient can only be used once.

Then availability is not enough: you also have to decide the order in which to spend scarce items. A priority order with a simulation (as coded) is a heuristic; the exact maximum is a resource-allocation problem (flow or integer programming).

FOLLOW-UP 2 Blocks that do not tile the matrix exactly.

The last block row or column is smaller; the single-pass update handles that automatically because the block index is r // k and c // k.

FOLLOW-UP 3 Detect which recipes are impossible because of a cycle.

After Kahn's algorithm, recipes whose counts never reached zero are either missing an ingredient or on a cycle; a DFS with colours over just those recipes tells the two cases apart.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve matrix groups and recipe inventory
#
# This pair of problems evaluates block-wise grid
# processing and aggregation for minimum-value selection
# alongside dependency-resolution and resource-allocation
# reasoning for recipe inventory planning, testing
# competencies in array manipulation, tie-breaking and
# constraint handling, graph-based dependency management
# and cycle detection, and algorithmic complexity
# analysis; they are commonly asked to gauge an
# applicant's ability to implement correct, efficient
# solutions under input-size and edge-case constraints.
# They belong to the Coding & Algorithms domain with
# subdomains in data structures (arrays/grids), algorithm
# design and analysis, and graph theory, and primarily
# require practical application of algorithmic techniques
# with attention to time/space complexity rather than
# purely theoretical proofs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Ingredients have quantities and a shared ingredient
# can only be used once.
# 2. Blocks that do not tile the matrix exactly.
# 3. Detect which recipes are impossible because of a
# cycle.
#
# INTERVIEW NOTES
# Blocks: visit each cell once and update the minimum of
# its block, indexed by (row // k, col // k). Recipes:
# this is a dependency graph; a recipe is makeable when
# all its ingredients are supplies or makeable recipes.
# Count unmet ingredients per recipe and process recipes
# as they become available (Kahn's algorithm); recipes on
# a cycle never become available.
#
# BRUTE FORCE (say it first): Blocks: collect each block's
# values and call min
# For every block gather its cells into a list and take
# the minimum. Correct and the oracle; it copies every
# block. Time O(m * n). Space O(m * n).


from collections import defaultdict, deque
```

```
# BRUTE FORCE (say it first): Blocks: collect each block's
# values and call min | time O(m * n)
def block_mins_collect(matrix, k):
    m, n = len(matrix), len(matrix[0])
    return [
        [
            min(
                matrix[r][c]
                for r in range(i, min(i + k, m))
                for c in range(j, min(j + k, n))
            )
            for j in range(0, n, k)
```

```
        ]
        for i in range(0, m, k)
    ]


# BASE QUESTION (optimal): Blocks: one pass updating a
# minimum per block
# Idea: Each cell updates the minimum of block (r // k, c
# // k). No block list is built; the last blocks may be
# smaller when k does not divide the size.
def block_mins(matrix, k):
    # the minimum of every k x k block; edge blocks may be
    # smaller, which r // k and c // k handle by
    # themselves
    m, n = len(matrix), len(matrix[0])
    rows, cols = (m + k - 1) // k, (n + k - 1) // k
    out = [[float("inf")] * cols for _ in range(rows)]
    for r in range(m):
        for c in range(n):
            cell = out[r // k][c // k]
            out[r // k][c // k] = min(cell, matrix[r][c])
    return out


# PART 2: which recipes can be made
def possible_recipes(recipes, supplies):
    # recipes: name -> list of ingredients (which may be
    # other
    # recipes). Kahn's algorithm over missing-ingredient
    # counts.
    have = set(supplies)
    missing, users = {}, defaultdict(list)
    for name, items in recipes.items():
```

```
        missing[name] = 0
        for item in set(items):
            if item not in have:
                missing[name] += 1
                users[item].append(name)
    queue = deque(r for r, m in missing.items() if m == 0)
    made = []
    while queue:
        name = queue.popleft()
        made.append(name)
        # a finished recipe is a new ingredient for its
        # users
        for other in users[name]:
            missing[other] -= 1
            if missing[other] == 0:
                queue.append(other)
    return made


# FOLLOW-UP 1: Ingredients have quantities and a shared
# ingredient can only be used once.
# Answer: Then availability is not enough: you also have
# to decide the order in which to spend scarce items.
def recipes_with_quantities(recipes, needs, stock):
    # ingredients have amounts and stock is used up. Make
    # each recipe at most once, in priority order, while
    # the
    # stock allows (a made recipe is new stock). This is a
    # heuristic: the exact best choice is a flow problem.
    stock, made = dict(stock), []
    progress = True
    while progress:
        progress = False
        for name, need in zip(recipes, needs):
            if name in made:
                continue
            enough = all(stock.get(x, 0) >= q
                         for x, q in need.items())
            if enough:
```

```
                for x, q in need.items():
                    stock[x] -= q
                stock[name] = stock.get(name, 0) + 1
                made.append(name)
                progress = True
    return made
```

```
# FOLLOW-UP 3: Detect which recipes are impossible because
# of a cycle.
# Answer: After Kahn's algorithm, recipes whose counts
# never reached zero are either missing an ingredient or
# on a cycle; a DFS with colours over just those recipes
# tells the two cases apart.
def why_impossible(recipes, supplies):
    # for each recipe that cannot be made: a missing raw
    # ingredient, or a cycle
    made = set(possible_recipes(recipes, supplies))
    known = set(supplies) | set(recipes)
    reasons = {}
    for name, items in recipes.items():
        if name in made:
            continue
        gone = [i for i in items if i not in known]
        reasons[name] = "cycle"
        if gone:
            reasons[name] = "missing " + gone[0]
    return reasons


if __name__ == "__main__":
    matrix = [[5, 3, 8, 1], [2, 9, 4, 7], [6, 0, 3, 2]]
    # 2 x 2 blocks, the last row of blocks is only 1 row
    # tall
    assert block_mins(matrix, 2) == [[2, 1], [0, 2]]
    assert block_mins([[4]], 3) == [[4]]
    recipes = {"bread": ["flour", "yeast"],
               "sandwich": ["bread", "meat"],
               "burger": ["bread", "meat", "bun"]}
    stock = ["flour", "yeast", "meat"]
    got = possible_recipes(recipes, stock)
    assert got == ["bread", "sandwich"]
    # two recipes that need each other, and a missing item
    cyc = {"a": ["b"], "b": ["a"], "c": ["x"]}
    assert possible_recipes(cyc, []) == []
    assert why_impossible(cyc, []) == {"a": "cycle",
                                       "b": "cycle",
                                       "c": "missing x"}
    # amounts: one bread uses the only 2 flour, so the
    # cake
    # (which needs 2 flour too) cannot be made
```

```
    names = ["bread", "cake"]
    needs = [{"flour": 2}, {"flour": 2}]
    two = {"flour": 2}
    made = recipes_with_quantities(names, needs, two)
    assert made == ["bread"]
    four = {"flour": 4}
    both = recipes_with_quantities(names, needs, four)
    assert both == ["bread", "cake"]
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[int]:
```

Sample call

```
solution([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11
,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]])
```

Test cases

Input	Expected
([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]],)	(0, 3, 3)
([[-1,5,6,-1,7,8],[9,10,11,12,13,14],[15,16,17,18,19,20],[21,22,23,24,25,26],[27,28,29,30,31,32],[33,34,35,36,37,38]],)	(-1, 0, 0)

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 542 | PracHub | Technical Screen | 8 of 25

Compute distance to nearest taxi in grid

POSTED 2026-01-10 | 283 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 36.5** **30 DAYS 0** **PRIORITY 102/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #542 01 Matrix

LeetCode match: LC 542 01 Matrix (inferred from overview)

Posted on PracHub: 2026-01-10 (updated 2026-06-20)

Original: https://prachub.com/coding-questions/compute-distance-to-nearest-taxi-in-grid

QUESTION

Overview: This question evaluates understanding of graph traversal and shortest-path reasoning on grids, algorithmic problem-solving for computing distances from many targets, and the ability to manage time and space complexity on large inputs.

APPROACH

Multi-source BFS from all taxis simultaneously; the BFS level is the distance for each cell. O(mn).

HOW TO THINK ABOUT IT

Asking for the distance from every cell to its nearest source is one BFS started from all sources at once, not one BFS per cell. Put all taxis in the queue at distance 0; the first time the search reaches a cell is its shortest distance, because BFS visits cells in order of distance.

Basis: LeetCode 542, 01 Matrix, with 'T' for a taxi, '#' for a blocked cell and '.' for open ground. Distance counts moves through open cells (4-directional).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def nearest_taxi(grid, start):
    queue, seen = deque([(start, 0)]), {start}
    while queue:
        cell, d = queue.popleft()
        if is_taxi(grid, cell):
            return d
        for nxt in neighbours(grid, cell):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, d + 1))
    return -1
answer = {}
for c in cells:
    answer[c] = nearest_taxi(grid, c)
```

BRUTE FORCE

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

A separate BFS per cell, O((m * n)^2). One BFS that starts from all taxis at once reaches every cell at its nearest-taxi distance, O(m * n).

OPTIMAL SOLUTION · One multi-source BFS

TIME	SPACE
O(m * n)	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A new taxi appears.

Start a BFS from the new taxi but only continue into cells where you improve the stored distance. The distances can only decrease, so the work is bounded by the cells whose answer actually changes.

FOLLOW-UP 2 Taxis move or disappear.

Distances can increase, which the decrease-only update cannot handle. Rebuild with one multi-source BFS (O(mn)), perhaps lazily every few updates and accepting bounded staleness between rebuilds.

FOLLOW-UP 3 Return which taxi is nearest, not only the distance.

Store a source id with each cell in the BFS: a cell copies the id of the cell that first reached it. Ties go to whichever reached it first, so say how you want ties broken.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute distance to nearest taxi in grid
#
# This question evaluates understanding of graph traversal
# and shortest-path reasoning on grids, algorithmic
# problem-solving for computing distances from many
# targets, and the ability to manage time and space
# complexity on large inputs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A new taxi appears.
# 2. Taxis move or disappear.
# 3. Return which taxi is nearest, not only the distance.
#
# INTERVIEW NOTES
```

```
# Asking for the distance from every cell to its nearest
# source is one BFS started from all sources at once, not
# one BFS per cell. Put all taxis in the queue at distance
# 0; the first time the search reaches a cell is its
# shortest distance, because BFS visits cells in order of
# distance.
#
# BRUTE FORCE (say it first): BFS from every cell to find
# its nearest taxi
# For each open cell run a BFS until the first taxi is
# met. Correct, but it repeats almost the same search from
# every cell. Time O((m * n)^2). Space O(m * n).


from collections import deque


# BRUTE FORCE (say it first): BFS from every cell to find
# its nearest taxi | time O((m * n)^2)
def taxi_distances_brute(grid):
    R, C = len(grid), len(grid[0])
    out = [[-1] * C for _ in range(R)]
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "#":
                continue
            seen, queue = {(r, c)}, deque([(r, c, 0)])
            while queue:
                x, y, d = queue.popleft()
                if grid[x][y] == "T":
                    out[r][c] = d
```

```
                    break
                for nx, ny in (
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1),
                ):
                    if (
                        0 <= nx < R
                        and 0 <= ny < C
                        and grid[nx][ny] != "#"
                        and (nx, ny) not in seen
                    ):
                        seen.add((nx, ny))
                        queue.append((nx, ny, d + 1))
    return out


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def spread(grid, dist, queue, better=False):
    # grow outward; with better=True a cell may be updated
    # when a
    # new, closer source reaches it (distances only go
    # down)
    R, C = len(grid), len(grid[0])
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if not (0 <= a < R and 0 <= b < C):
                continue
            if grid[a][b] == "#":
                continue
            new = dist[r][c] + 1
            unseen = dist[a][b] == -1
            if unseen or (better and new < dist[a][b]):
                dist[a][b] = new
                queue.append((a, b))
```

```
# BASE QUESTION (optimal): One multi-source BFS
# Idea: Initialise the queue with every taxi at distance 0
```

```
# and expand in rounds. A cell receives its distance the
# first time it is reached; blocked cells are never
# entered; cells that no taxi can reach stay at -1.
def taxi_distances(grid):
    # 'T' taxi, '.' road, '#' wall. One BFS that starts
    # from
    # every taxi at once gives each cell its nearest taxi.
    R, C = len(grid), len(grid[0])
    dist = [[-1] * C for _ in range(R)]
    queue = deque()
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "T":
                dist[r][c] = 0
                queue.append((r, c))
    spread(grid, dist, queue)
    return dist


# FOLLOW-UP 1: A new taxi appears.
# Answer: Start a BFS from the new taxi but only continue
# into cells where you improve the stored distance. The
# distances can only decrease, so the work is bounded by
# the cells whose answer actually changes.
def add_taxi(grid, dist, r, c):
    # a new taxi: only cells it beats are touched
    dist[r][c] = 0
    spread(grid, dist, deque([(r, c)]), better=True)


# FOLLOW-UP 3: Return which taxi is nearest, not only the
# distance.
# Answer: Store a source id with each cell in the BFS: a
# cell copies the id of the cell that first reached it.
# Ties go to whichever reached it first, so say how you
# want ties broken.
def nearest_taxi_ids(grid):
    # which taxi is nearest? each cell copies the id of
    # the
    # cell that reached it first (ties: the first to
    # arrive)
    R, C = len(grid), len(grid[0])
    owner = [[None] * C for _ in range(R)]
    queue, taxi_id = deque(), 0
    for r in range(R):
```

```
        for c in range(C):
            if grid[r][c] == "T":
                owner[r][c] = taxi_id
                taxi_id += 1
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            inside = 0 <= a < R and 0 <= b < C
            if not inside or grid[a][b] == "#":
                continue
            if owner[a][b] is None:
                owner[a][b] = owner[r][c]
                queue.append((a, b))
    return owner


if __name__ == "__main__":
    grid = ["T...", ".#..", "...T"]
    d = taxi_distances(grid)
    # the second taxi at the bottom right is closer to (0,
```

```
    # 3)
    assert d[0] == [0, 1, 2, 2]
    assert d[2] == [2, 2, 1, 0]
    assert d[1][1] == -1
    # a new taxi in the middle of the top row
    add_taxi(grid, d, 0, 2)
    assert d[0] == [0, 1, 0, 1]
    assert d[2][0] == 2
    # unreachable cells stay -1
    walled = ["T#.", "##."]
    assert taxi_distances(walled)[0][2] == -1
    ids = nearest_taxi_ids(grid)
    assert ids[0][0] == 0 and ids[2][3] == 1
    assert ids[1][1] is None
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0,0,0],[0,1,0],[0,0,0]])
```

Test cases

Input	Expected
`([[0,0,0],[0,1,0],[0,0,0]],)`	`[[2,1,2],[1,0,1],[2,1,2]]`
`([[1,0,0,0],[0,0,0,1],[0,0,0,0]],)`	`[[0,1,2,1],[1,2,1,0],[2,3,2,1]]`

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 207 | PracHub | Technical Screen | 9 of 25

★ TOP 75: rank 33 in the Medium and Hard priority list

★ Validate course catalog dependencies

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 51.9** **30 DAYS 33** **PRIORITY 30/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #207 Course Schedule

LeetCode match: LC 207 Course Schedule

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/validate-course-catalog-dependencies

QUESTION

Design a function to validate an e-learning course catalog. You are given: (1) a set of course IDs, and (2) a list of prerequisite pairs (u, v) meaning course v requires course u. Determine if the catalog is valid: no prerequisite references a non-existent course, and there are no cycles in the dependency graph. If valid, return true and a possible ordering of courses; otherwise return false and report the set of missing course references and at least one cycle you detect. Specify your algorithm, time and space complexity.

APPROACH

Check every prerequisite refers to a known course ID, then Kahn's topological sort or DFS coloring to detect cycles.

HOW TO THINK ABOUT IT

Two separate validity rules: every course mentioned must exist, and the dependency graph must have no cycle. The first is a set lookup per pair. For the second, Kahn's algorithm removes courses with no unmet prerequisites; anything left over is on a cycle or depends on one, and a DFS restricted to the leftovers reveals an actual cycle to report.

Basis: LeetCode 207 / 210 (Course Schedule) plus a check for references to unknown courses. A pair (u, v) means course v requires u.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def validate(courses, prereqs):
    for u, v in prereqs:
        if u not in courses or v not in courses:
            return False  # unknown id
    graph = build_graph(prereqs)
    for c in courses:
        if reaches(graph, c, c):
            return False  # cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Every course can trigger a walk of the whole graph, O(V * (V + E)). Kahn's algorithm is O(V + E) and its leftovers also reveal a cycle to report.

OPTIMAL SOLUTION · Missing references, then Kahn's algorithm, then a cycle from the leftovers

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return all missing course references and one cycle.

The function above returns the sorted unknown ids when any exist and otherwise a cycle (a list of courses starting and ending at the same course), found by walking backwards along prerequisites inside the set of courses Kahn's algorithm could not remove (every such course has a leftover prerequisite, so the walk cannot dead-end).

FOLLOW-UP 2 Group courses into semesters (the earliest round each can be taken).

Run Kahn's algorithm level by level: all courses with no remaining prerequisites form round 1, remove them, the next set forms round 2, and so on. The number of rounds is the longest dependency chain.

FOLLOW-UP 3 Find all cycles, not just one.

Compute strongly connected components (Tarjan or Kosaraju); every component with more than one course (or a self-loop) is a cycle group to report.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Validate course catalog dependencies
#
# Design a function to validate an e-learning course
# catalog. You are given: ( 1) a set of course IDs, and (
# 2) a list of prerequisite pairs (u, v) meaning course v
# requires course u. Determine if the catalog is valid: no
# prerequisite references a non-existent course, and there
# are no cycles in the dependency graph. If valid, return
# true and a possible ordering of courses; otherwise
# return false and report the set of missing course
# references and at least one cycle you detect. Specify
# your algorithm, time and space complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return all missing course references and one cycle.
# 2. Group courses into semesters (the earliest round each
# can be taken).
# 3. Find all cycles, not just one.
#
# INTERVIEW NOTES
# Two separate validity rules: every course mentioned must
# exist, and the dependency graph must have no cycle. The
# first is a set lookup per pair. For the second, Kahn's
# algorithm removes courses with no unmet prerequisites;
# anything left over is on a cycle or depends on one, and
# a DFS restricted to the leftovers reveals an actual
# cycle to report.
#
# BRUTE FORCE (say it first): DFS from every course
# looking for a cycle
# First collect references to unknown courses; then, for
# each course, search for a path back to itself. Correct
# and the oracle; the same parts of the graph are searched
# repeatedly. Time O(V * (V + E)). Space O(V).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): DFS from every course
# looking for a cycle | time O(V * (V + E))
def catalog_valid_brute(courses, prereqs):
    cs = set(courses)
    missing = sorted(
        {
```

```
            x
            for u, v in prereqs
            for x in (u, v)
            if x not in cs
        }
    )
    if missing:
        return False
    adj = defaultdict(list)
    for u, v in prereqs:
        adj[u].append(v)

    def reaches_self(start):
        seen, stack = set(), list(adj[start])
        while stack:
            x = stack.pop()
            if x == start:
                return True
            if x not in seen:
                seen.add(x)
                stack.extend(adj[x])
        return False

    return not any(reaches_self(c) for c in cs)


# HELPERS for the base question
# FOLLOW-UP 3: Find all cycles, not just one.
# Answer: Compute strongly connected components (Tarjan or
# Kosaraju); every component with more than one course (or
# a self-loop) is a cycle group to report.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def find_cycle(known, prereqs, indeg):
    # every leftover course still has a leftover
    # prerequisite,
    # so walking backwards must eventually repeat a course
    left = {c for c in known if indeg[c] > 0}
    before = defaultdict(list)
    for u, v in prereqs:
        if u in left and v in left:
            before[v].append(u)
    x = min(left, key=str)
    walk, pos = [], {}
    while x not in pos:
```

```
        pos[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    cycle = walk[pos[x]:] + [x]
    # walked against the edges: reverse to list
    # prerequisites first
    return cycle[::-1]


# BASE QUESTION (optimal): Missing references, then Kahn's
# algorithm, then a cycle from the leftovers
# Idea: Report unknown courses (any referenced id not in
# the set). If none, run Kahn's algorithm: repeatedly
# remove courses with no remaining prerequisites, building
# an order.
# FOLLOW-UP 1: Return all missing course references and
# one cycle.
# Answer: The function above returns the sorted unknown
# ids when any exist and otherwise a cycle (a list of
# courses starting and ending at the same course), found
# by walking backwards along prerequisites inside the set
# of courses Kahn's algorithm could not remove (every such
# course has a leftover prerequisite, so the walk cannot
# dead-end).
def validate_catalog(courses, prereqs):
    # prereqs (u, v): course v requires course u.
    # Returns (ok, missing ids, order or one cycle)
    known = set(courses)
    missing = sorted({x for pair in prereqs for x in pair
                      if x not in known}, key=str)
    if missing:
        return False, missing, None
    adj, indeg = defaultdict(list), {c: 0 for c in known}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    ready = [c for c in known if indeg[c] == 0]
    queue = deque(sorted(ready, key=str))
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
```

```
                queue.append(nxt)
    if len(order) == len(known):
        return True, [], order
    return False, [], find_cycle(known, prereqs, indeg)


# FOLLOW-UP 2: Group courses into semesters (the earliest
# round each can be taken).
# Answer: Run Kahn's algorithm level by level: all courses
# with no remaining prerequisites form round 1, remove
# them, the next set forms round 2, and so on. The number
# of rounds is the longest dependency chain.
def semesters(courses, prereqs):
    # earliest round each course can be taken: Kahn by
    # levels
    adj = defaultdict(list)
    indeg = {c: 0 for c in courses}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    level = sorted(c for c in courses if indeg[c] == 0)
    rounds = []
    while level:
        rounds.append(level)
        nxt = []
        for c in level:
            for v in adj[c]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    nxt.append(v)
        level = sorted(nxt)
    return rounds


if __name__ == "__main__":
    courses = ["a", "b", "c", "d"]
    pre = [("a", "b"), ("b", "c"), ("a", "d")]
    ok, missing, order = validate_catalog(courses, pre)
    assert ok and order[0] == "a"
    assert order.index("b") < order.index("c")
    # an unknown course id
    bad = validate_catalog(courses, pre + [("zzz", "a")])
    assert bad == (False, ["zzz"], None)
    # a cycle b -> c -> b
    loop = pre + [("c", "b")]
```

```
    ok, _, cycle = validate_catalog(courses, loop)
    assert not ok and cycle[0] == cycle[-1]
    assert set(cycle) == {"b", "c"}
    rounds = semesters(courses, pre)
    assert rounds == [["a"], ["b", "d"], ["c"]]
    assert validate_catalog([], []) == (True, [], [])
```

FUNCTION AND TEST CASES

Signature

```
def solution(course_ids: list[str], prereqs: list[tuple[st
r, str]]) -> tuple[bool, list[str], list[str], list[str]]:
```

Sample call

```
solution(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])
```

Test cases

Input	Expected
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])`	`(True, ['A', 'B', 'C'], [], [])`
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C'), ('C', 'A')])`	`(False, [], [], ['A', 'B', 'C', 'A'])`

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | PracHub | Technical Screen | 10 of 25

★ TOP 75: rank 27 in the Medium and Hard priority list

★ Solve Grid and Counting Problems

POSTED 2025-01-25 | 633 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 64.8** **30 DAYS 33** **PRIORITY 18/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #200 Number of Islands

LeetCode match: LC 200 Number of Islands (+ a counting task)

Posted on PracHub: 2025-01-25 (updated 2026-04-24)

Original: https://prachub.com/coding-questions/solve-grid-and-counting-problems

QUESTION

A software engineering intern phone screen included two coding tasks: 1.

Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.

Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character. Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.

Constraints

0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length

EXAMPLES

```
Input: []
```

Expected Output:

Explanation: Empty grid contains no land, so there are no islands.

```
Input: [['1']]
```

QUESTION

Expected Output:

Explanation: A single land cell forms one island.

Hints

Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.

Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.

Constraints

0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type

Examples

Input: []

Expected Output:

Explanation: No letters means it is impossible to form GOOGLE.

Input: ['G','O','O','G','L','E']

Expected Output:

Explanation: There is exactly one way to choose all six letters.

Hints

First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1 L and 1 E.

APPROACH

Islands: DFS/BFS or union-find over '1' cells. Second counting task: use prefix-sum + hash map or math counting per the statement.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two tasks. Islands: four-directional adjacency, grid up to 300 x 300, empty grid allowed. Counting: choose letters from an array so the chosen ones can be rearranged to spell GOOGLE, each index used at most once, different indices are different choices.

BRUTE FORCE TO SAY OUT LOUD

Islands: none needed, DFS is the standard. Counting: enumerate all 6-element subsets of the array and test each one. O(n^6).

OPTIMAL IDEA

Islands: scan cells, start a DFS or BFS (iterative, to avoid recursion limits) from each unvisited land cell, count the starts. O(mn). Counting: GOOGLE needs G twice, O twice, L once, E once. The number of ways is C(g, 2) * C(o, 2) * l * e, where g, o, l, e are the counts of those letters in the array, since the chosen multiset is fixed and only the indices vary. O(n) to count.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Islands: empty grid, a single land cell, diagonal-only neighbors (separate islands). Counting: missing a letter (0), exactly one G (0), lots of copies, an empty array.

LIKELY FOLLOW-UPS

Islands: add land one cell at a time with Union-Find. Counting: a different target word, or count ways when letters have to appear in order (subsequence DP).

PITFALLS

Treating diagonal cells as connected. Counting arrangements instead of choices of indices.

HOW TO THINK ABOUT IT

Islands: every unvisited land cell starts a new island; mark everything connected to it. Counting: the target multiset is fixed (G twice, O twice, L once, E once), so the number of ways to choose the indices is a product of binomial coefficients, one factor per distinct letter. Do not enumerate arrangements.

Basis: Two tasks: LeetCode 200 (Number of Islands) and counting the ways to choose letters from an array that can be rearranged into GOOGLE (each index used at most once).

BRUTE FORCE · Islands: recursive flood fill

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == "0" or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
```

TIME	SPACE
O(m * n)	O(m * n) recursion

WHY IT IS SLOW

Time is O(m * n), but the recursion can be m * n deep on a snake-shaped island and overflow the stack. A queue or an explicit stack avoids that.

BRUTE FORCE · Counting: enumerate every 6-element selection

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    count = 0
    for idx in combinations(range(len(a)), 6):
        letters = sorted(a[i] for i in idx)
        if letters == sorted("GGOOLE"):
            count += 1
    return count
```

BRUTE FORCE · Counting: enumerate every 6-element selection

TIME	SPACE
O(C(n, 6) * 6)	O(1)

WHY IT IS SLOW

C(n, 6) selections. The letters needed are fixed, so the answer is C(#G, 2) * C(#O, 2) * #L * #E from one pass of counts.

OPTIMAL SOLUTION · Islands: iterative BFS

TIME	SPACE
O(m * n)	O(min(m, n)) queue

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Counting: product of binomial coefficients

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Any target word, not only GOOGLE.

Count the needed letters of the word; the answer is the product over distinct letters of C(available, needed).

FOLLOW-UP 2 The letters must appear in order (a subsequence of the array spells the word).

Then it is the 'distinct subsequences' DP: ways[j] counts the ways to spell the first j letters; scan the array and update j from high to low. O(n * len(word)).

FOLLOW-UP 3 Islands arrive one cell at a time.

Use Union-Find: each new land cell makes the count go up by one and each union with a neighbouring different island makes it go down by one (LeetCode 305).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve Grid and Counting Problems
#
# A software engineering intern phone screen included two
# coding tasks: 1. **Count islands in a grid** Given an m
# x n grid of characters where '1' represents land and '0'
# represents water, return the number of islands. An
```

```
# island is formed by connecting adjacent land cells
# horizontally or vertically. You may assume all four
# edges of the grid are surrounded by water. 2. **Count
# ways to form a target word** Given an array of uppercase
# letters, count how many different ways you can choose
# letters from the array so that the chosen letters can be
# rearranged to spell GOOGLE. Each array element can be
# used at most once, and letters at different indices are
# considered different choices even if they contain the
# same character. Design efficient algorithms for both
# tasks, and discuss the time and space complexity of your
# approach.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Any target word, not only GOOGLE.
# 2. The letters must appear in order (a subsequence of
# the array spells the word).
# 3. Islands arrive one cell at a time.
#
# INTERVIEW NOTES
# Islands: every unvisited land cell starts a new island;
# mark everything connected to it. Counting: the target
# multiset is fixed (G twice, O twice, L once, E once), so
# the number of ways to choose the indices is a product of
# binomial coefficients, one factor per distinct letter.
# Do not enumerate arrangements.
#
# BRUTE FORCE (say it first): Islands: recursive flood
# fill
# Scan the grid; at each unvisited land cell, count one
# island and recurse into the four neighbours. Correct and
# the textbook answer; deep recursion can overflow on a
# large snake-like island. Time O(m * n). Space O(m * n)
# recursion.


from collections import Counter, deque
from math import comb
```

```
# BRUTE FORCE (say it first): Islands: recursive flood
# fill | time O(m * n)
def islands_recursive(grid):
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen = set()

    def fill(r, c):
        if (
            not (0 <= r < m and 0 <= c < n)
            or grid[r][c] != "1"
            or (r, c) in seen
        ):
            return
        seen.add((r, c))
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            fill(r + dr, c + dc)

    count = 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == "1" and (r, c) not in seen:
                count += 1
                fill(r, c)
    return count
```

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): Islands: iterative BFS
# Idea: Same scan, but spread with a queue instead of
# recursion, so the depth of an island cannot overflow the
# stack. Marking visited cells in place (or with a set)
# keeps each cell processed once.
def num_islands(grid):
    # '1' is land: a BFS marks each island once
```

```
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            queue = deque([(r, c)])
            while queue:
                x, y = queue.popleft()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < m and 0 <= b < n):
                        continue
                    land = grid[a][b] == "1"
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        queue.append((a, b))
    return count


# PART 2: count GOOGLE in the array
# FOLLOW-UP 1: Any target word, not only GOOGLE.
# Answer: Count the needed letters of the word; the answer
# is the product over distinct letters of C(available,
# needed).
def ways_to_spell(letters, word="GOOGLE"):
    # choose which array positions to use: for every
    # needed
    # letter choose that many of the available copies
    have, need = Counter(letters), Counter(word)
    total = 1
    for ch, k in need.items():
        total *= comb(have[ch], k)
    return total


# FOLLOW-UP 2: The letters must appear in order (a
# subsequence of the array spells the word).
# Answer: Then it is the 'distinct subsequences' DP:
# ways[j] counts the ways to spell the first j letters;
# scan the array and update j from high to low. O(n *
```

```
# len(word)).
def ways_in_order(letters, word="GOOGLE"):
    # the letters must appear in order (a subsequence):
    # the
    # "distinct subsequences" DP, updated from high to low
    ways = [1] + [0] * len(word)
    for ch in letters:
        for j in range(len(word), 0, -1):
            if word[j - 1] == ch:
                ways[j] += ways[j - 1]
    return ways[len(word)]


if __name__ == "__main__":
    assert num_islands([]) == 0
```

```
assert num_islands([["1"]]) == 1
grid = ["11000", "11000", "00100", "00011"]
assert num_islands(grid) == 3
assert ways_to_spell([]) == 0
assert ways_to_spell(list("GOOGLE")) == 1
# three O's: choose 2 of them -> 3 ways
assert ways_to_spell(list("GOOOGLE")) == 3
# an extra G and an extra E: 3 * 2 = 6 ways
assert ways_to_spell(list("GGGOOLEE")) == 3 * 2
assert ways_to_spell(list("GOGLE")) == 0
# any target word
assert ways_to_spell(list("AABB"), "AB") == 4
assert ways_in_order(list("GOOGLE")) == 1
assert ways_in_order(list("GGOOGLE")) == 2
assert ways_in_order(list("GOGOLE")) == 0
assert ways_in_order(list("EGOOGL")) == 0
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[str]]) -> int:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`0`
`[['1']]`	`1`

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 207 | Exponent | 11 of 25

★ TOP 75: rank 40 in the Medium and Hard priority list

★ Course Schedule

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE **ALL TIME 51.9** **30 DAYS 33** **PRIORITY 31/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: your app's interview approach (LeetCode #207)

LeetCode match: LC 207 Course Schedule

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +1017 interviews

Original: https://www.aced.io/questions/3313/course-schedule

QUESTION

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True Explanation: You can complete course 0 first, then course 1, and finally course 2.
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False Explanation: The prerequisites form a cycle, making it impossible to finish all courses.
Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True Explanation: You can complete the courses in order from course 0 to course 4.Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.

APPROACH

Kahn's topological sort (or DFS cycle detection); the answer is true iff every course is processed. O(V+E).

HOW TO THINK ABOUT IT

Finishing every course is possible exactly when the prerequisite graph has no cycle. Kahn's algorithm tests that by repeatedly taking courses with no unmet prerequisites; if you can take all of them there is no cycle, and the order you took them is a valid schedule.

Basis: LeetCode 207 (can you finish all courses) and 210 (return an order). A pair (a, b) means b must be taken before a.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_finish(n, prereqs):
    graph = build_graph(n, prereqs)
    for course in range(n):
        if reaches(graph, course, course):
            return False  # a cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

The same parts of the graph are walked again for many starting courses, O(V * (V + E)). Kahn's algorithm (take courses with no unmet prerequisites) is O(V + E).

OPTIMAL SOLUTION · Kahn's algorithm (topological sort)

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the order of courses (LeetCode 210).

The same Kahn loop records the order; if fewer than n courses are taken return an empty list. O(V + E).

FOLLOW-UP 2 The minimum number of semesters if you can take any number of courses per semester.

Run Kahn's algorithm level by level: each level is one semester. The number of levels is the longest chain of prerequisites (return -1 if there is a cycle).

FOLLOW-UP 3 Report an actual cycle.

After Kahn's algorithm, the courses left over all still have an unmet prerequisite among themselves; walk backwards along prerequisites inside that set until a course repeats to extract a cycle.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Course Schedule
#
# You are given numCourses and a list of prerequisites,
# where each element in prerequisites[i] = [a_i, b_i]
# means you must complete course b_i before taking course
# a_i. Write a function canFinish that returns True if
# it's possible to finish all courses, and False if it's
```

```
# not. You need to complete all the courses for the result
# to be True. Assume there are no duplicate prerequisites,
# and numCourses is a non-negative number. Examples Input:
# numCourses = 3, prerequisites = [[2, 1], [1, 0]] Output:
# True Explanation: You can complete course 0 first, then
# course 1, and finally course 2. Input: numCourses = 4,
# prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]] Output:
# False Explanation: The prerequisites form a cycle,
# making it impossible to finish all courses.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the order of courses (LeetCode 210).
# 2. The minimum number of semesters if you can take any
# number of courses per semester.
# 3. Report an actual cycle.
#
# INTERVIEW NOTES
# Finishing every course is possible exactly when the
# prerequisite graph has no cycle. Kahn's algorithm tests
# that by repeatedly taking courses with no unmet
# prerequisites; if you can take all of them there is no
# cycle, and the order you took them is a valid schedule.
#
# BRUTE FORCE (say it first): DFS from every course
# looking for a path back to itself
# For each course follow dependencies; reaching the start
# again is a cycle. Correct and the oracle; the same parts
# of the graph are walked repeatedly. Time O(V * (V + E)).
# Space O(V).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): DFS from every course
# looking for a path back to itself | time O(V * (V + E))
def can_finish_brute(n, prereqs):
    adj = defaultdict(list)
```

```
    for a, b in prereqs:
        adj[b].append(a)

    def cyc(start):
        seen, stack = set(), list(adj[start])
        while stack:
            x = stack.pop()
            if x == start:
                return True
            if x not in seen:
                seen.add(x)
                stack.extend(adj[x])
        return False

    return not any(cyc(c) for c in range(n))


# HELPERS for the base question
def build(n, prereqs):
    # (a, b): course b must be taken before course a
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    return adj, indeg


# FOLLOW-UP 1: Return the order of courses (LeetCode 210).
# Answer: The same Kahn loop records the order; if fewer
# than n courses are taken return an empty list. O(V + E).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def find_order(n, prereqs):
    # Kahn's algorithm: take courses with no unmet
```

```
    # prerequisite
    adj, indeg = build(n, prereqs)
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
```

```
    # fewer than n taken: a cycle blocks the rest
    return order if len(order) == n else []


# BASE QUESTION (optimal): Kahn's algorithm (topological
# sort)
# Idea: Count each course's unmet prerequisites and queue
# those with none. Take a course, reduce the counts of the
# courses that need it, and queue any that reach zero.
def can_finish(n, prereqs):
    return len(find_order(n, prereqs)) == n


# FOLLOW-UP 2: The minimum number of semesters if you can
# take any number of courses per semester.
# Answer: Run Kahn's algorithm level by level: each level
# is one semester. The number of levels is the longest
# chain of prerequisites (return -1 if there is a cycle).
def min_semesters(n, prereqs):
    # any number of courses per semester: one BFS level
    # each
    adj, indeg = build(n, prereqs)
    level = [c for c in range(n) if indeg[c] == 0]
    semesters = taken = 0
    while level:
        semesters += 1
        taken += len(level)
        nxt = []
        for c in level:
            for v in adj[c]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    nxt.append(v)
        level = nxt
    return semesters if taken == n else -1


# FOLLOW-UP 3: Report an actual cycle.
# Answer: After Kahn's algorithm, the courses left over
# all still have an unmet prerequisite among themselves;
# walk backwards along prerequisites inside that set until
# a course repeats to extract a cycle.
def find_cycle(n, prereqs):
    # the leftover courses all have a leftover
    # prerequisite, so
```

```
    # walking backwards along prerequisites repeats a
    # course
    adj, indeg = build(n, prereqs)
    queue = deque(c for c in range(n) if indeg[c] == 0)
    while queue:
        c = queue.popleft()
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    left = {c for c in range(n) if indeg[c] > 0}
    if not left:
        return []
    before = defaultdict(list)
    for a, b in prereqs:
```

```
        if a in left and b in left:
            before[a].append(b)
    x, walk, pos = min(left), [], {}
    while x not in pos:
        pos[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    return walk[pos[x]:]


if __name__ == "__main__":
    assert can_finish(2, [(1, 0)])
    assert not can_finish(2, [(1, 0), (0, 1)])
    diamond = [(1, 0), (2, 0), (3, 1), (3, 2)]
    assert find_order(4, diamond)[0] == 0
    assert find_order(2, [(1, 0), (0, 1)]) == []
    assert can_finish(3, []) and can_finish(0, [])
    assert min_semesters(4, diamond) == 3
    assert min_semesters(2, [(1, 0), (0, 1)]) == -1
    assert min_semesters(3, []) == 1
    pairs = [(1, 0), (2, 1), (1, 2), (3, 2)]
    cycle = find_cycle(4, pairs)
    assert set(cycle) == {1, 2}
    assert find_cycle(2, [(1, 0)]) == []
```

OFFICIAL PAGE TEXT

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples

```
Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]
]
Output: True
Explanation: You can complete course 0 first, then cou
rse 1, and finally course 2.

Input: numCourses = 4, prerequisites = [[3, 2], [2, 1]
, [1, 0], [0, 3]]
Output: False
Explanation: The prerequisites form a cycle, making it
 impossible to finish all courses.

Input: numCourses = 5, prerequisites = [[4, 2], [3, 1]
, [2, 0]]
Output: True
Explanation: You can complete the courses in order fro
m course 0 to course 4.
```

OFFICIAL PAGE TEXT

Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.Our solution processes the input data to determine if it's possible to finish all courses given the prerequisite constraints. It constructs a directed graph where nodes represent courses and edges represent the prerequisite relationships. The solution employs a queue to manage nodes with zero in-degrees (courses with no remaining prerequisites) and uses a list to keep track of the number of prerequisites (in-degrees) for each course. For each course with zero in-degrees, the solution dequeues it and processes its neighbors, reducing their in-degrees. If a neighbor's in-degree becomes zero, it is added to the queue. This approach ensures that all courses with no remaining prerequisites are processed and their dependencies are updated accordingly. By maintaining a count of the number of processed courses, the solution can determine if all courses can be finished. If the count of processed courses matches the total number of courses, it implies that there are no cyclic dependencies, and thus all courses can be completed. The solution uses a queue to efficiently manage courses with zero in-degrees and a vector to keep track of in-degrees. This method ensures that each course is processed only once, making the

approach both clear and efficient. NoteWe are using a BFS (implemented by a queue) for this solution because it helps process nodes level by level (course by course), ensuring that we first handle courses with no dependencies before moving to those that rely on other courses. This approach ensures that we can detect cycles in the course prerequisites, which would make it impossible to complete all courses. Time Complexity: The solution has a time complexity of O(numCourses + numPrerequisites). This is because we process each course (vertex) and each prerequisite pair (edge) exactly once. The time complexity reflects the total number of courses (numCourses) and the number of prerequisite relationships (numPrerequisites). Space Complexity: The solution uses O(numCourses + numPrerequisites) space for the adjacency list and the in-degree vector. The space complexity accounts for the storage needed for both the graph representation and the queue, which in the worst case, contains all courses.No answers contributed by the community yet.

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 322 | Exponent | 12 of 25

Coin Change

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TYPE **ALL TIME 48.5** **30 DAYS 0** **PRIORITY 72/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: your app's interview approach (LeetCode #322)

LeetCode match: LC 322 Coin Change

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +27 interviews

Original: https://www.aced.io/questions/1142/coin-change

QUESTION

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples PythonInput: coins = [1, 2, 5], amount = 11 Output: 3 Explanation: The minimum number of coins needed is 3 (11 = 5 + 5 + 1).
Input: coins = [4, 5], amount = 7 Output: -1 Explanation: It is not possible to make up the amount 7 with coins of denominations 4 and 5.Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each

recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the

APPROACH

Bottom-up dp[a] = min(dp[a - c] + 1) over coins; -1 if unreachable. O(amount * coins).

HOW TO THINK ABOUT IT

The fewest coins for amount a is one coin plus the fewest for a - c, for the best coin c; that is a table over amounts built from 0 upward. Greedy (always take the biggest coin) is wrong for general coin systems, which is the classic trap: with coins 1, 3, 4 the amount 6 needs two coins (3 + 3), not three (4 + 1 + 1).

Basis: LeetCode 322, Coin Change: the fewest coins that sum to the amount, or -1; unlimited coins of each value.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fewest(a):
    if a == 0:
        return 0
    best = INF
    for c in coins:
        if c <= a:
            best = min(best, 1 + fewest(a - c))
    return best
```

TIME	SPACE
O(coins^amount)	O(amount)

WHY IT IS SLOW

The same amount is recomputed through many coin orders, exponential. Filling fewest[a] for a from 0 upward is O(amount * coins).

OPTIMAL SOLUTION · Bottom-up DP over amounts

TIME	SPACE
O(amount * coins)	O(amount)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count the number of combinations that make the amount (LeetCode 518).

Loop over coins in the outer loop and amounts in the inner loop, adding ways[a - c]; the order of the loops makes combinations (not permutations) count once.

FOLLOW-UP 2 Return which coins are used.

Remember the last coin chosen for each amount (a parent array) and walk back from the target.

FOLLOW-UP 3 Does greedy work?

Only for special coin systems (like US coins). For general coins it can be wrong (coins 1, 3, 4, amount 6). Mention the counterexample, then give the DP.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Coin Change
#
# You are given an integer array coins representing
# different coin denominations and an integer amount
# representing a total amount of money. Write a function
# coinChange that returns the fewest number of coins
# needed to make up that amount. If that amount cannot be
# made up by any combination of the coins, return -1. You
# may assume that you have an infinite number of each kind
# of coin. Examples PythonInput: coins = [1, 2, 5], amount
# = 11 Output: 3 Explanation: The minimum number of coins
# needed is 3 (11 = 5 + 5 + 1). Input: coins = [4, 5],
# amount = 7 Output: -1 Explanation: It is not possible to
# make up the amount 7 with coins of denominations 4 and
# 5.Solution 1: Brute-Force approach The naive approach to
# solving the coin change problem involves a recursive
# strategy to explore all possible combinations of coins
# to find the minimum number needed to make up the target
# amount.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the number of combinations that make the amount
# (LeetCode 518).
# 2. Return which coins are used.
# 3. Does greedy work?
#
# INTERVIEW NOTES
# The fewest coins for amount a is one coin plus the
# fewest for a - c, for the best coin c; that is a table
# over amounts built from 0 upward. Greedy (always take
# the biggest coin) is wrong for general coin systems,
# which is the classic trap: with coins 1, 3, 4 the amount
# 6 needs two coins (3 + 3), not three (4 + 1 + 1).
#
# BRUTE FORCE (say it first): Recursion over the last coin
# min_coins(a) = 1 + min over coins c <= a of min_coins(a
# - c). Exact and the oracle; the same amounts are
# recomputed exponentially often. Time O(coins^amount).
# Space O(amount).


from collections import deque


# BRUTE FORCE (say it first, it also checks the answers)
```

```
def brute_bfs(coins, amount):
    # oracle: breadth first search over the amount left
    seen, queue = {amount}, deque([(amount, 0)])
    while queue:
        left, steps = queue.popleft()
        if left == 0:
            return steps
        for c in coins:
            if left - c >= 0 and left - c not in seen:
                seen.add(left - c)
                queue.append((left - c, steps + 1))
    return -1


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Bottom-up DP over amounts
# Idea: best[0] = 0, and for a from 1 to amount, best[a] =
# 1 + min(best[a - c]) over coins c <= a (infinity if none
# works). The answer is best[amount] or -1. Each amount is
# solved once.
def coin_change(coins, amount):
    # best[a] = fewest coins that make exactly a
    best = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
    return -1 if best[amount] == INF else best[amount]


# FOLLOW-UP 1: Count the number of combinations that make
# the amount (LeetCode 518).
# Answer: Loop over coins in the outer loop and amounts in
# the inner loop, adding ways[a - c]; the order of the
# loops makes combinations (not permutations) count once.
def count_ways(coins, amount):
    # follow-up (LeetCode 518): how many COMBINATIONS.
    # Coins
    # in the outer loop means each combination is counted
    # once, not once per order.
    ways = [1] + [0] * amount
    for c in coins:
        for a in range(c, amount + 1):
            ways[a] += ways[a - c]
    return ways[amount]


# FOLLOW-UP 2: Return which coins are used.
# Answer: Remember the last coin chosen for each amount (a
# parent array) and walk back from the target.
def coins_used(coins, amount):
    # follow-up: which coins. Remember the last coin
    # chosen
    # for every amount, then walk back from the target.
    best = [0] + [INF] * amount
    last = [0] * (amount + 1)
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
                last[a] = c
    if best[amount] == INF:
        return None
    used, a = [], amount
    while a:
        used.append(last[a])
        a -= last[a]
    return sorted(used, reverse=True)


# FOLLOW-UP 3: Does greedy work?
# Answer: Only for special coin systems (like US coins).
# For general coins it can be wrong (coins 1, 3, 4, amount
# 6). Mention the counterexample, then give the DP.
def greedy(coins, amount):
    # follow-up: does greedy work? Only for special coin
    # systems. Take the biggest coin that still fits.
    used = []
    for c in sorted(coins, reverse=True):
        while amount >= c:
            amount -= c
            used.append(c)
    return used if amount == 0 else None


if __name__ == "__main__":
    assert coin_change([1, 2, 5], 11) == 3
```

```
assert coin_change([2], 3) == -1
assert coin_change([1], 0) == 0
assert count_ways([1, 2, 5], 5) == 4
assert count_ways([2], 3) == 0
assert count_ways([10], 10) == 1
assert count_ways([1, 2], 0) == 1
assert coins_used([1, 2, 5], 11) == [5, 5, 1]
assert coins_used([2], 3) is None
# greedy takes 4 + 1 + 1 (3 coins), the best is 3 + 3
assert greedy([1, 3, 4], 6) == [4, 1, 1]
assert coin_change([1, 3, 4], 6) == 2
assert sorted(coins_used([1, 3, 4], 6)) == [3, 3]
import random
for _ in range(300):
    size = random.randint(1, 3)
    coins = random.sample(range(1, 9), size)
    amount = random.randint(0, 25)
    assert coin_change(coins, amount) == brute_bfs(
        coins, amount)
    used = coins_used(coins, amount)
    if used is not None:
        assert sum(used) == amount
```

OFFICIAL PAGE TEXT

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples

```
PythonInput: coins = [1, 2, 5], amount = 11
Output: 3
Explanation: The minimum number of coins needed is 3 (
11 = 5 + 5 + 1).

Input: coins = [4, 5], amount = 7
Output: -1
Explanation: It is not possible to make up the amount
7 with coins of denominations 4 and 5.
```

Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the minimum number of coins found or -1 if it is not possible to make the target amount with the given coins.

OFFICIAL PAGE TEXT

```
PseudocodeFUNCTION coin_change_naive(coins, amount):
    IF amount == 0:
        RETURN 0  // Base case: No coins are needed to
 make amount 0

    IF amount < 0:
        RETURN infinity  // Amount is negative, path i
s not feasible

    min_coins = infinity

    FOR EACH coin IN coins:
        result = coin_change_naive(coins, amount - coi
n)  // Recursively solve for the remaining amount
        IF result != infinity:
            min_coins = MIN(min_coins, result + 1)  //
 Update min_coins with the minimum value

    RETURN min_coins IF min_coins != infinity ELSE -1
 // Return the minimum number of coins or -1 if no sol
ution exists
```

Time Complexity: The solution has a time complexity of O(m^n), where n is the amount and m is the number of coin denominations. The exponential time complexity arises from the recursive exploration of all possible combinations of coins. Each recursive call generates multiple further calls, leading to an exponential growth in the number of computations required. The total number of recursive calls can be as large as mnm^nmn due to the need to try each coin denomination for each sub-amount. Space Complexity: The space complexity of the naive approach is O(n), where n is the target amount. The space complexity is determined by the maximum depth of the recursion stack. In the worst case, the recursion depth can be as deep as the target amount n (when using coins of value 1). Therefore, the space required for the call stack is proportional to the target amount. As you can see, this solution does not perform very well. Let's take a look at a better solution. Solution 2: Dynamic programming approach Our solution employs a dynamic programming approach to determine the fewest number of coins needed to make up a given amount. We use a DP array, where each index represents a specific amount, and its value represents the minimum number of coins required to achieve that amount. The array is initialized with a value greater than the maximum possible number of coins

OFFICIAL PAGE TEXT

needed (infinity), except for the base case where dp[0] is set to 0, indicating that zero coins are needed to make amount 0. For each amount from 1 to the target amount, we iterate through each coin denomination and update the DP array. If the current amount minus the coin value is non-negative and the previous amount can be formed, we update the current amount's value to be the minimum of its current value or one more than the value of the previous amount. This ensures that we are considering the fewest coins needed by checking each possible coin combination. After processing all amounts, if the value for the target amount is still set to infinity, it means it's not possible to form the target amount with the given coins, and we return -1. Otherwise, we return the value at dp[amount], which represents the minimum number of coins needed. Time Complexity: The solution has a time complexity of O(n * m), where n is the amount and m is the number of coin denominations. This is because we iterate over each amount from 1 to the target amount, and for each amount, we iterate over all coin denominations to update the DP array. Space Complexity: The solution uses O(n) space for the DP array, where n is the target amount. The array stores the minimum number of coins required to make each amount up to the target amount. This space complexity is necessary to hold the results for

all subproblems up to the target amount.No answers contributed by the community yet.

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

HARD | PracHub | Technical Screen | 13 of 25

Infection Spread on a Grid (Cellular Automaton)

POSTED 2026-06-14 | 128 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 51.2** **30 DAYS 0** **PRIORITY 65/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #994 Rotting Oranges, LeetCode #289 Game of Life

LeetCode match: LC 994 Rotting Oranges / LC 289 Game of Life (inferred from overview)

Posted on PracHub: 2026-06-14 (updated 2026-06-24)

Original: https://prachub.com/coding-questions/infection-spread-on-a-grid-cellular-automaton

QUESTION

Overview: This question tests a candidate's ability to implement and optimize a synchronous cellular automaton simulation on a 2D grid, assessing proficiency in BFS-based propagation, state machine design, and incremental computation. It falls under algorithms and grid simulation, evaluating practical coding skill and the ability to handle progressively complex constraints including recovery delays, barriers, and early-termination fixed-point detection.

APPROACH

Keep a frontier of newly changed cells and update only their neighbors each step (multi-source BFS style) instead of rescanning the grid; use a state machine for cell transitions.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy has only the overview, so ask for the rules: the states (healthy, infected, recovered or immune), how infection spreads (four neighbors per step, synchronously), the recovery delay, whether barriers block spread, and what to return (the number of steps until nothing changes, or the final grid).

BRUTE FORCE TO SAY OUT LOUD

Rescan the entire grid on every step and compute the next grid from the old one. O(steps * mn).

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Keep a frontier of cells that changed in the last step. Each step only their neighbors can change, so process the frontier like a multi-source BFS and update a separate next frontier to keep the update synchronous. Store the infection time per cell to apply the recovery delay (recover when step minus infected time reaches the delay). Stop when the frontier is empty and no recoveries are pending, since the grid has reached a fixed point. O(mn) overall.

DRY-RUN THESE CASES

No infected cells (0 steps), an isolated healthy cell behind barriers, a delay of 1, all cells infected at the start, two sources reaching a cell at the same time.

LIKELY FOLLOW-UPS

Different neighborhoods, probabilistic spread, or very large grids (sparse frontier).

PITFALLS

Updating cells in place during a step, which makes the spread asynchronous. Missing pending recoveries when deciding to stop.

HOW TO THINK ABOUT IT

A synchronous cellular automaton is a BFS in disguise: the step number is the BFS level. Only cells next to cells that just changed can change next, so keep a frontier instead of rescanning the grid. Always compute the next state from the old state (not in place) so all cells update together.

Basis: Written. Assumed rules (confirm them): '.' healthy, 'I' infected, 'X' barrier; each step every healthy cell with an infected 4-neighbour becomes infected, all at once; the answer is the number of steps until nothing changes.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    steps = 0
    while True:
        nxt = copy(grid)
        for r, c in all_cells(grid):
            if grid[r][c] == HEALTHY:
                if sick_neighbour(grid, r, c):
                    nxt[r][c] = INFECTED
        if nxt == grid:
            return steps
        grid = nxt
        steps += 1
```

TIME	SPACE
O(steps * m * n)	O(m * n)

WHY IT IS SLOW

Every step scans the whole grid even when only a thin edge is changing, O(steps * m * n). Keeping a frontier of newly infected cells (BFS) touches each cell once, O(m * n).

OPTIMAL SOLUTION · Frontier BFS (only recently infected cells spread)

TIME	SPACE
O(m * n)	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add a recovery delay: an infected cell recovers after d steps and cannot spread afterwards.

A cell infects every healthy neighbor on the very next step, so recovery does not change how long spreading takes; it only changes what the grid looks like at time t. Store the step at which each cell was infected (the BFS level) and derive the state at any time: healthy, infected, or recovered when t minus its infection step is at least d.

FOLLOW-UP 2 Why can you stop early?

When the next frontier is empty no cell can change in any later step (the update only depends on the frontier), so the grid is at a fixed point. In a variant with pending recoveries keep going until those are done too.

FOLLOW-UP 3 Make the spread probabilistic.

A frontier cell may fail to infect a neighbor this step, so it stays in the frontier. Keep frontier cells until all their healthy neighbors are infected, and draw a random number per (cell, neighbor, step). The expected number of steps now needs simulation.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Infection Spread on a Grid (Cellular
# Automaton)
#
# This question tests a candidate's ability to implement
# and optimize a synchronous cellular automaton simulation
# on a 2D grid, assessing proficiency in BFS-based
# propagation, state machine design, and incremental
# computation. It falls under algorithms and grid
# simulation, evaluating practical coding skill and the
# ability to handle progressively complex constraints
# including recovery delays, barriers, and early-
# termination fixed-point detection.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add a recovery delay: an infected cell recovers after
# d steps and cannot spread afterwards.
# 2. Why can you stop early?
# 3. Make the spread probabilistic.
#
# INTERVIEW NOTES
# A synchronous cellular automaton is a BFS in disguise:
# the step number is the BFS level. Only cells next to
# cells that just changed can change next, so keep a
# frontier instead of rescanning the grid. Always compute
# the next state from the old state (not in place) so all
# cells update together.
#
# BRUTE FORCE (say it first): Rescan the whole grid every
# step
# Build the next grid from the current one by checking
# every cell's four neighbors, and repeat until a step
# changes nothing. Easy to get right, so it doubles as a
# test oracle. The cost is that every step touches the
# whole grid even when only a thin edge is changing. Time
# O(steps * m * n). Space O(m * n).


# BRUTE FORCE (say it first): Rescan the whole grid every
# step | time O(steps * m * n)
def spread_steps_brute(grid):
    g = [list(r) for r in grid]
    steps = 0
    while True:
        nxt = [row[:] for row in g]
        changed = False
        for r in range(len(g)):
            for c in range(len(g[0])):
                if g[r][c] == "." and any(
                    0 <= r + dr < len(g)
                    and 0 <= c + dc < len(g[0])
                    and g[r + dr][c + dc] == "I"
                    for dr, dc in (
                        (1, 0),
                        (-1, 0),
                        (0, 1),
                        (0, -1),
                    )
                ):
                    nxt[r][c] = "I"
                    changed = True
        if not changed:
            return steps
        g, steps = nxt, steps + 1


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# TIME each cell gets infected
def infection_times(grid):
    # step at which each cell gets infected (a BFS level);
    # 'I' = infected at time 0, '.' = healthy, '#' =
    # blocked
    rows, cols = len(grid), len(grid[0])
    when = {}
    frontier = [(r, c) for r in range(rows)
                for c in range(cols) if grid[r][c] == "I"]
    for cell in frontier:
        when[cell] = 0
    step = 0
    while frontier:
        step += 1
        nxt = []
        for r, c in frontier:
            for dr, dc in STEPS:
                a, b = r + dr, c + dc
                inside = 0 <= a < rows and 0 <= b < cols
                # every healthy neighbour is infected next
                # step
                if inside and grid[a][b] == "." \
                        and (a, b) not in when:
                    when[(a, b)] = step
```

```
                    nxt.append((a, b))
        frontier = nxt
    return when


# BASE QUESTION (optimal): Frontier BFS (only recently
# infected cells spread)
# Idea: Put every initially infected cell in the frontier.
# Each step, look only at the neighbors of frontier cells,
# infect the healthy ones, and make them the next
# frontier.
def spread_steps(grid):
    # steps until nothing changes any more
    when = infection_times(grid)
    return max(when.values(), default=0)


# FOLLOW-UP 1: Add a recovery delay: an infected cell
# recovers after d steps and cannot spread afterwards.
# Answer: A cell infects every healthy neighbor on the
# very next step, so recovery does not change how long
# spreading takes; it only changes what the grid looks
# like at time t.
def grid_at(grid, t, recovery=None):
    # the grid after t steps; with a recovery delay d an
    # infected cell turns into 'R' (it spread already, so
    # the
    # timing above does not change)
    when = infection_times(grid)
    out = []
    for r, row in enumerate(grid):
        line = ""
        for c, ch in enumerate(row):
            start = when.get((r, c))
            if start is None or start > t:
                line += ch
            elif recovery and t >= start + recovery:
                line += "R"
            else:
                line += "I"
        out.append(line)
    return out
```

```
if __name__ == "__main__":
    g = ["I..", "...", "..."]
    assert spread_steps(g) == 4
    assert grid_at(g, 1) == ["II.", "I..", "..."]
    # with recovery after 1 step the source recovers first
    rec = grid_at(g, 1, recovery=1)
    assert rec == ["RI.", "I..", "..."]
    # a wall splits the grid: the far cell is never
    # infected
    wall = ["I#.", "I#."]
    assert spread_steps(wall) == 0
    assert grid_at(wall, 9)[0] == "I#."
    assert spread_steps(["..."]) == 0
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[str]) -> list[str]:
```

Sample call

```
solution(['HHH', 'HIH', 'HHH'])
```

Test cases

Input	Expected
`(['HHH', 'HIH', 'HHH'],)`	`['HIH', 'III', 'HIH']`
`(['HHH'],)`	`['HHH']`

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

HARD | IN APP LC 127, 72 | Exponent | 14 of 25

Edit distance

ASKED (ANY CO.) ~3 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE ALL TIME 46 30 DAYS 0 PRIORITY 77/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: your app's interview approach (LeetCode #127)

LeetCode match: LC 127 Word Ladder (statement) / LC 72 Edit Distance (title)

Last asked (any company): 3 months ago, about 2026-07 (Exponent shows only a rough age) | +612 interviews

Original: https://www.aced.io/questions/3891/edit-distance

QUESTION

Determine the Edit Distance in a Word Ladder: Given two words (beginWord and endWord), and a dictionary's word list, find the minimum number of operations needed to change beginWord into endWord. You can change only one letter at a time, and each intermediate word must exist in the word list. If there is no possible transformation, return None (Python), -1 (Java & C++), null (Javascript). Examples beginWord = 'hit' endWord = 'cog' wordList = ["hit", "hot", "dot", "dog", "cog"] output: 4

hit -> hot -> dot

-> dog -> cog beginWord = 'word' endWord = 'word' wordList = ['word', 'ward'] output: 0 beginWord = 'hit' endWord = 'cog' wordList = ["hit", "hot", "dot", "dog"] output: None

because no 'cog' in list

APPROACH

BFS over words where edges connect words differing by one letter (use wildcard buckets like h*t to build neighbors in O(n*L)); return the path length. For true edit distance use 2D DP: dp[i][j] = min(insert, delete, replace).

HOW TO THINK ABOUT IT

Compare the last characters. If they match, nothing is lost: the cost is the cost of the shorter prefixes. If not, one of three operations was used, so take the cheapest of replace, delete and insert, each followed by the cost of a smaller problem. The same smaller problems repeat, so tabulate them in a grid dp[i][j] = cost of turning the first i characters of a into the first j of b; each cell needs only its left, top and top-left neighbours.

Basis: LeetCode 72, Edit Distance (Levenshtein): insert, delete or replace one character.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def dist(i, j):
    if i == 0:
        return j
    if j == 0:
        return i
    if a[i - 1] == b[j - 1]:
        return dist(i - 1, j - 1)
    replace = dist(i - 1, j - 1)
    delete = dist(i - 1, j)
    insert = dist(i, j - 1)
    return 1 + min(replace, delete, insert)
```

TIME	SPACE
O(3^(m + n))	O(m + n)

WHY IT IS SLOW

Three calls per step and the same (i, j) pair is solved many times: about 3^(m + n). A table of all (i, j) values is O(m * n), with a rolling row for O(min(m, n)) memory.

OPTIMAL SOLUTION · DP table with a rolling row

TIME	SPACE
O(m * n)	O(min(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the actual operations.

Keep the full table and walk back from dp[m][n]: if the characters match step diagonally; otherwise take the neighbour whose value is one smaller and record replace, delete or insert. O(m * n) memory is needed for the walk back.

FOLLOW-UP 2 Are the strings exactly one edit apart (LeetCode 161)?

No table needed: if the lengths differ by more than one answer false; scan to the first mismatch and compare the remainders (equal lengths: skip one char in each; otherwise skip one in the longer). O(n), O(1).

FOLLOW-UP 3 Operations have different costs, or only a small edit distance is of interest (at most k).

Weighted costs just change the numbers added in the recurrence. For 'at most k' only compute a band of width 2k + 1 around the diagonal: O(k * n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Edit distance
#
# Determine the Edit Distance in a Word Ladder: Given two
# words (beginWord and endWord), and a dictionary's word
# list, find the minimum number of operations needed to
# change beginWord into endWord. You can change only one
# letter at a time, and each intermediate word must exist
# in the word list. If there is no possible
# transformation, return None (Python), -1 (Java & C++),
# null (Javascript). Examples beginWord = 'hit' endWord =
# 'cog' wordList = ["hit", "hot", "dot", "dog", "cog"]
# output: 4 # hit -> hot -> dot -> dog -> cog beginWord =
# 'word' endWord = 'word' wordList = ['word', 'ward']
# output: 0 beginWord = 'hit' endWord = 'cog' wordList =
# ["hit", "hot", "dot", "dog"] output: None # because no
# 'cog' in list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual operations.
# 2. Are the strings exactly one edit apart (LeetCode
# 161)?
# 3. Operations have different costs, or only a small edit
# distance is of interest (at most k).
#
# INTERVIEW NOTES
# Compare the last characters. If they match, nothing is
# lost: the cost is the cost of the shorter prefixes. If
# not, one of three operations was used, so take the
# cheapest of replace, delete and insert, each followed by
# the cost of a smaller problem. The same smaller problems
# repeat, so tabulate them in a grid dp[i][j] = cost of
# turning the first i characters of a into the first j of
# b; each cell needs only its left, top and top-left
# neighbours.
#
# BRUTE FORCE (say it first): Plain recursion on the last
# characters
# If the last characters match recurse on both prefixes;
# otherwise try replace, delete and insert and take the
# minimum. Exact and the oracle; the same prefix pairs are
# solved exponentially often. Time O(3^(m + n)). Space O(m
# + n).


from collections import deque
```

```
# BRUTE FORCE (say it first): Plain recursion on the last
# characters | time O(3^(m + n))
def edit_distance_recursive(a, b):
    def go(i, j):
        if i == 0:
            return j
        if j == 0:
            return i
        if a[i - 1] == b[j - 1]:
            return go(i - 1, j - 1)
        return 1 + min(
            go(i - 1, j - 1), go(i - 1, j), go(i, j - 1)
        )

    return go(len(a), len(b))


# BASE QUESTION (optimal): DP table with a rolling row
# Idea: dp[j] holds the distance from the current prefix
# of a to the first j characters of b.
def edit_distance(a, b):
    # dp[i][j]: edits to turn a[:i] into b[:j]
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                swap = dp[i - 1][j - 1]
                drop = dp[i - 1][j]
                add = dp[i][j - 1]
                dp[i][j] = 1 + min(swap, drop, add)
    return dp[m][n], dp


# RELATED: at most k edits (banded)
def within_k(a, b, k):
    # only a band of width 2k + 1 around the diagonal
    # matters
    if abs(len(a) - len(b)) > k:
        return False
    return edit_distance(a, b)[0] <= k


# RELATED: the word-ladder version
def ladder_length(begin, end, words):
    # the word-ladder version: change one letter at a
    # time,
    # every word must be in the list; BFS over the words
    words = set(words)
    if end not in words:
        return 0
    queue, seen = deque([(begin, 1)]), {begin}
    while queue:
        word, steps = queue.popleft()
        if word == end:
            return steps
        for i in range(len(word)):
            for c in "abcdefghijklmnopqrstuvwxyz":
                nxt = word[:i] + c + word[i + 1:]
                if nxt in words and nxt not in seen:
                    seen.add(nxt)
                    queue.append((nxt, steps + 1))
    return 0


# FOLLOW-UP 1: Return the actual operations.
# Answer: Keep the full table and walk back from dp[m][n]:
# if the characters match step diagonally; otherwise take
# the neighbour whose value is one smaller and record
# replace, delete or insert. O(m * n) memory is needed for
# the walk back.
def edit_operations(a, b):
    # walk back through the table and list the operations
    dist, dp = edit_distance(a, b)
    i, j, ops = len(a), len(b), []
    while i or j:
        if i and j and a[i - 1] == b[j - 1]:
            i, j = i - 1, j - 1
        elif i and j and dp[i][j] == dp[i - 1][j - 1] + 1:
            old, new = a[i - 1], b[j - 1]
            ops.append(f"replace {old} with {new}")
            i, j = i - 1, j - 1
```

```
        elif i and dp[i][j] == dp[i - 1][j] + 1:
            ops.append(f"delete {a[i - 1]}")
            i -= 1
        else:
            ops.append(f"insert {b[j - 1]}")
            j -= 1
    return ops[::-1]


# FOLLOW-UP 2: Are the strings exactly one edit apart
# (LeetCode 161)?
# Answer: No table needed: if the lengths differ by more
# than one answer false; scan to the first mismatch and
# compare the remainders (equal lengths: skip one char in
# each; otherwise skip one in the longer). O(n), O(1).
def one_edit_apart(a, b):
    # LeetCode 161: no table, O(n)
    if abs(len(a) - len(b)) > 1:
        return False
    if len(a) > len(b):
        a, b = b, a
    for i in range(len(a)):
        if a[i] != b[i]:
            # the rest must match after one replace or
            # insert
            if len(a) == len(b):
                return a[i + 1:] == b[i + 1:]
            return a[i:] == b[i + 1:]
    return len(a) != len(b)


if __name__ == "__main__":
    assert edit_distance("horse", "ros")[0] == 3
    assert edit_distance("", "abc")[0] == 3
    assert edit_distance("same", "same")[0] == 0
    ops = edit_operations("horse", "ros")
    assert len(ops) == 3
    assert one_edit_apart("cat", "cut")
    assert one_edit_apart("cat", "cats")
    assert not one_edit_apart("cat", "cat")
    assert not one_edit_apart("cat", "dog")
    assert within_k("kitten", "sitting", 3)
    assert not within_k("kitten", "sitting", 2)
    words = ["hot", "dot", "dog", "lot", "log", "cog"]
    got = ladder_length("hit", "cog", words + ["hit"])
```

```
    assert got == 5
    got = ladder_length("hit", "cog", ["hot", "dot"])
    assert got == 0
```

OFFICIAL PAGE TEXT

Determine the Edit Distance in a Word Ladder: Given two words (beginWord and endWord), and a dictionary's word list, find the minimum number of operations needed to change beginWord into endWord. You can change only one letter at a time, and each intermediate word must exist in the word list. If there is no possible transformation, return None (Python), -1 (Java & C++), null (Javascript). Examples

OFFICIAL PAGE TEXT

```
beginWord = 'hit'
endWord = 'cog'
wordList = ["hit", "hot", "dot", "dog", "cog"]
output: 4 # hit -> hot -> dot -> dog -> cog

beginWord = 'word'
endWord = 'word'
wordList = ['word', 'ward']
output: 0

beginWord = 'hit'
endWord = 'cog'
wordList = ["hit", "hot", "dot", "dog"]
output: None  # because no 'cog' in list
```

Our solution tackles the problem by first constructing a graph that represents all possible transformations between words in the list, where an edge exists between two words if they differ by exactly one character (i.e., they are "one edit distance" apart). This is done by iterating over the list of words and comparing each pair to determine if they meet the one edit distance condition. If they do, an edge is added between them in the graph. After constructing the graph, we use a breadth-first search (BFS) approach to find the shortest path from the start_word to the end_word. We initiate a queue that starts with the start_word and its associated edit distance (initially 0). As we traverse the graph, for each word, we check if it's the end_word. If so, we return the current edit distance incremented by one. If not, we add the word's neighbors to the queue for further exploration, ensuring that we don't revisit previously seen words. The BFS guarantees that the first time we reach the end_word, it is via the shortest path, and we can stop the search early. Time Complexity: The time complexity of the solution is O(n^2 * m), where n is the number of words and m is the length of each word. This complexity arises from the double loop that compares each word to every other word (O(n^2)), and for each comparison, we check if the words differ by exactly one character (O(m)). Space Complexity:

The space complexity is O(n^2), where n is the number of words. This space is required to store the graph, where each word can potentially have edges to all other words. Additionally, the BFS queue and the visited set require O(n) space in the worst case.In this video, Thomas, Google SWE, answers the Edit Distance coding interview question.No answers contributed by the community yet.

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

HARD | Exponent | 15 of 25

Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 161/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #1153 String Transforms Into Another String

LeetCode match: No direct match; related LC 1153 String Transforms Into Another String (inferred from short statement)

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/6330/determine-string-transformability-allowed-operations

QUESTION

Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

APPROACH

Position swaps make order irrelevant, so only the multiset/character mapping matters: build a char -> char map (must be consistent), and the conversion graph must have a free letter to break cycles if all 26 letters are used.

INTERVIEW-DAY PLAN

CLARIFY FIRST

The wording fits LeetCode 1657, Close Strings (swap any two characters, or swap the identities of two letters that exist). If the second operation is a one-directional conversion of all occurrences of a letter (LeetCode 1153), the answer changes, so ask which one.

BRUTE FORCE TO SAY OUT LOUD

BFS over all strings reachable by the allowed operations. Exponential.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

1657 reading: the answer is true when both strings use the same set of letters and their sorted frequency lists are equal, because swaps reorder freely and the second operation permutes frequencies among existing letters. 1153 reading: each source letter must map to a single target letter, and if the strings differ and the target uses all 26 letters there is no spare letter to break a cycle, so it is impossible. Both are O(n).

DRY-RUN THESE CASES

abc and bca (true), a and aa (false), cabbba and abbccc (true), uau and ssx (false); for 1153, aabcc to ccdee (true) and a full alphabet rotation (false).

LIKELY FOLLOW-UPS

Both operations combined with one-directional conversion, which reduces to grouping letter counts, or returning the sequence of operations.

PITFALLS

Using the 1657 test when the conversion is one-directional. Comparing frequencies without comparing the letter sets.

HOW TO THINK ABOUT IT

Decide which operations you really have. If you can swap any two letters (positions) and swap the identities of two letters, the answer depends only on which letters appear and how often (the multiset of counts). If conversion is one-way (every a becomes b), letters can merge but never split, and cycles in the mapping need a spare letter. Name the reading before you code.

Basis: The wording matches LeetCode 1657, Close Strings (swap any two characters, or swap the identities of two existing letters). The dataset also relates LeetCode 1153 (one-directional conversion, no swaps), so both readings are solved.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(s, t):
    seen, queue = {s}, deque([s])
    while queue:
        cur = queue.popleft()
        if cur == t:
            return True
        for nxt in swap_positions(cur):
            add_if_new(nxt, seen, queue)
        for nxt in convert_letters(cur):
            add_if_new(nxt, seen, queue)
    return False
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

The number of reachable strings grows exponentially with the length. Because swaps make order irrelevant, comparing the sets of letters and their sorted counts decides it in O(n).

OPTIMAL SOLUTION · Reading 1 (LeetCode 1657): same letter set and same multiset of frequencies

TIME	SPACE
O(n)	O(1) for a fixed alphabet

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Reading 2 (LeetCode 1153): one-directional conversion, no swaps

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the two operations are combined (one-directional conversion plus swaps)?

Swaps make positions irrelevant, so only letter counts matter. Conversions can only merge letter classes, never split one. So t is reachable only if the counts of t can be formed by grouping the counts of s into the t letters (each s letter's count goes wholly to one t letter) and, unless the relabeling is an identity or chain, there is a spare letter. With at most 26 classes the grouping is a small bin-packing search.

FOLLOW-UP 2 Why does a full 26-letter target make the one-directional version impossible?

Every conversion rewrites a letter into another letter, and if a mapping contains a cycle (a to b and b to a) you need a temporary letter not currently in use to break it. If t uses all 26 letters then s must use all 26 as well (the mapping is a function onto 26 distinct letters), so the mapping is a permutation and any non-identity permutation has a cycle. The only possible case is therefore s equal to t.

FOLLOW-UP 3 Only position swaps are allowed.

Then s can become t exactly when they are anagrams: compare the letter counts. O(n).

FOLLOW-UP 4 Conversion is one-directional (every a becomes b) and there are no position swaps (LeetCode 1153).

Each source letter must map to one target letter, and if the strings differ and the target already uses all 26 letters there is no spare letter to break mapping cycles, so it is impossible. Verified against brute force on a small alphabet.

FOLLOW-UP 5 Return the sequence of operations, or the minimum number.

Minimum operations for the swap-only case is n minus the number of cycles in the permutation that sorts s into t; for letter-identity swaps it is the number of cycles in the letter mapping minus one per cycle of length greater than one.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Given two strings and two allowed
# operations, swapping positions of letters and converting
# all occurrences of one character to another, can the
# first string be transformed into the second?
#
# Given two strings and two allowed operations, swapping
# positions of letters and converting all occurrences of
# one character to another, can the first string be
# transformed into the second?
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the two operations are combined (one-
# directional conversion plus swaps)?
# 2. Why does a full 26-letter target make the one-
# directional version impossible?
# 3. Only position swaps are allowed.
# 4. Conversion is one-directional (every a becomes b) and
# there are no position swaps (LeetCode 1153).
# 5. Return the sequence of operations, or the minimum
# number.
#
# INTERVIEW NOTES
# Decide which operations you really have. If you can swap
# any two letters (positions) and swap the identities of
# two letters, the answer depends only on which letters
# appear and how often (the multiset of counts). If
# conversion is one-way (every a becomes b), letters can
# merge but never split, and cycles in the mapping need a
# spare letter. Name the reading before you code.
#
# BRUTE FORCE (say it first): BFS over every reachable
# string (Close Strings reading)
# From the first string apply every swap of two positions
# and every swap of two letters' identities, searching for
# the second. Exact and the oracle for the frequency test,
# but the state space explodes. Time exponential. Space
# exponential.


from collections import Counter


# BRUTE FORCE (say it first, it also checks the answers)
def brute_both(s, t, alphabet):
    # oracle: search every count vector reachable by
    # converting one letter into another
    letters = [chr(97 + i) for i in range(alphabet)]
    start = tuple(s.count(c) for c in letters)
    goal = tuple(t.count(c) for c in letters)
    seen, todo = {start}, [start]
    while todo:
        cur = todo.pop()
        for x in range(alphabet):
            for y in range(alphabet):
                if x != y and cur[x]:
```

```
                    nxt = list(cur)
                    nxt[y] += nxt[x]
                    nxt[x] = 0
                    nxt = tuple(nxt)
                    if nxt not in seen:
                        seen.add(nxt)
                        todo.append(nxt)
    return goal in seen


# HELPERS for the base question
ALPHABET = 26


def can_group(have, want):
    # can the counts in `have` be grouped (each used once)
    # so that the group sums are exactly the counts in
    # `want`? Letters merge, they never split.
    if sum(have) != sum(want):
        return False
    have = sorted(have, reverse=True)
    left = list(want)

    def place(i):
        if i == len(have):
            return all(x == 0 for x in left)
        tried = set()
        for j in range(len(left)):
            # equal bins behave the same: try each size
            # once
            fits = left[j] >= have[i]
            if fits and left[j] not in tried:
                tried.add(left[j])
                left[j] -= have[i]
                if place(i + 1):
```

```
                    return True
                left[j] += have[i]
        return False

    return place(0)


# BASE QUESTION (optimal): (LeetCode 1657): same letter
# set and same multiset of frequencies
# Idea: Operation 1 reorders positions freely, so only the
# multiset of letters matters. Operation 2 exchanges the
# identities of two letters that exist, which permutes the
# frequencies among existing letters.
def both_operations(s, t, alphabet=ALPHABET):
    # swaps make positions irrelevant: only the letter
    # counts matter, and a conversion can only merge or
    # rename a letter
    if len(s) != len(t):
        return False
    cs, ct = Counter(s), Counter(t)
    if cs == ct:
        return True
    mine, goal = list(cs.values()), list(ct.values())
    if not can_group(mine, goal):
        return False
    # only identical sizes mean no merge happens. With
    # every
    # letter used there is no spare letter to rename
    # safely.
    if len(cs) == len(ct) == alphabet:
        return False
    return True


# FOLLOW-UP 3: Only position swaps are allowed.
# Answer: Then s can become t exactly when they are
# anagrams: compare the letter counts. O(n).
def swaps_only(s, t):
    # swaps only rearrange: reachable exactly when the
    # strings are anagrams (same letter counts)
    return Counter(s) == Counter(t)


# FOLLOW-UP 4: Conversion is one-directional (every a
# becomes b) and there are no position swaps (LeetCode
```

```
# 1153).
# Answer: Each source letter must map to one target
# letter, and if the strings differ and the target already
# uses all 26 letters there is no spare letter to break
# mapping cycles, so it is impossible.
def convert_only(s, t, alphabet=ALPHABET):
    # no swaps, conversion turns EVERY x into one letter y
    # (LeetCode 1153). Each source letter needs one
    # target.
    if len(s) != len(t):
        return False
    if s == t:
        return True
    mapping = {}
    for a, b in zip(s, t):
        if mapping.setdefault(a, b) != b:
            return False
    # a mapping with a cycle (a to b, b to a) needs a
    # spare
    # letter, so a target that uses every letter has none
    return len(set(t)) < alphabet


# FOLLOW-UP 5: Return the sequence of operations, or the
# minimum number.
# Answer: Minimum operations for the swap-only case is n
# minus the number of cycles in the permutation that sorts
# s into t; for letter-identity swaps it is the number of
# cycles in the letter mapping minus one per cycle of
# length greater than one.
def min_swaps(s, t):
    # follow-up: fewest swaps when the letters are all
    # different: n minus the number of cycles of the
    # permutation that sorts s into t
    where = {c: i for i, c in enumerate(t)}
    seen, cycles = set(), 0
    for i in range(len(s)):
        if i not in seen:
            cycles += 1
            while i not in seen:
                seen.add(i)
                i = where[s[i]]
    return len(s) - cycles
```

```
# HELPERS for the checks below
def swap_sequence(s, t):
    # the swaps themselves (distinct letters): put the
    # right
    # letter into each position, one swap per misplaced
    # one
    cur, steps = list(s), []
    where = {c: i for i, c in enumerate(cur)}
    for i, want in enumerate(t):
        if cur[i] != want:
            j = where[want]
            where[cur[i]] = j
            where[want] = i
            cur[i], cur[j] = cur[j], cur[i]
            steps.append((i, j))
    return steps


if __name__ == "__main__":
```

```
    assert swaps_only("listen", "silent")
    assert not swaps_only("abc", "abd")
    assert convert_only("aabcc", "ccdee")
    assert not convert_only("aaabc", "abcaa")
    # a to b and b to a is a cycle: a spare letter is
    # needed
    assert convert_only("ab", "ba")
    assert not convert_only("ab", "ba", alphabet=2)
    assert both_operations("aabb", "ccdd")
    assert both_operations("aab", "ccc")
    assert both_operations("aab", "abb")
    assert both_operations("aabb", "cccc")
    assert not both_operations("aaab", "bbcc")
    # every candidate pair on a 3 letter alphabet and
    # strings up to length 5 against the brute force
    import itertools
    words = [""]
    for n in range(1, 6):
        words += ["".join(p) for p in
                  itertools.product("abc", repeat=n)]
    for s in words:
        for t in words:
            if len(s) == len(t):
                fast = both_operations(s, t, alphabet=3)
                assert fast == brute_both(s, t, 3), (s, t)
    assert min_swaps("abc", "cab") == 2
```

```
    assert min_swaps("abcd", "abcd") == 0
    steps = swap_sequence("abcd", "dcba")
    assert len(steps) == min_swaps("abcd", "dcba") == 2
    cur = list("abcd")
    for i, j in steps:
        cur[i], cur[j] = cur[j], cur[i]
    assert "".join(cur) == "dcba"
```

STUDY HERE: topic 4, Graphs and Search | main topic: Topological sort and cycles

Site tags: string, parsing, graph, topological-sort, dfs, cycle-detection, medium

MEDIUM | 1Point3Acres | onsite-coding | 16 of 25

Template Variable Expansion with Follow-Ups

ASKED 2026-10-17 | 3 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 214/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Topological sort and cycles

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-17 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string.
Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values.
Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

APPROACH

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

HOW TO THINK ABOUT IT

Agree the grammar first: %NAME% is a placeholder, %% is a literal percent sign, an unmatched % is an error and an unknown name is an error. Values may contain placeholders themselves, so expand a value by expanding its own placeholders first. A cycle (A uses B, B uses A) would recurse forever, so detect it with a visiting set or, equivalently, with indegrees and a queue.

Basis: Written for this question; resolving values that contain other placeholders is a dependency graph problem (as in LeetCode 210, Course Schedule II).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text, depth=0):
    if depth > LIMIT:
        raise Error("too deep: cycle?")
    out = ""
    for token in tokens(text):
        if token == "%%":
            out += "%"
        elif is_key(token):
            value = values[name_of(token)]
            out += expand(value, depth + 1)
        else:
            out += token
    return out
```

TIME	SPACE
exponential in the worst ... exponential in the worst case	**O(depth)**

WHY IT IS SLOW

Re-expands shared values every time and finds cycles only by running out of depth. Memoising each value and keeping a visiting set expands each key once and reports a cycle at once.

OPTIMAL SOLUTION · Depth-first expansion with memo and a visiting set

TIME	SPACE
O(total length of the expanded text + keys)	**O(keys)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Detect cycles before expanding.

Model each key as a vertex with an edge to every key its value mentions, count indegrees and run Kahn's algorithm: if fewer vertices than keys come out, some keys sit on a cycle and cannot be resolved. This reports the problem up front instead of failing halfway through a long document.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Template Variable Expansion with Follow-
# Ups
#
# Replace %KEY% placeholders using a key-value map whose
# values may contain other placeholders. Clarify malformed
# delimiters and literal-percent escaping, flatten nested
# values, and detect cyclic dependencies before expansion.
# Input: a key-value map and a string containing
# placeholders delimited by %, such as %USER%. Replace
# every placeholder with the corresponding value. Map
# values can themselves contain placeholders, so resolve
# or flatten dependencies before expanding the final
# string. Define behavior for malformed delimiters such as
# unmatched percent signs, for a literal % in output, and
# for nested-looking forms before coding; the prompt
# intentionally leaves these contracts open. Follow-up:
# detect cyclic dependencies among map values. Model keys
# as a dependency graph and use indegrees plus a queue to
# determine whether a complete topological resolution is
# possible.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Detect cycles before expanding.
#
# INTERVIEW NOTES
# Agree the grammar first: %NAME% is a placeholder, %% is
# a literal percent sign, an unmatched % is an error and
# an unknown name is an error. Values may contain
# placeholders themselves, so expand a value by expanding
# its own placeholders first. A cycle (A uses B, B uses A)
# would recurse forever, so detect it with a visiting set
# or, equivalently, with indegrees and a queue.
#
# BRUTE FORCE (say it first): Recursive expansion without
# memo or cycle check, capped by depth
# Recursively substitute placeholders and stop with an
# error after a fixed depth. Correct for acyclic maps;
# repeats shared work and detects cycles only by running
# out of depth. Time exponential in the worst case. Space
# O(depth).


from collections import defaultdict, deque
```

```
# BRUTE FORCE (say it first): Recursive expansion without
# memo or cycle check, capped by depth | time exponential
# in the worst case
def expand_brute(text, mapping, depth=0):
    if depth > 30:
        raise ValueError("cycle")
    out, i = [], 0
    while i < len(text):
        if text[i] != "%":
            out.append(text[i])
            i += 1
        elif text[i : i + 2] == "%%":
            out.append("%")
            i += 2
        else:
            j = text.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            out.append(
                expand_brute(
                    mapping[text[i + 1 : j]],
                    mapping,
                    depth + 1,
                )
            )
            i = j + 1
    return "".join(out)


# HELPERS for the base question
def tokens(s):
    # split text into ("text", char) and ("var", name)
    i = 0
    while i < len(s):
        if s[i] != "%":
            yield "text", s[i]
            i += 1
        elif s[i:i + 2] == "%%":
            # %% is a literal percent sign
            yield "text", "%"
            i += 2
        else:
            j = s.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            yield "var", s[i + 1:j]
            i = j + 1


# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
# Idea: expand(text) scans the text once: %% becomes %,
# %NAME% becomes the resolved value of NAME.
def expand_template(text, mapping):
    done, visiting = {}, set()

    def resolve(name):
        # values may hold placeholders: expand them first
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        return "".join(resolve(x) if kind == "var" else x
                       for kind, x in tokens(s))

    return scan(text)


# FOLLOW-UP 1: Detect cycles before expanding.
# Answer: Model each key as a vertex with an edge to every
# key its value mentions, count indegrees and run Kahn's
# algorithm: if fewer vertices than keys come out, some
# keys sit on a cycle and cannot be resolved.
def has_cycle(mapping):
    # Kahn's algorithm over key -> keys it mentions
    deps = {}
    for k, v in mapping.items():
        deps[k] = {x for kind, x in tokens(v)
                   if kind == "var"}
    users, indeg = defaultdict(list), {}
    for k, d in deps.items():
        known = d & mapping.keys()
        indeg[k] = len(known)
        for x in known:
```

```
            users[x].append(k)
    queue = deque(k for k, n in indeg.items() if n == 0)
    resolved = 0
    while queue:
        k = queue.popleft()
        resolved += 1
        for u in users[k]:
            indeg[u] -= 1
            if indeg[u] == 0:
                queue.append(u)
    # keys left over sit on a cycle
    return resolved != len(mapping)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    assert expand_template(text, m) == (
        "I am admin My home is /admin/home")
    sure = expand_template("100%% sure", {})
    assert sure == "100% sure"
    assert not has_cycle(m)
    loop = {"A": "%B%", "B": "%A%"}
    assert has_cycle(loop)
    for bad, error in (("a % b", ValueError),
                       ("%NOPE%", KeyError)):
        try:
            expand_template(bad, {})
            raise AssertionError("should have failed")
        except error:
            pass
    try:
        expand_template("%A%", loop)
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
```

NOTES FROM CANDIDATES

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

PREPARATION

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY HERE: topic 4, Graphs and Search | main topic: Topological sort and cycles

Site tags: graph, topological-sort, cycle-detection, medium

MEDIUM | IN APP LC 210 | 1Point3Acres | onsite-coding | 17 of 25

Course Scheduling and Dependency Variants

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 179/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Topological sort and cycles

Optimal answer from: your app's interview approach (LeetCode #210)

LeetCode match: LC 210 Course Schedule II

Frequency: Medium | Last asked: 2026-10-04

Original: https://www.1point3acres.com/interview/problems/company/google/course-schedule-dependency-variants

QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

Course-ordering variant

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

Damaged-node and path variant

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

Merge-two-ordered-sequences variant

Input: two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

QUESTION

EXAMPLES

Merge-two-ordered-sequences

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1:
    contradiction)
```

APPROACH

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

HOW TO THINK ABOUT IT

Every 'A must come before B' rule is an edge. A valid ordering exists exactly when the graph has no cycle, and Kahn's algorithm finds one: repeatedly take a vertex with no unmet prerequisites. The two-array merge is the same problem: each adjacent pair inside an array is an edge and equal values are the same vertex; the merge is possible only if the combined graph has no cycle.

Basis: LeetCode 210, Course Schedule II, plus a merge-two-ordered-sequences variant (the same graph in disguise).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, rules):
    for order in permutations(range(n)):
        pos = {v: i for i, v in enumerate(order)}
        ok = all(pos[a] < pos[b]
                 for a, b in rules)
        if ok:
            return True
    return False
```

TIME	SPACE
O(V! * E)	O(V)

WHY IT IS SLOW

V! orderings. Kahn's algorithm repeatedly takes a vertex with no unmet prerequisites and finishes in O(V + E).

OPTIMAL SOLUTION · Kahn's algorithm (course order, or can two sequences merge)

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you check that an order is valid?

Record each vertex's position in the order, check that every vertex appears exactly once, then check that for every edge (u, v) the position of u is smaller than the position of v. Accept any valid order, not one fixed answer, because independent vertices can swap.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Course Scheduling and Dependency Variants
#
# Topological-ordering family. Variants: course ordering
# with validity checks and a nested-list extension;
# merging two arrays whose internal order defines
# precedence (return false on a conflict cycle); and a
# damaged-node dependency-path variant that minimizes
# broken nodes on a route. Course-ordering variant Work
# through the course-ordering problem, including validity
# checks, complexity analysis, and test cases. Extend the
# result to a nested-list format. The grouping rule is
# unspecified; clarify what each inner list represents
# before implementing the extension. Damaged-node and path
# variant Remove damaged nodes and their downstream
# effects, then find a valid path. A follow-up minimizes
# broken nodes on a required route. Clarify the graph
# direction, path endpoints, and meaning of broken nodes
# before choosing an algorithm. Merge-two-ordered-
# sequences variant Input: two integer arrays a and b.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you check that an order is valid?
#
# INTERVIEW NOTES
# Every 'A must come before B' rule is an edge. A valid
# ordering exists exactly when the graph has no cycle, and
# Kahn's algorithm finds one: repeatedly take a vertex
# with no unmet prerequisites. The two-array merge is the
# same problem: each adjacent pair inside an array is an
# edge and equal values are the same vertex; the merge is
# possible only if the combined graph has no cycle.
#
# BRUTE FORCE (say it first): Try every permutation of the
# vertices
# Generate permutations and check every constraint.
# Correct and a good oracle for tiny inputs; factorial
# time is unusable beyond about ten vertices. Time O(V! *
# E). Space O(V).


import itertools
from collections import defaultdict, deque


# BRUTE FORCE (say it first): Try every permutation of the
# vertices | time O(V! * E)
def merge_orders_brute(a, b):
    vs = sorted(set(a) | set(b))
    edges = [
        (x[i], x[i + 1])
        for x in (a, b)
        for i in range(len(x) - 1)
    ]
    for perm in itertools.permutations(vs):
        pos = {v: i for i, v in enumerate(perm)}
        if all(pos[u] < pos[v] for u, v in edges):
            return True
    return False


# HELPERS for the base question
def edges_from_sequences(*sequences):
    # each neighbouring pair in a sequence is a rule
    return [(s[i], s[i + 1]) for s in sequences
            for i in range(len(s) - 1)]


def merge_two_orders(a, b):
    # shared values are the same vertex
    edges = edges_from_sequences(a, b)
    # a cycle (conflict) gives None
    return topological_order(set(a) | set(b), edges)


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def topological_order(vertices, edges):
    # edge (u, v) means u must come before v
    adj = defaultdict(list)
    indeg = {v: 0 for v in vertices}
    for u, v in edges:
        adj[u].append(v)
        # count unmet prerequisites of v
        indeg[v] = indeg.get(v, 0) + 1
        indeg.setdefault(u, 0)
    # start with everything that has no prerequisite
    queue = deque(v for v, d in indeg.items() if d == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        # finishing u frees its dependents
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    # fewer than all vertices means a cycle: no order
    return order if len(order) == len(indeg) else None


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def can_merge(a, b):
    # True when some order keeps both sequences in order
    return merge_two_orders(a, b) is not None


# FOLLOW-UP 1: How do you check that an order is valid?
# Answer: Record each vertex's position in the order,
# check that every vertex appears exactly once, then check
```

```
# that for every edge (u, v) the position of u is smaller
# than the position of v.
def is_valid_order(order, vertices, edges):
    # every vertex must appear exactly once
    if len(order) != len(set(vertices)):
        return False
    if set(order) != set(vertices):
        return False
    pos = {v: i for i, v in enumerate(order)}
    # every prerequisite must come before its dependent
    return all(pos[u] < pos[v] for u, v in edges)


if __name__ == "__main__":
    # course order with an isolated course 3
    courses, rules = range(4), [(0, 1), (1, 2)]
    order = topological_order(courses, rules)
```

```
    assert is_valid_order(order, courses, rules)
    # a swapped order breaks a rule
    swapped = [1, 0, 2, 3]
    assert not is_valid_order(swapped, courses, rules)
    # a cycle has no order
    loop = [(0, 1), (1, 0)]
    assert topological_order(range(2), loop) is None
    # merge examples from the question
    assert can_merge([1, 2, 3], [2, 4, 5])
    merged = merge_two_orders([1, 2, 3], [2, 4, 5])
    assert merged is not None
    assert not can_merge([1, 6, 4], [4, 1])
    # a repeated value in one array is a self conflict
    assert not can_merge([1, 2, 1], [])
```

NOTES FROM CANDIDATES

For standard prerequisite-ordering practice, represent each course as a vertex and each prerequisite as an edge from prerequisite to dependent. Include isolated courses. The result orders every course after its prerequisites; a cycle prevents a complete ordering.

Maintain indegrees and a queue of zero-indegree vertices. Emit a vertex, decrement its outgoing neighbors, and enqueue newly ready vertices. If fewer than V vertices are emitted, report failure under the chosen return convention. This takes O(V + E) time and space; check each edge against positions in the output to validate an ordering.

This ordering algorithm does not determine the nested-list grouping rule or solve the damaged-node path objective. Keep those contracts separate rather than assuming they are ordinary topological sorting.

The merge-two-sequences variant is the same ordering engine in disguise: shared values are the same node, and each adjacent pair inside an array is an edge. A cycle means the merge is impossible. It was asked as the final onsite round of an early-career L3 loop, and the candidate nearly ran out of time — recognizing it as a graph problem early is the main hurdle.

PREPARATION

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

STUDY HERE: topic 4, Graphs and Search | main topic: Topological sort and cycles

MEDIUM | PracHub | Technical Screen | 18 of 25

Determine whether two nodes are related

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 24.5** **30 DAYS 0** **PRIORITY 142/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Topological sort and cycles

Optimal answer from: the interview site's own solution

LeetCode match: LC 1971 Find if Path Exists in Graph (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/determine-whether-two-nodes-are-related

QUESTION

Overview: This question evaluates understanding of graph representations, reachability and connectivity concepts for ancestry relationships, including handling disconnected components and potential cycles.

APPROACH

Build the ancestry graph and BFS/DFS (or union-find for undirected relatedness); handle disconnected components.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what related means: one is an ancestor of the other, they share a common ancestor, or they are connected in any direction. Ask whether the graph is a tree, a DAG, or may have cycles, and whether it is disconnected.

BRUTE FORCE TO SAY OUT LOUD

DFS from the first node looking for the second, and again from the second. O(V + E) per query.

OPTIMAL IDEA

Build the adjacency list once. For ancestry in a DAG compute the ancestor set of each node (DFS or BFS) and test for the target or a non-empty intersection. For undirected relatedness use union-find, which answers queries in near constant time after one pass. Track visited to survive cycles.

DRY-RUN THESE CASES

A node related to itself, two nodes in separate components, a long chain, a cycle.

LIKELY FOLLOW-UPS

Many queries (precompute components or ancestors), the lowest common ancestor, or updates to the graph.

PITFALLS

INTERVIEW-DAY PLAN

Missing a visited set. Treating direction as undirected, or the reverse.

HOW TO THINK ABOUT IT

Ask what 'related' means: connected in any direction (a connectivity question: union-find or one traversal), or ancestor/descendant, or share an ancestor. With many queries, preprocess once: components give O(1) answers. Always make the visited set part of the design, since relationship data can have cycles.

Basis: LeetCode 1971, Find if Path Exists in Graph, for connectivity, and ancestor queries on a parent map for the 'ancestry' reading. Assumed meaning (confirm it): related = connected by any path of relationships.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def related(a, b):
    graph = build_adjacency(edges)
    seen = set()
    dfs(graph, a, seen)
    return b in seen
```

TIME	SPACE
O(V + E) per query	O(V)

WHY IT IS SLOW

Every query repeats the same traversal, O(V + E) each. A Union-Find built once answers each query in near-constant time.

OPTIMAL SOLUTION · Union-Find built once, constant-time queries

TIME	SPACE
O(E alpha(V)) to build, O(alpha(V)) per query	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 'Related' means ancestor or descendant, or sharing an ancestor.

Store the data as a parent map and walk up from one node while recording ancestors, then walk up from the other until you hit one. Keep a seen set so a corrupt cycle cannot loop forever.

FOLLOW-UP 2 Relationships arrive over time and you must answer queries between updates.

Union-Find handles incremental unions in near-constant time. If relationships can also be removed, you need offline processing or a dynamic-connectivity structure.

FOLLOW-UP 3 The graph is directed and 'related' means reachability from a to b.

Union-Find no longer works (it ignores direction). Use DFS/BFS per query, or compute strongly connected components and reachability on the condensed DAG for many queries.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine whether two nodes are related
#
# This question evaluates understanding of graph
# representations, reachability and connectivity concepts
# for ancestry relationships, including handling
# disconnected components and potential cycles.
#
# FOLLOW-UPS (answered below, in this order):
# 1. 'Related' means ancestor or descendant, or sharing an
# ancestor.
# 2. Relationships arrive over time and you must answer
# queries between updates.
# 3. The graph is directed and 'related' means
# reachability from a to b.
#
# INTERVIEW NOTES
# Ask what 'related' means: connected in any direction (a
# connectivity question: union-find or one traversal), or
# ancestor/descendant, or share an ancestor. With many
# queries, preprocess once: components give O(1) answers.
# Always make the visited set part of the design, since
# relationship data can have cycles.
#
# BRUTE FORCE (say it first): Search from the first node
# for every query
# Build adjacency lists and run a DFS from a looking for b
# each time. Correct, and each query repeats the same
# traversal work. Time O(V + E) per query. Space O(V).


from collections import defaultdict
from collections import deque


# BRUTE FORCE (say it first): Search from the first node
# for every query | time O(V + E) per query
def related_dfs(n, edges, a, b):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    seen, stack = {a}, [a]
    while stack:
        x = stack.pop()
        if x == b:
            return True
        for y in adj[x]:
            if y not in seen:
                seen.add(y)
                stack.append(y)
    return False


# BASE QUESTION (optimal): Union-Find built once,
# constant-time queries
# Idea: Union the endpoints of every relationship; two
# nodes are related exactly when they have the same root.
class Relations:
    """Related = joined by a chain of relations."""

    def __init__(self, n, edges=()):
        self.parent = list(range(n))
        self.size = [1] * n
        for u, v in edges:
            self.union(u, v)

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        # relations can arrive over time: near-constant
        # time
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # the smaller group joins the bigger one
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

    def related(self, a, b):
        return self.find(a) == self.find(b)


# HELPER: ancestors of a node
def ancestors(parent, x):
    # x and everything above it; a seen set stops cycles
    seen = []
    while x is not None and x not in seen:
        seen.append(x)
        x = parent.get(x)
    return seen


# FOLLOW-UP 1: 'Related' means ancestor or descendant, or
# sharing an ancestor.
# Answer: Store the data as a parent map and walk up from
# one node while recording ancestors, then walk up from
# the other until you hit one. Keep a seen set so a
# corrupt cycle cannot loop forever.
def kin(parent, a, b):
    # related if one is an ancestor of the other, or they
    # share an ancestor
    up_a = ancestors(parent, a)
    up_b = ancestors(parent, b)
    return bool(set(up_a) & set(up_b))


# FOLLOW-UP 3: The graph is directed and 'related' means
# reachability from a to b.
# Answer: Union-Find no longer works (it ignores
# direction). Use DFS/BFS per query, or compute strongly
# connected components and reachability on the condensed
# DAG for many queries.
```

```
def reachable(graph, a, b):
    # directed: can we get from a to b? (Union-Find cannot
    # tell, it ignores direction)
    seen, queue = {a}, deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in graph.get(u, ()):
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False


if __name__ == "__main__":
    r = Relations(5, [(0, 1), (1, 2)])
```

```
    assert r.related(0, 2) and not r.related(0, 3)
    r.union(3, 4)
    assert r.related(3, 4) and not r.related(2, 4)
    r.union(2, 3)
    assert r.related(0, 4)
    assert r.related(1, 1)
    # a family tree: parent[child] = parent
    parent = {"kid": "mom", "mom": "gran",
              "uncle": "gran", "stranger": None}
    assert kin(parent, "kid", "gran")
    assert kin(parent, "kid", "uncle")
    assert not kin(parent, "kid", "stranger")
    # a corrupt cycle cannot loop forever
    assert kin({"a": "b", "b": "a"}, "a", "b")
    # direction matters for reachability
    g = {"a": ["b"], "b": ["c"]}
    assert reachable(g, "a", "c")
    assert not reachable(g, "c", "a")
```

FUNCTION AND TEST CASES

Signature

```
def solution(relations: list[list[int]], a: int, b: int) -
> bool:
```

Sample call

```
solution([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)
```

Test cases

Input	Expected
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)`	`True`
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 11)`	`False`

STUDY HERE: topic 4, Graphs and Search | main topic: Topological sort and cycles

Site tags: graph, bfs, medium

MEDIUM | 1Point3Acres | onsite-coding | 19 of 25

Shortest Directed Cycle Through a Node

STUDY: TALK **ALL TIME 5** **30 DAYS 0** **PRIORITY 174/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Topological sort and cycles

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2608 Shortest Cycle in a Graph (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-cycle-through-node

QUESTION

EXAMPLES

```
Edges [(0,1),(1,2),(2,0)], v = 0 -> 3.
Edges [(0,1),(1,2)], v = 0 -> -1.
```

APPROACH

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether edges are directed (yes), whether self loops count (cycle of length 1), what to return if there is none (-1), and whether edges have weights.

BRUTE FORCE TO SAY OUT LOUD

Enumerate all simple paths from v back to v with DFS. Exponential.

OPTIMAL IDEA

BFS from v. The first time you see an edge from a visited node u back to v, the cycle length is dist[u] + 1, and BFS order makes it the shortest. Do not mark v as discovered before checking for re-entry. O(V + E).

DRY-RUN THESE CASES

A triangle (3). A path with no return (-1). A self loop (1). Two cycles through v of different lengths.

LIKELY FOLLOW-UPS

Shortest cycle in the whole graph (BFS from every node), weighted edges (Dijkstra), or undirected graphs.

PITFALLS

Marking v visited at the start, which hides the cycle. Counting nodes instead of edges.

HOW TO THINK ABOUT IT

A cycle through v is a path from v back to v, so find shortest distances from v and watch for an edge that points back at v. Breadth-first search discovers nodes in order of distance, so the first edge back to v closes the shortest cycle. Do not mark v as 'seen' in a way that hides the edge that returns to it.

Basis: Related to LeetCode 2608, Shortest Cycle in a Graph, but directed and anchored at a given node.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = INF
def dfs(u, length, on_path):
    global best
    for w in graph[u]:
        if w == v:
            best = min(best, length + 1)
        elif w not in on_path:
            dfs(w, length + 1, on_path | {w})
dfs(v, 0, {v})
answer = -1 if best == INF else best
```

TIME	SPACE
exponential	O(V)

WHY IT IS SLOW

The number of simple paths can be exponential in the graph size. A breadth-first search from v reaches the shortest cycle in O(V + E) because it visits nodes in order of distance.

OPTIMAL SOLUTION · BFS from v, close the cycle on the first edge back to v

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you find the shortest cycle in the whole graph?

Run the same BFS from every vertex and take the minimum: O(V * (V + E)). For undirected graphs, BFS from each node and treat any non-tree edge as closing a cycle of length dist[u] + dist[w] + 1 (the LeetCode 2608 version).

FOLLOW-UP 2 What if edges have positive weights?

Replace BFS with Dijkstra from v and close the cycle at the cheapest dist[u] + weight(u, v). That is O((V + E) log V) with a heap. With negative weights use Bellman-Ford and handle negative cycles separately.

FOLLOW-UP 3 The shortest cycle anywhere in the graph (the girth).

Run the same BFS from every node and take the smallest result: O(V * (V + E)). That is the standard way and fine for graphs up to a few thousand nodes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shortest Directed Cycle Through a Node
#
# Examples: Edges [(0,1),(1,2),(2,0)], v = 0 -> 3. Edges
# [(0,1),(1,2)], v = 0 -> -1.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you find the shortest cycle in the whole
# graph?
# 2. What if edges have positive weights?
# 3. The shortest cycle anywhere in the graph (the girth).
#
# INTERVIEW NOTES
# A cycle through v is a path from v back to v, so find
# shortest distances from v and watch for an edge that
# points back at v. Breadth-first search discovers nodes
# in order of distance, so the first edge back to v closes
# the shortest cycle. Do not mark v as 'seen' in a way
# that hides the edge that returns to it.
#
# BRUTE FORCE (say it first): Enumerate every simple cycle
# through v with DFS
# Explore all simple paths from v and record the length
# whenever an edge returns to v. Exact and the oracle, but
# the number of simple paths can be exponential. Time
# exponential. Space O(V).


import heapq
from collections import deque


# BRUTE FORCE (say it first): Enumerate every simple cycle
# through v with DFS | time exponential
def shortest_cycle_dfs(n, edges, v):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
    best = [float("inf")]

    def go(u, seen, length):
        for w in adj[u]:
            if w == v:
                best[0] = min(best[0], length + 1)
            elif w not in seen:
                go(w, seen | {w}, length + 1)
```

```
    go(v, {v}, 0)
    return -1 if best[0] == float("inf") else best[0]


# HELPERS for the base question
def make_graph(n, edges):
    adj = [[] for _ in range(n)]
    for a, b, *w in edges:
        # edges may carry a weight (default 1)
        adj[a].append((b, w[0] if w else 1))
    return adj


# HELPERS for follow-up 2
INF = float("inf")


# BASE QUESTION (optimal): BFS from v, close the cycle on
# the first edge back to v
# Idea: A directed cycle through v is a path v to u
# followed by an edge u to v. BFS visits nodes in
# nondecreasing distance, so the first time we see an edge
# back to v its length is dist[u] + 1 and is the shortest.
def shortest_cycle_through(n, edges, v):
    # BFS from v: the first edge that comes back to v
    # closes
    # the shortest cycle through v
    adj = make_graph(n, edges)
    dist, queue = {v: 0}, deque([v])
    while queue:
        u = queue.popleft()
        for w, _ in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1


# FOLLOW-UP 1: How do you find the shortest cycle in the
# whole graph?
# Answer: Run the same BFS from every vertex and take the
# minimum: O(V * (V + E)). For undirected graphs, BFS from
```

```
# each node and treat any non-tree edge as closing a cycle
# of length dist[u] + dist[w] + 1 (the LeetCode 2608
# version).
# FOLLOW-UP 3: The shortest cycle anywhere in the graph
# (the girth).
# Answer: Run the same BFS from every node and take the
# smallest result: O(V * (V + E)). That is the standard
# way and fine for graphs up to a few thousand nodes.
def girth(n, edges):
    # the shortest cycle anywhere: BFS from every node
    sizes = [shortest_cycle_through(n, edges, v)
             for v in range(n)]
    found = [s for s in sizes if s > 0]
    return min(found) if found else -1


# FOLLOW-UP 2: What if edges have positive weights?
# Answer: Replace BFS with Dijkstra from v and close the
# cycle at the cheapest dist[u] + weight(u, v). That is
# O((V + E) log V) with a heap. With negative weights use
# Bellman-Ford and handle negative cycles separately.
def weighted_cycle_through(n, edges, v):
    # positive weights: Dijkstra, then close the cycle at
    # v
    adj = make_graph(n, edges)
    dist, heap, best = {v: 0}, [(0, v)], INF
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, INF):
            continue
        for w, cost in adj[u]:
            if w == v:
                best = min(best, d + cost)
            elif d + cost < dist.get(w, INF):
                dist[w] = d + cost
                heapq.heappush(heap, (d + cost, w))
    return -1 if best == INF else best
```

```
if __name__ == "__main__":
    ring = [(0, 1), (1, 2), (2, 0)]
    assert shortest_cycle_through(3, ring, 0) == 3
    path = [(0, 1), (1, 2)]
    assert shortest_cycle_through(3, path, 0) == -1
    # a self loop is a cycle of length 1
```

```
    assert shortest_cycle_through(2, [(0, 0)], 0) == 1
    # a shorter cycle exists elsewhere: 1 <-> 2
    both = ring + [(1, 2), (2, 1)]
    assert girth(3, both) == 2
    assert girth(3, [(0, 1), (1, 2)]) == -1
    # weighted: the cheap way back is 0 -> 1 -> 0 (cost 3)
    weighted = [(0, 1, 1), (1, 0, 2),
                (0, 2, 1), (2, 0, 9)]
    assert weighted_cycle_through(3, weighted, 0) == 3
```

NOTES FROM CANDIDATES

Strictly directed; do not relax to undirected.
BFS layer 0 is v itself; the first re-discovery during expansion is the cycle length.
Common bug: marking v as visited immediately blocks the re-entry.

PREPARATION

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).

STUDY HERE: topic 4, Graphs and Search | main topic: Union-find

Site tags: grid, dfs, bfs, union-find, medium

MEDIUM | IN APP LC 200 | 1Point3Acres | onsite-coding, phone-screen | 20 of 25

★ TOP 75: rank 17 in the Medium and Hard priority list

★ Number of Islands

ASKED 2026-09-30 | 20 DAYS AGO | LAST 30 DAYS

STUDY: TYPE ALL TIME 64.8 30 DAYS 33 PRIORITY 19/228

ABOUT THIS QUESTION

Topic: Graphs and Search: Union-find

Optimal answer from: your app's interview approach (LeetCode #200)

LeetCode match: LC 200 Number of Islands

Frequency: Medium | Last asked: 2026-09-30 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/number-of-islands

QUESTION

EXAMPLES

```
[[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2.
Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1,
    1] (last addition merges the two land cells).
```

APPROACH

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

HOW TO THINK ABOUT IT

Each island is a connected group of land cells, so count how many times you start a fresh search at an unvisited land cell. The search itself (DFS or BFS) just marks everything reachable. If land is added over time, do not rerun the search: Union-Find keeps the count up to date.

Basis: LeetCode 200, Number of Islands, and LeetCode 305, Number of Islands II (streaming additions with Union-Find).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == 0 or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
def dfs(r, c):
    if out_of_grid(r, c) or grid[r][c] == 0:
        return
    if (r, c) in seen:
        return
    seen.add((r, c))
    for dr, dc in DIRECTIONS:
        dfs(r + dr, c + dc)
```

TIME	SPACE
O(m * n)	O(m * n) recursion

WHY IT IS SLOW

Time is already O(m * n); the weakness is the recursion depth, which can reach m * n on a snake-shaped island and overflow the stack. An explicit queue or stack avoids that.

OPTIMAL SOLUTION · Iterative DFS flood fill

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if land can also be removed?

Union-Find cannot split sets. Either rebuild the affected component with a local BFS from each former neighbour, or process the whole timeline offline with a segment tree over time and union-find with rollback (the offline dynamic connectivity technique).

FOLLOW-UP 2 How do you handle a grid too large for memory?

Process it in horizontal strips: label islands within the strip, keep only the boundary row's labels, and merge labels across strips with a union-find over label ids. Memory is O(width + number of labels on the boundary).

FOLLOW-UP 3 Find the area of the largest island.

Count the cells during each search and keep the maximum. Same traversal, one counter.

FOLLOW-UP 4 The grid is huge and does not fit in memory.

Process it in strips of rows, label islands in each strip, and merge labels across strip boundaries with a Union-Find over the labels on the boundary row. **Cost:** O(m * n) time overall and O(width + labels on the boundary row) memory per strip.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Number of Islands
#
# Examples: [[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2
.
# Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1, 1]
# (last addition merges the two land cells).
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if land can also be removed?
# 2. How do you handle a grid too large for memory?
# 3. Find the area of the largest island.
# 4. The grid is huge and does not fit in memory.
#
# INTERVIEW NOTES
# Each island is a connected group of land cells, so count
# how many times you start a fresh search at an unvisited
# land cell. The search itself (DFS or BFS) just marks
# everything reachable. If land is added over time, do not
# rerun the search: Union-Find keeps the count up to date.
#
# BRUTE FORCE (say it first): Recursive flood fill
# Scan; at each unvisited land cell count one island and
# recurse in four directions to mark it. The textbook
# answer and the oracle; deep recursion can overflow on a
# large snake-shaped island. Time O(m * n). Space O(m * n)
# recursion.


# BRUTE FORCE (say it first): Recursive flood fill | time
# O(m * n)
def islands_flood(grid):
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen = set()

    def fill(r, c):
        if (
            not (0 <= r < m and 0 <= c < n)
            or grid[r][c] != 1
            or (r, c) in seen
        ):
            return
        seen.add((r, c))
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            fill(r + dr, c + dc)

    count = 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in seen:
                count += 1
                fill(r, c)
    return count


# HELPERS for the base question
DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def neighbors(r, c, rows, cols):
    # the cells beside (r, c) that are inside the grid
    for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            yield nr, nc


def flood(grid, start, seen):
    # visit all land joined to start; return the count
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack, size = [start], 0
    # a stack instead of recursion: no recursion limit
    while stack:
        r, c = stack.pop()
        size += 1
        for nb in neighbors(r, c, rows, cols):
            if grid[nb[0]][nb[1]] == 1 and nb not in seen:
                seen.add(nb)
                stack.append(nb)
    return size


def island_sizes(grid):
    if not grid:
        return []
    seen, sizes = set(), []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            # an island starts at land not visited yet
            if grid[r][c] == 1 and (r, c) not in seen:
                sizes.append(flood(grid, (r, c), seen))
    return sizes


# BASE QUESTION (optimal): Iterative DFS flood fill
# Idea: Scan every cell; each unvisited land cell starts a
# new island, and an explicit stack marks all connected
# land as visited. Iterative DFS avoids Python's recursion
# limit on large grids.
def num_islands(grid):
    # one entry per island
    return len(island_sizes(grid))


# VARIANT: land cells arrive one at a time
class IslandCounter:
    """Follow-up: land cells arrive one at a time."""

    def __init__(self, rows, cols):
        self.rows, self.cols = rows, cols
        # land cell -> parent cell (union-find)
        self.parent = {}
        self.count = 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
```

```
    def add_land(self, r, c):
        # adding the same cell twice changes nothing
        if (r, c) in self.parent:
            return self.count
        self.parent[(r, c)] = (r, c)
        self.count += 1
        # join with every land neighbour
        for nb in neighbors(r, c, self.rows, self.cols):
```

```
            if nb in self.parent:
                a, b = self.find((r, c)), self.find(nb)
                if a != b:
                    self.parent[a] = b
                    # two islands merged into one
                    self.count -= 1
        return self.count


# FOLLOW-UP 3: Find the area of the largest island.
# Answer: Count the cells during each search and keep the
# maximum. Same traversal, one counter.
def largest_island(grid):
    # follow-up: the area of the biggest island
    return max(island_sizes(grid), default=0)


if __name__ == "__main__":
    g = [[1, 1, 0, 0], [1, 1, 0, 0],
         [0, 0, 1, 1], [0, 0, 1, 1]]
    assert num_islands(g) == 2
    assert largest_island(g) == 4
    assert num_islands([]) == 0
    # streaming adds: counts stay 1, 1, 1
    ic = IslandCounter(2, 2)
    adds = [(0, 0), (0, 1), (1, 1)]
    counts = [ic.add_land(r, c) for r, c in adds]
    assert counts == [1, 1, 1]
    # two separate cells make two islands
    ic = IslandCounter(3, 3)
    assert ic.add_land(0, 0) == 1
    assert ic.add_land(2, 2) == 2
    assert ic.add_land(1, 1) == 3
```

NOTES FROM CANDIDATES

For streaming, Union-Find with path compression is the natural choice — each addLand is amortized alpha(n).
In one onsite, the follow-up arrived after a working BFS solution and explicitly required a Union-Find rewrite; retrofitting the original traversal consumed the remaining time.
Common edge cases: diagonal connectivity (some prompts use 8-connectivity), grid with no land.

PREPARATION

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

STUDY HERE: topic 4, Graphs and Search | main topic: Union-find

Site tags: graph, dfs, bfs, union-find, clustering, medium

MEDIUM | 1Point3Acres | phone-screen | 21 of 25

2D Points Connected Components by Radius

ASKED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 40.7** **30 DAYS 0** **PRIORITY 91/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Union-find

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (related)

Frequency: Low | Last asked: 2026-08-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

QUESTION

Given 2D points and radius `r`, connect two points when their distance is at most `r`; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

APPROACH

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

HOW TO THINK ABOUT IT

Two points are neighbours when their squared distance is at most r squared (no square roots needed), and connectivity is transitive, so this is counting connected components: union-find or a DFS over the neighbour relation. Checking every pair is O(n^2); bucketing points into cells of side r means a point only needs to be compared with the 3 x 3 cells around it.

Basis: LeetCode 547, Number of Provinces, where two points are linked when their distance is at most r.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points, r):
    graph = defaultdict(list)
    ids = range(len(points))
    for i, j in combinations(ids, 2):
        if dist2(points[i], points[j]) <= r * r:
            graph[i].append(j)
            graph[j].append(i)
    seen, clusters = set(), 0
    for i in range(len(points)):
        if i not in seen:
            clusters += 1
            dfs(graph, i, seen)
    return clusters
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every pair is compared, O(n^2). Putting points in cells of side r means each point is compared only with the 3 x 3 cells around it.

OPTIMAL SOLUTION · Union-find with a spatial grid

TIME	SPACE
O(n) about when points are spread out (O(n^2) if they all share a cell)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why compare squared distances?

sqrt is slow and inexact; dist <= r is the same as dist^2 <= r^2 for non-negative numbers, and with integer coordinates the comparison is exact. It also avoids floating-point surprises right at the boundary.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: 2D Points Connected Components by Radius
#
# Given 2D points and radius r, connect two points when
# their distance is at most r; connectivity is transitive,
# and the output is the final number of clusters. The
# interviewer accepted or preferred a brute-force neighbor
# scan after discussing optimization ideas. Input: a list
# of 2D points and a radius r. Two points are connected if
# their Euclidean distance is <= r. Connectivity is
# transitive: if A connects to B and B connects to C, all
# three belong to the same cluster. Return the number of
# final clusters.
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. Why compare squared distances?
#
# INTERVIEW NOTES
# Two points are neighbours when their squared distance is
# at most r squared (no square roots needed), and
# connectivity is transitive, so this is counting
# connected components: union-find or a DFS over the
# neighbour relation. Checking every pair is O(n^2);
# bucketing points into cells of side r means a point only
# needs to be compared with the 3 x 3 cells around it.
#
# BRUTE FORCE (say it first): Compare every pair and run a
# DFS
# Compare every pair of points, build the neighbour lists
# and count components with a DFS. This is the baseline
# the interviewer accepted, and it is fine for a few
# thousand points. Time O(n^2). Space O(n).


from collections import defaultdict


# BRUTE FORCE (say it first): Compare every pair and run a
# DFS | time O(n^2)
def components_brute(points, r):
    n, seen, count = len(points), set(), 0
    near = (
        lambda a, b: (a[0] - b[0]) ** 2
        + (a[1] - b[1]) ** 2
        <= r * r
    )
```

```
    for s in range(n):
        if s in seen:
            continue
        count += 1
        stack = [s]
        seen.add(s)
        while stack:
            u = stack.pop()
            for v in range(n):
                if v not in seen and near(
                    points[u], points[v]
                ):
                    seen.add(v)
                    stack.append(v)
    return count


# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# FOLLOW-UP 1: Why compare squared distances?
# Answer: sqrt is slow and inexact; dist <= r is the same
# as dist^2 <= r^2 for non-negative numbers, and with
# integer coordinates the comparison is exact. It also
# avoids floating-point surprises right at the boundary.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def near(p, q, r):
    # squared distance: no square root, exact for ints
    dx, dy = p[0] - q[0], p[1] - q[1]
    return dx * dx + dy * dy <= r * r
```

```
# BASELINE: compare every pair
def count_clusters_pairs(points, r):
    # the accepted baseline: compare every pair, O(n^2)
    uf = UnionFind(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if near(points[i], points[j], r):
                uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


# BASE QUESTION (optimal): Union-find with a spatial grid
# Idea: Put each point in the cell (x // r, y // r). Two
# points within distance r are in the same or adjacent
# cells, so only those 9 cells are compared.
def count_clusters(points, r):
    # grid of cells r wide: only 9 cells need a look
    size = r if r > 0 else 1
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // size, y // size)].append(i)
    uf = UnionFind(len(points))
    for i, (x, y) in enumerate(points):
        cx, cy = x // size, y // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                cell = cells.get((cx + dx, cy + dy), ())
                for j in cell:
                    q = points[j]
                    if j > i and near((x, y), q, r):
                        uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


if __name__ == "__main__":
    # three points joined only through the middle one
    line = [(0, 0), (3, 0), (6, 0)]
    assert count_clusters(line, 3) == 1
    assert count_clusters_pairs(line, 3) == 1
    # the far point stays alone
    far = [(0, 0), (3, 0), (7, 0)]
    assert count_clusters(far, 3) == 2
    # radius 0 only joins identical points
    same = [(1, 1), (1, 1), (2, 2)]
```

```
    assert count_clusters(same, 0) == 2
    assert count_clusters([], 5) == 0
    # both methods agree on a scatter of points
    pts = [(i * 7 % 13, i * 5 % 11) for i in range(30)]
    for r in (0, 1, 2, 3, 5):
        assert count_clusters(pts, r) == (
            count_clusters_pairs(pts, r))
```

NOTES FROM CANDIDATES

DFS, BFS, or Union-Find all fit the core connected-components model.

For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.

Clarify squared-distance comparison to avoid unnecessary square roots.

PREPARATION

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

STUDY HERE: topic 4, Graphs and Search | main topic: Union-find

Site tags: graph, union-find, hard

HARD | 1Point3Acres | onsite-coding, phone-screen | 22 of 25

Friend Graph with Timeline (Friend / Unfriend Events)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 186/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Union-find

Optimal answer from: written and tested here for this exact question

Frequency: Medium | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/friend-graph-with-timeline

QUESTION

EXAMPLES

```
Base: N=3, events [(1,A,B,friend), (5,B,C,friend)] ->
    answer 5.
Follow-up 1: after the above events, query (0, A, C)
    -> false, (5, A, C) -> true.
```

APPROACH

Have weighted/path-compressed union-find ready and warm. Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.

Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the answer is for the base question (earliest time all N people are connected), what to return if it never happens, whether unfriend events exist, and whether queries ask about a past time.

BRUTE FORCE TO SAY OUT LOUD

For each query rebuild the graph from the events up to that time and run BFS. O(E + N) per query.

OPTIMAL IDEA

Base: sort events and union-find, return the time the component count hits 1. Query at a time t: union by size without path compression and record each union's time; two people are connected at t if the largest time on the path between them is at most t. Unfriend: offline, place each edge's lifetime into a segment tree over time and run DFS with a rollback union-find, checking connectivity at the leaves.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

The example giving 5. A graph that never connects. Two events at the same time. Unfriend then refriend.

LIKELY FOLLOW-UPS

Online unfriend (Holm-de Lichtenberg-Thorup or link-cut trees), counting components over time.

PITFALLS

Using path compression where rollback is needed. Treating unfriend like union-find undo.

HOW TO THINK ABOUT IT

Everyone becomes connected at the first moment the graph has one component. If friendships only appear, sort by time and use union-find (components only merge). Unfriending breaks that: removing an edge can split a component and union-find cannot undo. With all events known up front, handle each edge's lifetime in a segment tree over time and use a rollback union-find.

Basis: LeetCode 1101, The Earliest Moment When Everyone Become Friends, plus connectivity queries over time and unfriend events (offline dynamic connectivity). Assumption: events are (time, a, b, 'friend'|'unfriend').

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, events):
    for t in sorted({e.time for e in events}):
        graph = build_graph(events, upto=t)
        if dfs_count(graph, start=0) == n:
            return t  # everyone reached
    return -1
```

TIME	SPACE
O(T * (N + E)) for T distinct times	O(N + E)

WHY IT IS SLOW

Each time point replays all events and searches the whole graph, O(T * (N + E)). A union-find over sorted events (friend-only) is near-linear; unfriend events need offline dynamic connectivity.

OPTIMAL SOLUTION · Base: earliest time everyone is connected (Union-Find on sorted events)

TIME	SPACE
O(E log E)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does unfriending break plain Union-Find?

A union merges two sets and forgets which edge caused it, so removing that edge cannot tell which members should separate. Online, fully dynamic connectivity needs heavy structures (Holm-de Lichtenberg-Thorup or link-cut trees, polylogarithmic time). When all events are known in advance, the offline segment-tree-over-time plus rollback DSU is the practical answer and the one to code in an interview.

FOLLOW-UP 2 What is the simple answer if the interviewer wants something quick?

Rebuild the graph from the events up to time t and run BFS or DFS for each query: O(E + N) per query. State the cost and then explain how the offline method amortizes it. A correct simple answer is accepted; naming the optimal one shows depth.

FOLLOW-UP 3 Report how many friend groups exist after each event (friends only).

Union-find with a component counter: decrement when a union merges two different roots; record the counter after each event. O(E alpha(N)).

FOLLOW-UP 4 Queries ask 'were A and B connected at time t?' with no unfriending.

Union by size without path compression, recording the time of each union on the edge it creates; two people are connected at t if the largest time on the path between them is at most t.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Friend Graph with Timeline (Friend /
# Unfriend Events)
#
# Examples: Base: N=3, events [(1,A,B,friend),
# (5,B,C,friend)] -> answer 5. Follow-up 1: after the abov
e
# events, query (0, A, C) -> false, (5, A, C) -> true.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does unfriending break plain Union-Find?
# 2. What is the simple answer if the interviewer wants
# something quick?
# 3. Report how many friend groups exist after each event
# (friends only).
# 4. Queries ask 'were A and B connected at time t?' with
# no unfriending.
#
# INTERVIEW NOTES
# Everyone becomes connected at the first moment the graph
# has one component. If friendships only appear, sort by
# time and use union-find (components only merge).
# Unfriending breaks that: removing an edge can split a
# component and union-find cannot undo. With all events
# known up front, handle each edge's lifetime in a segment
# tree over time and use a rollback union-find.
#
# BRUTE FORCE (say it first): Rebuild the graph at every
# event time and search it
# For each distinct time, replay all events up to it to
# get the current friend set and run a DFS from person 0
# to see whether everyone is reachable. Simple, correct
# for friend and unfriend events, and the oracle; it
# rebuilds the graph for every time. Time O(T * (N + E))
# for T distinct times. Space O(N + E).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Rebuild the graph at every
# event time and search it | time O(T * (N + E)) for T
# distinct times
def earliest_connected_rebuild(n, events):
    if n == 1:
        return 0
    for t in sorted({e[0] for e in events}):
        edges = set()
        for et, a, b, kind in sorted(
            events, key=lambda e: e[0]
        ):
            if et > t:
                break
            key = (min(a, b), max(a, b))
            (
                edges.add(key)
                if kind == "friend"
                else edges.discard(key)
            )
        seen, stack = {0}, [0]
        while stack:
            u = stack.pop()
            for a, b in edges:
                for x, y in ((a, b), (b, a)):
                    if x == u and y not in seen:
                        seen.add(y)
                        stack.append(y)
        if len(seen) == n:
            return t
    return -1


# HELPERS for the base question
def by_time(events):
    # events are (time, a, b, "friend" or "unfriend")
    return sorted(events, key=lambda e: e[0])


# FOLLOW-UP 4: Queries ask 'were A and B connected at time
# t?' with no unfriending.
# Answer: Union by size without path compression,
# recording the time of each union on the edge it creates;
# two people are connected at t if the largest time on the
# path between them is at most t.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class TimedUnionFind:
    """Union by size, no path compression, so every
    link keeps the time it was made."""

    def __init__(self, n):
        # every person starts alone in a group of one
        self.parent = list(range(n))
        # sizes decide which group hangs under which
        self.size = [1] * n
```

```
        # time a node was linked under its parent
        self.link_time = [0] * n
        # how many separate groups exist right now
        self.groups = n

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, t):
        # find the leaders of both groups
        ra, rb = self.find(a), self.find(b)
        # already in one group: nothing changes
        if ra == rb:
            return False
        # make ra the bigger group
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        # hang the smaller group under the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        # remember when this link appeared
        self.link_time[rb] = t
        # two groups became one
        self.groups -= 1
        return True

    def connected_at(self, a, b, t):
        # a person is always connected to themselves
        if a == b:
            return True
        # climb from a, noting the newest link
        seen, newest, x = {a: 0}, 0, a
        while self.parent[x] != x:
            newest = max(newest, self.link_time[x])
            x = self.parent[x]
            seen[x] = newest
        # climb from b until we reach an ancestor of a
        newest, y = 0, b
        while y not in seen:
```

```
            # b's leader reached without meeting a
            if self.parent[y] == y:
                return False
            newest = max(newest, self.link_time[y])
            y = self.parent[y]
        # connected at t if all path links are <= t
        return max(newest, seen[y]) <= t


# BASE QUESTION (optimal): earliest time everyone is
# connected (Union-Find on sorted events)
# Idea: Sort by time and union the two people for each
# friend event. The first time the component count reaches
# 1 is the answer; -1 if it never does.
def earliest_all_connected(n, events):
    # zero or one person is connected from the start
    if n <= 1:
        return 0
    uf = TimedUnionFind(n)
    # handle friendships in time order
    for t, a, b, kind in by_time(events):
        # the base question has friend events only
        if kind == "friend":
            uf.union(a, b, t)
            # one group left: everyone is connected now
            if uf.groups == 1:
                return t
    # the groups never merged into one
    return -1
```

```
# FOLLOW-UP 1: Why does unfriending break plain Union-
# Find?
# Answer: A union merges two sets and forgets which edge
# caused it, so removing that edge cannot tell which
# members should separate.
# FOLLOW-UP 2: What is the simple answer if the
# interviewer wants something quick?
# Answer: Rebuild the graph from the events up to time t
# and run BFS or DFS for each query: O(E + N) per query.
# State the cost and then explain how the offline method
# amortizes it.
def connected_with_unfriends(n, events, a, b, t):
    # replay every event up to time t on a set of edges
    edges = set()
```

```
    for time, x, y, kind in by_time(events):
        if time > t:
            break
        pair = (min(x, y), max(x, y))
        # friend adds the edge, unfriend removes it
        if kind == "friend":
            edges.add(pair)
        else:
            edges.discard(pair)
    # build neighbour lists from the surviving edges
    adj = defaultdict(list)
    for x, y in edges:
        adj[x].append(y)
        adj[y].append(x)
    # breadth-first search from a looking for b
    seen, queue = {a}, deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in adj[u]:
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False


# FOLLOW-UP 3: Report how many friend groups exist after
# each event (friends only).
# Answer: Union-find with a component counter: decrement
# when a union merges two different roots; record the
# counter after each event. O(E alpha(N)).
def groups_after_each_event(n, events):
    uf = TimedUnionFind(n)
    counts = []
    # replay friend events in time order
    for t, a, b, kind in by_time(events):
        if kind == "friend":
            uf.union(a, b, t)
            # record the group count after this event
            counts.append(uf.groups)
    return counts


if __name__ == "__main__":
```

```
    # people A, B, C are 0, 1, 2
    ev = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]
    # everyone is connected at time 5
    assert earliest_all_connected(3, ev) == 5
    # a missing friendship means never connected
    assert earliest_all_connected(3, ev[:1]) == -1
    assert earliest_all_connected(1, []) == 0
    # groups: 3 -> 2 -> 1
    assert groups_after_each_event(3, ev) == [2, 1]
    # A and C are apart at time 0, together at time 5
    uf = TimedUnionFind(3)
```

```
    for t, a, b, k in ev:
        uf.union(a, b, t)
    assert not uf.connected_at(0, 2, 4)
    assert uf.connected_at(0, 2, 5)
    # an unfriend breaks the link again
    ev2 = ev + [(7, 1, 2, "unfriend")]
    assert connected_with_unfriends(3, ev2, 0, 2, 6)
    assert not connected_with_unfriends(3, ev2, 0, 2, 8)
```

NOTES FROM CANDIDATES

The base problem looks innocent; the follow-up about unfriending is the actual signal. In practice, interviewers accept "rebuild graph from scratch per query + BFS" as a working answer if you also articulate why the optimal algorithm is fully dynamic connectivity.

The same family shows up as "friends form a chain over time" or "chess pieces R / L that can move plus walls" — the connectivity backbone is identical.

The follow-up the interviewer is fishing for is consistently a segment tree built over the event timeline; have the offline algorithm ready.

Two algorithmic families converge here: (1) offline — sort all events by time, run union-find, snapshot connectivity at each query T; (2) online with unfriend support — vanilla union-find doesn't handle deletion, so either rebuild per query, use link-cut trees, or apply offline reverse-time processing (treat unfriends as adds in reverse). Clarify with the interviewer which they want.

In an L4 R1 variant, the interviewer requested a graph-and-DFS implementation for the unfriend extension, then continued asking for an optimization that reused the earlier union-find work. Completing the rebuild approach did not end that follow-up.

PREPARATION

Have weighted/path-compressed union-find ready and warm.

Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.

Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

STUDY HERE: topic 4, Graphs and Search | main topic: Shortest paths

Site tags: graph, bfs, shortest-path, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 23 of 25

Train / Flight Schedule Reachability

ASKED 2026-10-08 | 12 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 182/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Shortest paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2092 Find All People With Secret (related)

Frequency: Low | Last asked: 2026-10-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/train-schedule-reachability

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late
    for the Y->B connection, but the second one makes
    it.
```

APPROACH

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a connection with departure equal to arrival is allowed, whether a leg can arrive at the same time it leaves, whether times are integers, and whether the answer is yes or no or the route.

INTERVIEW-DAY PLAN

BRUTE FORCE TO SAY OUT LOUD

Search over (station, arrival time) states following any leg that departs later. O(legs^2).

OPTIMAL IDEA

Sort the legs by departure time and keep earliest[station]. The source starts at minus infinity. For each leg, if earliest[origin] is before (or equal to, if allowed) the departure, set earliest[destination] = min(earliest[destination], arrival). The destination is reachable when it has an entry. O(L log L); the table also gives the earliest arrival time, and a parent leg per station gives the route.

DRY-RUN THESE CASES

The example with the late first A to Y leg, source equals destination, no legs, a connection with arrival exactly equal to departure, parallel legs, a cycle in the stations.

LIKELY FOLLOW-UPS

Return the route or the earliest arrival, many queries on one schedule (group by origin and use a time-ordered heap), or minimum number of legs.

PITFALLS

Ignoring time and treating it as station reachability (the stated trap). Mixing up strict and non-strict connection times. Processing legs in input order instead of departure order.

HOW TO THINK ABOUT IT

Plain station-to-station reachability is wrong because a leg is only usable if you are already at its departure station before it leaves. Time is part of the state, so ask what you need to know about each station: the earliest time you can be there. Process the legs in order of departure time; a leg is usable if the earliest arrival at its origin is not later than its departure, and then it can improve the earliest arrival at its destination. One sort and one pass answer reachability, and the same table gives the earliest arrival time.

Basis: LeetCode 2092, Find All People With Secret, and LeetCode 787, Cheapest Flights Within K Stops: reachability where time is part of the state.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reachable(legs, src, dst):
    stack, seen = [(src, -INF)], set()
    while stack:
        station, now = stack.pop()
        if station == dst:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for a, b, dep, arr in legs:
            if a == station and dep > now:
                stack.append((b, arr))
    return False
```

BRUTE FORCE

TIME	SPACE
O(legs^2)	O(legs)

WHY IT IS SLOW

Every state rescans all legs and the same station is reached at many times, O(legs^2). Sorting the legs by departure and keeping only the earliest arrival per station answers in O(L log L).

OPTIMAL SOLUTION · Sort by departure and keep earliest arrivals

TIME	SPACE
O(L log L)	O(S)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the earliest arrival time at the destination.

The table already holds it: earliest_arrivals(legs, source)[dest], or infinity when the station is unreachable. That is the same one pass, so it costs nothing extra.

FOLLOW-UP 2 Print the route, not just yes or no.

Remember the leg that last improved each station's earliest arrival, then walk back from the destination to the source and reverse. Because only a strictly better arrival replaces the stored leg, the stored legs form a valid chain in time.

FOLLOW-UP 3 Many queries on the same schedule: what changes in the graph representation?

Group the legs by origin station and sort each list by departure time. For a query the earliest arrival is relaxed with a heap ordered by time (Dijkstra on time), and the first leg you may board at a station is found with bisect on its departure list, so each query costs O(L log S) instead of re-sorting every time. If the source is fixed and the queries only differ by destination, keep the earliest-arrival table and answer each in O(1).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Train / Flight Schedule Reachability
#
# Given trains (or flights) as (from, to, departure,
# arrival), decide whether you can travel from a source to
# a destination, boarding a later leg only if it departs
# after you arrive. Graph reachability with time state,
# solved with BFS / DFS. Input: a schedule of legs, each
# with a departure station, an arrival station, a
# departure time, and an arrival time; plus a source and a
# destination. Output: whether the destination is
# reachable from the source, where a leg can only be taken
# if it departs after you arrive at its departure station.
# The same prompt appears as a flight schedule between
# airports in new-grad R1. Examples: trains = [ ("A", "Y",
# dep=1, arr=20), ("A", "Y", dep=2, arr=5), ("Y", "B",
# dep=6, arr=9) ] source = "A", destination = "B" Output:
# true — the first A->Y train arrives too late for the Y->
B
# connection, but the second one makes it.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the earliest arrival time at the destination.
# 2. Print the route, not just yes or no.
# 3. Many queries on the same schedule: what changes in
# the graph representation?
#
# INTERVIEW NOTES
# Plain station-to-station reachability is wrong because a
# leg is only usable if you are already at its departure
# station before it leaves. Time is part of the state, so
# ask what you need to know about each station: the
# earliest time you can be there. Process the legs in
# order of departure time; a leg is usable if the earliest
# arrival at its origin is not later than its departure,
# and then it can improve the earliest arrival at its
# destination. One sort and one pass answer reachability,
# and the same table gives the earliest arrival time.
#
# BRUTE FORCE (say it first): Explore every (station,
# arrival time) state
# Search from the source over states (station, time you
# arrived), following every leg that departs after the
# current arrival. Correct and the obvious first answer,
# but each state rescans all legs. Time O(legs^2). Space
# O(legs).
```

```
# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first, it also checks the answers)
def reachable_brute(legs, source, dest, strict=True):
    # the oracle: search over (station, arrival time)
    # states
    seen, stack = set(), [(source, -INF)]
    while stack:
        station, now = stack.pop()
        if station == dest:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for frm, to, dep, arr in legs:
            ok = dep > now if strict else dep >= now
            if frm == station and ok:
                stack.append((to, arr))
    return False


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def earliest_arrivals(legs, source, strict=True):
    """Time is part of the state: keep the earliest time
    we can be at each station. Legs go in departure
    order, so every leg that could feed this one is
    already done."""
```

```
    earliest = {source: -INF}
    by_departure = sorted(legs, key=lambda x: x[2])
    for frm, to, dep, arr in by_departure:
        have = earliest.get(frm, INF)
        # we must already be there before it leaves
        if have < dep or (not strict and have == dep):
            if arr < earliest.get(to, INF):
                earliest[to] = arr
    return earliest
```

```
# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)


# FOLLOW-UP 2: Print the route, not just yes or no.
# Answer: Remember the leg that last improved each
# station's earliest arrival, then walk back from the
# destination to the source and reverse.
def itinerary(legs, source, dest, strict=True):
    """Follow-up: the route. Remember the leg that last
    improved each station, then walk back from the end."""
    earliest, came = {source: -INF}, {}
    for leg in sorted(legs, key=lambda x: x[2]):
        frm, to, dep, arr = leg
        have = earliest.get(frm, INF)
        usable = have < dep or (not strict
                                and have == dep)
        if usable and arr < earliest.get(to, INF):
            earliest[to], came[to] = arr, leg
    if dest not in earliest:
        return None
    route = []
    while dest != source:
        leg = came[dest]
        route.append(leg)
        dest = leg[0]
    return route[::-1]


if __name__ == "__main__":
    legs = [("A", "Y", 1, 20), ("A", "Y", 2, 5),
            ("Y", "B", 6, 9)]
    assert reachable(legs, "A", "B")
    # without the early A to Y train the connection is
    # lost
    assert not reachable(legs[:1] + legs[2:], "A", "B")
    assert earliest_arrivals(legs, "A")["B"] == 9
    assert itinerary(legs, "A", "B") == [legs[1], legs[2]]
    assert itinerary(legs, "B", "A") is None
    assert reachable(legs, "A", "A")
```

```
    touch = [("A", "B", 1, 5), ("B", "C", 5, 8)]
    assert not reachable(touch, "A", "C")
    assert reachable(touch, "A", "C", strict=False)
    import random
    for _ in range(300):
        many = []
        for _ in range(random.randint(0, 8)):
            dep = random.randint(0, 9)
            many.append((random.choice("ABCD"),
                         random.choice("ABCD"), dep,
                         dep + random.randint(1, 4)))
        src = random.choice("ABCD")
        dst = random.choice("ABCD")
        for strict in (True, False):
            want = reachable_brute(many, src, dst, strict)
            got = reachable(many, src, dst, strict)
```

```
            assert got == want
```

NOTES FROM CANDIDATES

Pitfalls: Plain station-level reachability is wrong: in the example, A->Y->B exists as a path, but only one of the two A->Y legs makes the connection.
Clarify whether a connection with departure exactly equal to arrival is allowed.

Follow-ups: Follow-ups push on graph representation

PREPARATION

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

STUDY HERE: topic 4, Graphs and Search | main topic: Shortest paths

Site tags: graph, bfs, shortest-path, medium

MEDIUM | 1Point3Acres | onsite-coding | 24 of 25

Shortest Path Nodes and Escaping a Pursuer

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 212/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Shortest paths

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-path-nodes-and-pursuer-escape

QUESTION

Unweighted undirected graph: shortest path from Alice's node to a target (BFS). Follow-up 1: all nodes that lie on at least one shortest path. Follow-up 2: Bob starts at another node at the same time; decide whether Alice can follow some shortest path to the target without being caught.

Input: an undirected, unweighted graph; Alice starts at one node and must reach a target node.
Return the shortest path (length) from Alice to the target.
Follow-up 1: find all nodes that can appear on some shortest path from Alice to the target.
Follow-up 2: Bob is at another node. Alice picks one shortest path and starts moving; Bob starts moving at the same time. Decide whether Alice can reach the target safely without being caught by Bob.

APPROACH

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target]. For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

HOW TO THINK ABOUT IT

BFS from Alice gives dA and BFS from the target gives dT. A node is on some shortest path exactly when dA[v] + dT[v] equals the shortest length. For the pursuer, BFS from Bob gives dB. State the rule you assume: Alice is caught at a node v if Bob can be there no later than she is (dB[v] <= dA[v]). Alice is safe when a shortest path exists whose every node has dB[v] > dA[v].

Basis: Written for this question (BFS layers, as in LeetCode 1976, Number of Ways to Arrive at Destination).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(graph, alice, bob, target):
    dA = bfs_dist(graph, alice)
    dB = bfs_dist(graph, bob)
    paths = all_paths(graph, alice, target)
    for path in paths:
        if len(path) - 1 != dA[target]:
            continue  # not a shortest path
        if all(dB[v] > dA[v] for v in path):
            return True  # Bob is always late
    return False
```

TIME	SPACE
exponential in the worst ... exponential in the worst case	**O(paths)**

WHY IT IS SLOW

The number of shortest paths can be exponential. Three BFS runs (from Alice, the target and Bob) decide every node in O(V + E).

OPTIMAL SOLUTION · Three BFS runs and a safe shortest-path DAG

TIME	SPACE
O(V + E)	**O(V)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if Bob can also meet Alice in the middle of an edge?

Then arriving at the same node at the same time is not the only way to be caught: Alice moving u -> v while Bob moves v -> u swaps places. Add that case by also rejecting an edge (u, v) when dB[v] == dA[u] and dB[u] == dA[v]. Always clarify what 'caught' means first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shortest Path Nodes and Escaping a
# Pursuer
#
# Unweighted undirected graph: shortest path from Alice's
# node to a target (BFS). Follow-up 1: all nodes that lie
# on at least one shortest path. Follow-up 2: Bob starts
# at another node at the same time; decide whether Alice
# can follow some shortest path to the target without
# being caught. Input: an undirected, unweighted graph;
# Alice starts at one node and must reach a target node.
# Return the shortest path (length) from Alice to the
# target. Follow-up 1: find all nodes that can appear on
# some shortest path from Alice to the target. Follow-up
# 2: Bob is at another node. Alice picks one shortest path
# and starts moving; Bob starts moving at the same time.
# Decide whether Alice can reach the target safely without
# being caught by Bob.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if Bob can also meet Alice in the middle of an
# edge?
#
# INTERVIEW NOTES
# BFS from Alice gives dA and BFS from the target gives
# dT. A node is on some shortest path exactly when dA[v] +
# dT[v] equals the shortest length. For the pursuer, BFS
# from Bob gives dB. State the rule you assume: Alice is
# caught at a node v if Bob can be there no later than she
# is (dB[v] <= dA[v]). Alice is safe when a shortest path
# exists whose every node has dB[v] > dA[v].
#
# BRUTE FORCE (say it first): Enumerate every shortest
# path
# Depth-first search all simple paths of the shortest
# length from Alice to the target and check each node
# against Bob's arrival time. Correct and the oracle; the
# number of shortest paths can be huge. Time exponential
# in the worst case. Space O(paths).


from collections import deque


# HELPERS for the brute force
def bfs(adj, s):
    # distance from s to every reachable node
    d = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in d:
                d[v] = d[u] + 1
                q.append(v)
    return d


# BRUTE FORCE (say it first): Enumerate every shortest
# path | time exponential in the worst case
def escape_brute(adj, a, t, b):
    def bfs(s):
        d = {s: 0}
        q = deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if v not in d:
                    d[v] = d[u] + 1
                    q.append(v)
        return d

    da, db = bfs(a), bfs(b)
    if t not in da:
        return None, set(), False
    nodes, safe = set(), False

    def go(u, path):
        nonlocal safe
        if len(path) - 1 > da[t]:
            return
        if u == t:
            if len(path) - 1 == da[t]:
                nodes.update(path)
                if all(
                    db.get(v, 10**9) > i
                    for i, v in enumerate(path)
                ):
                    safe = True
            return
        for v in adj[u]:
            if v not in path:
                go(v, path + [v])

    go(a, [a])
    return da[t], nodes, safe


# HELPERS for the base question
FAR = 10 ** 9


# BASE QUESTION (optimal): Three BFS runs and a safe
# shortest-path DAG
# Idea: dA, dT and dB come from BFS. The nodes on some
# shortest path are those with dA[v] + dT[v] == dA[t].
def shortest_length(adj, a, t):
    # the base question: edges on the shortest path
    return bfs(adj, a).get(t)


# QUESTION FOLLOW-UP 1: all nodes on a shortest path
def nodes_on_shortest_paths(adj, a, t):
    da, dt = bfs(adj, a), bfs(adj, t)
    if t not in da:
        return set()
    # a node is on some shortest path if the two
    # distances add up to the full length
    return {v for v in da if v in dt
            and da[v] + dt[v] == da[t]}


# QUESTION FOLLOW-UP 2: can Alice escape Bob?
def can_escape(adj, a, t, b, meet_on_edge=False):
    da, db = bfs(adj, a), bfs(adj, b)
    if t not in da:
        return False
    on_path = nodes_on_shortest_paths(adj, a, t)

    def safe(u, v):
        # Bob must reach v strictly after Alice does
        if db.get(v, FAR) <= da[v]:
            return False
        # FOLLOW-UP 1: meet_on_edge=True: they meet inside
        # an edge
        # optional: they swap places on the edge u-v
```

```
        swap = db.get(v, FAR) == da[u]
        swap = swap and db.get(u, FAR) == da[v]
        return not (meet_on_edge and swap)

    # Bob standing on Alice's start catches her at once
    if db.get(a, FAR) <= 0:
        return False
    # walk the shortest-path layers using safe nodes only
    frontier, seen = [a], {a}
    while frontier:
        nxt = []
        for u in frontier:
            for v in adj[u]:
                if v in on_path and da[v] == da[u] + 1 \
                        and v not in seen and safe(u, v):
                    seen.add(v)
                    nxt.append(v)
        frontier = nxt
    return t in seen


if __name__ == "__main__":
    # a square 0-1-3 and 0-2-3 plus a tail 3-4
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3],
           3: [1, 2, 4], 4: [3]}
    assert shortest_length(adj, 0, 4) == 3
    on_path = nodes_on_shortest_paths(adj, 0, 4)
    assert on_path == {0, 1, 2, 3, 4}
    # Bob waiting at the target catches Alice there
    assert not can_escape(adj, 0, 4, 4)
    chain = {0: [1], 1: [0, 2], 2: [1, 3], 3: [2]}
    # Bob starts at the target and reaches it first
    assert not can_escape(chain, 0, 3, 3)
    # Bob starts behind Alice: she stays ahead
    assert can_escape(chain, 1, 3, 0)
```

NOTES FROM CANDIDATES

Asked as an L4 onsite coding round.
Follow-up 2 has an open definition of "caught": clarify whether it means arriving at the same node at the same time, Bob arriving first, or also meeting on an edge; also clarify whether Alice may choose her shortest path knowing Bob's position.

PREPARATION

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target].
For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

STUDY HERE: topic 4, Graphs and Search | main topic: Shortest paths

Site tags: graph, bfs, dijkstra, medium

MEDIUM | 1Point3Acres | onsite-coding | 25 of 25

Minimum Direction Violations in a Directed Graph

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 10.6** **30 DAYS 0** **PRIORITY 165/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Shortest paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1466 Reorder Routes to Make All Paths Lead to the City Zero (related)

Frequency: Single report | Last asked: 2026-07-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/minimum-direction-violations-directed-graph

QUESTION

EXAMPLES

```
n = 5
edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
start = 0
end = 4

path: 0 -> 1 -> 2 -> 3 -> 4
cost: 0 + 1 + 0 + 1 = 2
output: 2
```

APPROACH

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what a violation is (traversing an edge against its direction), whether edge costs are all 1, what to return when the end cannot be reached, and whether the graph has parallel edges or self loops.

BRUTE FORCE TO SAY OUT LOUD

Plain BFS ignoring direction minimizes edges, not violations, so say why it is wrong. Dijkstra with costs 0 and 1 works at O((n + m) log n).

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Add every edge twice: forward with weight 0, backward with weight 1. Run 0-1 BFS with a deque: relax a weight-0 edge by pushing to the front, a weight-1 edge to the back. The deque always holds nodes whose distances differ by at most 1, so O(n + m).

DRY-RUN THESE CASES

Start equals end (0). A graph with no path. A path that needs every edge reversed. A shortcut that costs more edges but fewer violations.

LIKELY FOLLOW-UPS

General edge costs (Dijkstra), shortest path itself (parent pointers), or negative costs (Bellman-Ford).

PITFALLS

Using visited flags instead of a distance check. Pushing weight-0 edges to the back.

HOW TO THINK ABOUT IT

Going against an edge costs 1 and going with it costs 0, so it is a shortest-path problem with costs of only 0 and 1. Plain BFS counts edges, not violations, so it is the wrong tool. A deque lets 0-cost moves jump the queue (push front) and 1-cost moves wait their turn (push back), which is Dijkstra without the heap.

Basis: Related to LeetCode 1466, Reorder Routes: a shortest path where traveling an edge backwards costs 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, start, end):
    arcs = []
    for u, v in edges:
        arcs.append((u, v, 0))  # along the edge
        arcs.append((v, u, 1))  # against it
    dist = [INF] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[end]
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Every round scans all edges, V - 1 rounds: O(V * E). Costs of only 0 and 1 allow a deque search (0-1 BFS) in O(V + E).

OPTIMAL SOLUTION · 0-1 BFS on the doubled graph

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is 0-1 BFS correct, and how does it compare with Dijkstra?

The deque always holds nodes whose distances differ by at most one: front elements are at distance d, back elements at d + 1. Relaxing a zero-weight edge keeps the distance at d (front), a one-weight edge makes d + 1 (back). That is the same order a heap would produce, so it is Dijkstra without the log factor: O(n + m) versus O((n + m) log n).

FOLLOW-UP 2 What if edge costs can be arbitrary non-negative numbers?

Use Dijkstra with a heap on the same doubled graph (forward cost = 0 or the edge cost, backward cost = the violation cost). With negative costs use Bellman-Ford and watch for negative cycles.

FOLLOW-UP 3 Edge costs are arbitrary non-negative numbers (different for each direction).

Use Dijkstra with a heap on the same doubled graph, giving each arc its own cost. 0-1 BFS only works because the costs are 0 or 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimum Direction Violations in a
# Directed Graph
#
# Examples: n = 5 edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
# start = 0 end = 4 path: 0 -> 1 -> 2 -> 3 -> 4 cost: 0 +
# 1 + 0 + 1 = 2 output: 2
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is 0-1 BFS correct, and how does it compare with
# Dijkstra?
# 2. What if edge costs can be arbitrary non-negative
# numbers?
# 3. Edge costs are arbitrary non-negative numbers
# (different for each direction).
#
# INTERVIEW NOTES
# Going against an edge costs 1 and going with it costs 0,
# so it is a shortest-path problem with costs of only 0
# and 1. Plain BFS counts edges, not violations, so it is
# the wrong tool. A deque lets 0-cost moves jump the queue
# (push front) and 1-cost moves wait their turn (push
# back), which is Dijkstra without the heap.
#
# BRUTE FORCE (say it first): Bellman-Ford on the doubled
# graph
# Give every edge a forward copy with cost 0 and a
# backward copy with cost 1, and relax all edges V - 1
# times. Correct and simple, and the oracle for the 0-1
# BFS; it is slower because it repeats full passes over
# the edges. Time O(V * E). Space O(V).


import heapq
from collections import deque


# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Bellman-Ford on the doubled
# graph | time O(V * E)
def violations_bellman_ford(n, edges, start, end):
    INF = float("inf")
    dist = [INF] * n
    dist[start] = 0
    arcs = [(u, v, 0) for u, v in edges] + [
        (v, u, 1) for u, v in edges
    ]
    for _ in range(max(0, n - 1)):
        changed = False
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    return -1 if dist[end] == INF else dist[end]


# BASE QUESTION (optimal): 0-1 BFS on the doubled graph
# Idea: Add every edge twice: forward with weight 0,
# backward with weight 1. Shortest path with weights in
# {0, 1} is a deque BFS: push a zero-weight neighbour to
# the front and a one-weight neighbour to the back.
def min_violations(n, edges, start, end):
    # 0-1 BFS: following an edge is free, reversing costs
    # 1
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # cost 0 goes to the front, cost 1 to the
                # back
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return -1 if dist[end] == INF else dist[end]


# FOLLOW-UP 3: Edge costs are arbitrary non-negative
# numbers (different for each direction).
# Answer: Use Dijkstra with a heap on the same doubled
# graph, giving each arc its own cost. 0-1 BFS only works
# because the costs are 0 or 1.
def cheapest_route(n, edges, start, end):
    # edges are (u, v, forward_cost, backward_cost)
    graph = [[] for _ in range(n)]
    for u, v, fwd, back in edges:
        graph[u].append((v, fwd))
        graph[v].append((u, back))
    dist = [INF] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
```

```
        d, u = heapq.heappop(heap)
        # skip outdated heap entries
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return -1 if dist[end] == INF else dist[end]


if __name__ == "__main__":
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    # path 0 -> 1 -> 2 -> 3 -> 4 reverses two edges
    assert min_violations(5, edges, 0, 4) == 2
    assert min_violations(5, edges, 0, 0) == 0
    assert min_violations(3, [(0, 1)], 0, 2) == -1
    # the same graph with every reversal costing 1
    costed = [(u, v, 0, 1) for u, v in edges]
    assert cheapest_route(5, costed, 0, 4) == 2
    # different cost per direction
    assert cheapest_route(2, [(0, 1, 5, 2)], 0, 1) == 5
    assert cheapest_route(2, [(0, 1, 5, 2)], 1, 0) == 2
```

NOTES FROM CANDIDATES

Represent each original edge twice: the forward direction with weight 0 and the reverse direction with weight 1.
A regular BFS minimizes the number of edges, not the number of direction violations.
The linear-time target is O(n + m) time and space.

PREPARATION

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.